

Physlib Monthly Report

1–30 April 2026

Contributors: Alex Meiburg, doxtor6, Gregory Loges, Joseph Tooby-Smith, juanjfndz, Nikolai Kashcheev, Zhi Kai Pong

Reviewers: Adomas Baliuka, Alex Meiburg, Daniel Morrison, Joseph Tooby-Smith, Juan José Fernández, Rodolfo R. Soldati, Zhi Kai Pong

Generated 14 August 2026 from commit `cd22b0c`.

Abstract

Physlib is a community library of formalised physics for the [Lean 4](#) proof assistant: definitions and results from across physics, stated in a formal language and checked by machine. This report is an automatically generated record of one month’s additions to it.

It covers 1–30 April 2026 on branch `master` of [leanprover-community/physlib](#), between commits `f2fd447` and `cd22b0c`. Over that period 7 contributors landed 37 commits, changing 21,333 lines across 543 files and adding 574 new Lean declarations, each catalogued with its statement in Section 2. The complete diff is available at [GitHub compare](#).

Contents

1	Introduction	2
1.1	Contributors	2
1.2	Reviewers	2
1.3	Modified subfolders	2
1.4	New declarations by kind	3
2	Details by file	3
2.1	Physlib.ClassicalFieldTheory.Local.Variation	3
2.2	Physlib.Cosmology.FLRW.Basic	5
2.3	Physlib.Electromagnetism.Kinematics.EMPotential	6
2.4	Physlib.Electromagnetism.Kinematics.FieldStrength	8
2.5	Physlib.Electromagnetism.Kinematics.ScalarPotential	8
2.6	Physlib.Electromagnetism.Kinematics.VectorPotential	8
2.7	Physlib.Electromagnetism.ThreeDimension.MaxwellEquations	8
2.8	Physlib.Electromagnetism.Vacuum.HarmonicWave	9
2.9	Physlib.Mathematics.DataStructures.FourTree.Basic	9
2.10	Physlib.Mathematics.DataStructures.FourTree.UniqueMap	13
2.11	Physlib.Mathematics.Distribution.Basic	15
2.12	Physlib.Mathematics.Distribution.PowMul	17
2.13	Physlib.Mathematics.Fin	17
2.14	Physlib.Mathematics.Fin.Involutions	22
2.15	Physlib.Mathematics.List	25
2.16	Physlib.Mathematics.List.InsertIdx	35
2.17	Physlib.Mathematics.List.InsertionSort	38
2.18	Physlib.Mathematics.PiTensorProduct	44

2.19	Physlib.Mathematics.RatComplexNum	47
2.20	Physlib.Mathematics.SpecialFunctions.PhysHermite	50
2.21	Physlib.Meta.AllFilePaths	56
2.22	Physlib.Meta.Basic	56
2.23	Physlib.Meta.Informal.Post	57
2.24	Physlib.Meta.Notes.Basic	58
2.25	Physlib.Meta.Notes.HTMLNote	58
2.26	Physlib.Meta.Notes.NoteFile	59
2.27	Physlib.Meta.Remark.Basic	59
2.28	Physlib.Meta.Sorry	59
2.29	Physlib.Meta.TODO.Basic	61
2.30	Physlib.Particles.SuperSymmetry.SU5.ChargeSpectrum.Basic	61
2.31	Physlib.QFT.PerturbationTheory.WickAlgebra.Basic	61
2.32	Physlib.QFT.QED.AnomalyCancellation.Odd.BasisLinear	61
2.33	Physlib.QuantumMechanics.DDimensions.Hydrogen.LaplaceRungeLenzVector	62
2.34	Physlib.QuantumMechanics.DDimensions.Operators.Commutation	63
2.35	Physlib.QuantumMechanics.DDimensions.Operators.Position	63
2.36	Physlib.QuantumMechanics.DDimensions.SpaceDHilbertSpace.PolyBddSchwartzSubmodule	66
2.37	Physlib.QuantumMechanics.DDimensions.SpaceDHilbertSpace.SchwartzSubmodule	67
2.38	Physlib.Relativity.SL2C.Basic	68
2.39	Physlib.Relativity.Tensors.Basic	68
2.40	Physlib.Relativity.Tensors.ComplexTensor.Basic	69
2.41	Physlib.Relativity.Tensors.ComplexTensor.Units.Pre	70
2.42	Physlib.Relativity.Tensors.ComplexTensor.Vector.Pre.Basic	71
2.43	Physlib.Relativity.Tensors.Constructors	71
2.44	Physlib.Relativity.Tensors.RealTensor.Basic	71
2.45	Physlib.Relativity.Tensors.RealTensor.ToComplex	72
2.46	Physlib.Relativity.Tensors.RealTensor.Vector.Pre.Basic	73
2.47	Physlib.Relativity.Tensors.RealTensor.Velocity.Basic	74
2.48	Physlib.Relativity.Tensors.TensorSpecies.Basic	74
2.49	Physlib.SpaceAndTime.Space.Derivatives.Curl	75
2.50	Physlib.SpaceAndTime.Space.Derivatives.Div	76
2.51	Physlib.SpaceAndTime.Space.Derivatives.Grad	77
2.52	Physlib.SpaceAndTime.Space.Derivatives.Iterated	77
2.53	Physlib.SpaceAndTime.Space.Derivatives.MultiIndex	79
2.54	Physlib.SpaceAndTime.Space.Module	82
2.55	Physlib.SpaceAndTime.SpaceTime.Derivatives	82
2.56	Physlib.SpaceAndTime.Time.Basic	82
2.57	Physlib.SpaceAndTime.Time.Derivatives	83
2.58	QuantumInfo.Finite.Braket	83
2.59	QuantumInfo.Finite.CPTPMap.Bundled	83
2.60	QuantumInfo.Finite.MState	84
2.61	scripts.check_file_imports	84
2.62	scripts.find_TODOs	84
2.63	scripts.mathlib_textLint_on_physlib	84
2.64	scripts.MetaPrograms.regex_lint	85
2.65	scripts.MetaPrograms.style_lint	85
2.66	scripts.MetaPrograms.TODO_to_yaml	85
2.67	scripts.type_former_lint	87

1 Introduction

During Apr 2026, 7 contributors submitted 37 commits that changed 21,333 lines (+10,272 / -11,061) across 543 files spanning 41 top-level areas of the repository. Of those, 11 files were newly added and 10 were removed outright. This report catalogues 574 newly added Lean declarations: 379 lemmas, 121 defs, 46 instances, 11 structures, 6 theorems, 6 inductives, and 5 abbrevs. We're delighted to recognize the following first-time contributors this month: [doxtor6](#) and [juanjfndz](#).

See the [full compare diff](#) and the [list of commits](#) on GitHub for the raw source.

1.1 Contributors

The following individuals authored commits merged during this reporting period, listed alphabetically.

Contributor	Lines Changed	Commits
Alex Meiburg	0	1
doxtor6	120	1
Gregory Loges	2,501	10
Joseph Tooby-Smith	16,660	17
juanjfndz	527	4
Nikolai Kashcheev	441	1
Zhi Kai Pong	442	3

1.2 Reviewers

The following individuals approved pull requests during this reporting period, listed alphabetically.

Reviewer	PRs Reviewed
Adomas Baliuka	1
Alex Meiburg	2
Daniel Morrison	2
Joseph Tooby-Smith	20
Juan José Fernández	1
Rodolfo R. Soldati	1
Zhi Kai Pong	12

1.3 Modified subfolders

Subfolder	Files	New	Deleted	Additions	Deletions
Physlib/Relativity	80	+2	—	+2,596	-1,610
docs/_data	2	—	-2	+0	-4,071
Physlib/QuantumMechanics	30	+3	—	+1,527	-226
Physlib/SpaceAndTime	33	+3	—	+1,494	-147
Physlib/QFT	72	—	—	+699	-485
PhysLean/QuantumMechanics	2	—	-2	+0	-1,030
scripts/MetaPrograms	15	—	—	+489	-487
Physlib/Electromagnetism	24	+1	—	+512	-333

QuantumInfo/Finite	31	—	—	+571	-207
QuantumInfo/ForMathlib	34	—	—	+537	-209
PhysLean/Relativity	3	—	-3	+0	-557
Physlib/Particles	64	—	—	+291	-173
Physlib.lean	1	+1	—	+414	-0
PhysLean.lean	1	—	-1	+0	-410
PhysLean/SpaceAndTime	1	—	-1	+0	-383
Physlib/Mathematics	33	—	—	+191	-109
scripts	14	—	—	+112	-113
QuantumInfo/ClassicalInfo	6	—	—	+135	-73
Physlib/Meta	15	—	—	+93	-90
Physlib/ClassicalMechanics	18	—	—	+80	-51
Physlib/ClassicalFieldTheory	1	+1	—	+126	-0
lake-manifest.json	1	—	—	+67	-57
Physlib/Cosmology	2	—	—	+83	-28
Physlib/Units	14	—	—	+50	-39
README.md	1	—	—	+29	-31
Physlib/StringTheory	13	—	—	+28	-29
QuantumInfo/StatMech	3	—	—	+34	-19
QuantumInfo.lean	1	—	—	+25	-23
QuantumInfo	3	—	—	+23	-15
Physlib/StatisticalMechanics	5	—	—	+16	-12
lakefile.lean	1	—	—	+16	-8
.github/workflows	2	—	—	+7	-8
.github/ISSUE_TEMPLATE	2	—	—	+6	-6
CONTRIBUTING.md	1	—	—	+4	-5
Physlib/CondensedMatter	2	—	—	+5	-4
Physlib/Thermodynamics	4	—	—	+5	-4
docs	3	—	—	+3	-3
.github	1	—	-1	+0	-3
Physlib/Optics	2	—	—	+1	-2
QuantumInfo/InfiniteDim	1	—	—	+2	-0
lean-toolchain	1	—	—	+1	-1

1.4 New declarations by kind

Kind	Count
lemma	379
def	121
instance	46
structure	11
theorem	6
inductive	6
abbrev	5

2 Details by file

2.1 Physlib.ClassicalFieldTheory.Local.Variation

[Physlib/ClassicalFieldTheory/Local/Variation.lean](#) on GitHub.

structure AdmissibleVariation[\[source\]](#)

An admissible local variation is a smooth compactly supported map $\text{Space } d \rightarrow U$.

```
structure AdmissibleVariation (d : ℕ) (U : Type*) [NormedAddCommGroup U] [NormedSpace ℝ U]
  where
  toFun : Space d → U
  isTestFunction : IsTestFunction toFun
```

instance CoeFun (AdmissibleVariation d U) (fun _ => Space d → U)[\[source\]](#)

```
instance : CoeFun (AdmissibleVariation d U) (fun _ => Space d → U) where
```

lemma hasCompactSupport[\[source\]](#)

An admissible variation has compact support.

```
lemma hasCompactSupport (η : AdmissibleVariation d U) : HasCompactSupport (η.toFun)
```

def toCompactlySupportedContinuousMap[\[source\]](#)

View an admissible variation as a compactly supported continuous map.

```
noncomputable def toCompactlySupportedContinuousMap (η : AdmissibleVariation d U) :
  CompactlySupportedContinuousMap (Space d) U
```

instance Zero (AdmissibleVariation d U)[\[source\]](#)

```
noncomputable instance : Zero (AdmissibleVariation d U) where
```

lemma zero_apply[\[source\]](#)

```
@[simp]
lemma zero_apply (x : Space d) : (0 : AdmissibleVariation d U) x = 0
```

instance Neg (AdmissibleVariation d U)[\[source\]](#)

```
noncomputable instance : Neg (AdmissibleVariation d U) where
```

lemma neg_apply[\[source\]](#)

```
@[simp]
lemma neg_apply (η : AdmissibleVariation d U) (x : Space d) : (-η) x = -η x
```

instance Add (AdmissibleVariation d U)[\[source\]](#)

```
noncomputable instance : Add (AdmissibleVariation d U) where
```

lemma add_apply[\[source\]](#)

```
@[simp]
lemma add_apply (η ξ : AdmissibleVariation d U) (x : Space d) : (η + ξ) x = η x + ξ x
```

instance Sub (AdmissibleVariation d U)

[\[source\]](#)

noncomputable instance : Sub (AdmissibleVariation d U) **where**

lemma sub_apply

[\[source\]](#)

@[simp]

lemma sub_apply (η ξ : AdmissibleVariation d U) (x : Space d) : (η - ξ) x = η x - ξ x

instance SMul ℝ (AdmissibleVariation d U)

[\[source\]](#)

noncomputable instance : SMul ℝ (AdmissibleVariation d U) **where**

lemma smul_apply

[\[source\]](#)

@[simp]

lemma smul_apply (c : ℝ) (η : AdmissibleVariation d U) (x : Space d) :
(c • η) x = c • η x

lemma coord

[\[source\]](#)

@[fun_prop]

lemma coord {m : ℕ} (η : AdmissibleVariation d (Space m)) (a : Fin m) :
IsTestFunction (fun x => (η.toFun x).coord a)

2.2 Physlib.Cosmology.FLRW.Basic

[Physlib/Cosmology/FLRW/Basic.lean](#) on GitHub.

lemma hubbleConstant_ne_zero

[\[source\]](#)

The Hubble constant is nonzero whenever $\partial_t a$ and a are both nonzero.

lemma hubbleConstant_ne_zero {a : Time → ℝ} {t : Time}
(hd_az : $\partial_t a \neq 0$) (haz : $a \neq 0$) :
hubbleConstant a t $\neq 0$

lemma deriv_hubbleConstant

[\[source\]](#)

Quotient-rule expression for the time derivative of the Hubble constant: $d_t H = \frac{a''}{a - (a')^2} / a^2$.

lemma deriv_hubbleConstant {a : Time → ℝ} {t : Time}
(ha : DifferentiableAt ℝ a t) (hd_a : DifferentiableAt ℝ (∂_t a) t)
(haz : $a \neq 0$) :
∂_t (hubbleConstant a) t =
(∂_t (∂_t a) t * a t - (∂_t a t) ^ 2) / (a t) ^ 2

lemma deriv_hubbleConstant_eq_neg_sq_mul

[\[source\]](#)

The time derivative of the Hubble constant equals $-H^2 (1 + q)$.

```

lemma deriv_hubbleConstant_eq_neg_sq_mul
  {a : Time → ℝ} {t : Time}
  (ha : DifferentiableAt ℝ a t) (hd_a : DifferentiableAt ℝ (∂ₜ a) t)
  (haz : a t ≠ 0) (hd_az : ∂ₜ a t ≠ 0) :
  ∂ₜ (hubbleConstant a) t =
    -(hubbleConstant a t) ^ 2 * (1 + decelerationParameter a t)

```

lemma deriv_hubbleConstant_neg_iff

[\[source\]](#)

Pointwise: $\partial_t H t < 0$ iff $-1 < q t$ (assuming a is twice differentiable at t and both $a t$ and $\partial_t a t$ are nonzero).

```

lemma deriv_hubbleConstant_neg_iff
  {a : Time → ℝ} {t : Time}
  (ha : DifferentiableAt ℝ a t) (hd_a : DifferentiableAt ℝ (∂ₜ a) t)
  (haz : a t ≠ 0) (hd_az : ∂ₜ a t ≠ 0) :
  ∂ₜ (hubbleConstant a) t < 0 ↔ -1 < decelerationParameter a t

```

lemma exists_deriv_hubbleConstant_neg_iff

[\[source\]](#)

There exists a time at which $\partial_t H < 0$ iff there exists a time with $q > -1$.

(The corresponding informal statement was written with $q < -1$. Since $d_t H = -H^2 (1 + q)$ and $H \neq 0$, one has $d_t H < 0 \leftrightarrow q > -1$, so the formal statement uses the corrected inequality $-1 < q$.)

```

lemma exists_deriv_hubbleConstant_neg_iff
  {a : Time → ℝ}
  (ha : ∀ t, DifferentiableAt ℝ a t) (hd_a : ∀ t, DifferentiableAt ℝ (∂ₜ a) t)
  (haz : ∀ t, a t ≠ 0) (hd_az : ∀ t, ∂ₜ a t ≠ 0) :
  (∃ t, ∂ₜ (hubbleConstant a) t < 0) ↔ (∃ t, -1 < decelerationParameter a t)

```

2.3 Physlib.Electromagnetism.Kinematics.EMPotential

[Physlib/Electromagnetism/Kinematics/EMPotential.lean](#) on GitHub.

lemma eq_of_val_eq

[\[source\]](#)

```

@[ext]
lemma eq_of_val_eq (A B : ElectromagneticPotential d) (h : A.val = B.val) : A = B

```

instance CoeFun (ElectromagneticPotential d) (fun _ => SpaceTime d → Lorentz.Vec... [\[source\]](#)

```

instance {d} : CoeFun (ElectromagneticPotential d)
  (fun _ => SpaceTime d → Lorentz.Vector d) where

```

instance Add (ElectromagneticPotential d)

[\[source\]](#)

```

instance {d} : Add (ElectromagneticPotential d) where

```

lemma add_val

[\[source\]](#)

```
@[simp]
lemma add_val {d} (A B : ElectromagneticPotential d) :
  (A + B).val = A.val + B.val
```

lemma add_apply[\[source\]](#)

```
lemma add_apply {d} (A B : ElectromagneticPotential d) (x : SpaceTime d) :
  (A + B) x = A x + B x
```

instance SMul $\mathbb{R}$ (ElectromagneticPotential d)[\[source\]](#)

```
noncomputable instance {d} : SMul  $\mathbb{R}$  (ElectromagneticPotential d) where
```

lemma smul_val[\[source\]](#)

```
@[simp]
lemma smul_val {d} (r :  $\mathbb{R}$ ) (A : ElectromagneticPotential d) :
  (r • A).val = r • A.val
```

lemma smul_apply[\[source\]](#)

```
lemma smul_apply {d} (r :  $\mathbb{R}$ ) (A : ElectromagneticPotential d) (x : SpaceTime d) :
  (r • A) x = r • A x
```

instance SMul (LorentzGroup d) (ElectromagneticPotential d)[\[source\]](#)

```
noncomputable instance {d} : SMul (LorentzGroup d) (ElectromagneticPotential d) where
```

lemma action_val[\[source\]](#)

```
lemma action_val {d} ( $\Lambda$  : LorentzGroup d) (A : ElectromagneticPotential d) :
  ( $\Lambda$  • A).val = fun x =>  $\Lambda$  • A ( $\Lambda^{-1}$  • x)
```

instance MulAction (LorentzGroup d) (ElectromagneticPotential d)[\[source\]](#)

```
noncomputable instance {d} : MulAction (LorentzGroup d) (ElectromagneticPotential d) where
```

lemma differentiable_action[\[source\]](#)

```
@[fun_prop]
lemma differentiable_action {d} ( $\Lambda$  : LorentzGroup d) (A : ElectromagneticPotential d)
  (hA : Differentiable  $\mathbb{R}$  A) : Differentiable  $\mathbb{R}$  (fun x =>  $\Lambda$  • A ( $\Lambda^{-1}$  • x))
```

lemma deriv_eq_tensorDeriv[\[source\]](#)

```
lemma deriv_eq_tensorDeriv {d} (A : ElectromagneticPotential d)
  (hA : Differentiable  $\mathbb{R}$  A) (x : SpaceTime d) :
  A.deriv x = tensorDeriv A.val x
```


lemma tensorDeriv_eval_eq[\[source\]](#)*Evaluation of the tensor components of $\partial_\mu A \times v$.*

```

lemma tensorDeriv_eval_eq {d} {A : ElectromagneticPotential d} (hA : Differentiable ℝ A)
  (x : SpaceTime d) (μ v : Fin 1 ⊗ Fin d) :
  toField {tensorDeriv A.val x | [μ] [v]}T = ∂_ μ A x v

```

2.4 Physlib.Electromagnetism.Kinematics.FieldStrength[Physlib/Electromagnetism/Kinematics/FieldStrength.lean](#) on GitHub.**lemma toFieldStrength_eq_deriv**[\[source\]](#)

```

lemma toFieldStrength_eq_deriv {d} (A : ElectromagneticPotential d) (x : SpaceTime d) :
  toFieldStrength A x =
  Tensorial.toTensor.symm (permT id PermCond.auto {(η d | μ μ' ⊗ A.deriv x | μ' v)
    + - (η d | v v' ⊗ A.deriv x | v' μ)}T)

```

lemma toFieldStrength_eq_tensorDeriv[\[source\]](#)

```

lemma toFieldStrength_eq_tensorDeriv {d} {A : ElectromagneticPotential d}
  (hA : Differentiable ℝ A) (x : SpaceTime d) :
  toFieldStrength A x =
  Tensorial.toTensor.symm (permT id PermCond.auto {(η d | μ μ' ⊗ tensorDeriv A x | μ' v)
    + - (η d | v v' ⊗ tensorDeriv A x | v' μ)}T)

```

2.5 Physlib.Electromagnetism.Kinematics.ScalarPotential[Physlib/Electromagnetism/Kinematics/ScalarPotential.lean](#) on GitHub.**lemma scalarPotential_contDiff_space_of_smooth**[\[source\]](#)

```

@[fun_prop]
lemma scalarPotential_contDiff_space_of_smooth {n : ℕ} {d} (c : SpeedOfLight)
  (A : ElectromagneticPotential d)
  (hA : ContDiff ℝ ∞ A) (t : Time) : ContDiff ℝ n (A.scalarPotential c t)

```

2.6 Physlib.Electromagnetism.Kinematics.VectorPotential[Physlib/Electromagnetism/Kinematics/VectorPotential.lean](#) on GitHub.**lemma vectorPotential_contDiff_of_smooth**[\[source\]](#)

```

@[fun_prop]
lemma vectorPotential_contDiff_of_smooth {n : ℕ} {d} {c : SpeedOfLight}
  (A : ElectromagneticPotential d) (hA : ContDiff ℝ ∞ A) :
  ContDiff ℝ n (A.vectorPotential c)

```

2.7 Physlib.Electromagnetism.ThreeDimension.MaxwellEquations[Physlib/Electromagnetism/ThreeDimension/MaxwellEquations.lean](#) on GitHub.

theorem gaussLawElectric[\[source\]](#)*Gauss's law for the electric field.*

```

theorem gaussLawElectric (t : Time) (x : Space)
  (h : IsExtrema  $\mathcal{F}$  V J4) (hV : ContDiff  $\mathbb{R} \infty$  V) (hJ : ContDiff  $\mathbb{R} \infty$  J4) :
  ( $\nabla \cdot E$  t) x =  $\rho$  t x /  $\epsilon_0$ 

```

theorem gaussLawMagnetic[\[source\]](#)*Gauss's law for the magnetic field.*

```

theorem gaussLawMagnetic (t : Time) (x : Space) (hV : ContDiff  $\mathbb{R} \infty$  V) :
  ( $\nabla \cdot B$  t) x = 0

```

theorem ampereLaw[\[source\]](#)*Ampère's law.*

```

theorem ampereLaw (t : Time) (x : Space)
  (h : IsExtrema  $\mathcal{F}$  V J4) (hV : ContDiff  $\mathbb{R} \infty$  V) (hJ : ContDiff  $\mathbb{R} \infty$  J4) :
  ( $\nabla \times B$  t) x =  $\mu_0 \cdot J$  t x +  $\mu_0 \cdot \epsilon_0 \cdot \partial_t$  (fun t => E t x) t

```

theorem faradayLaw[\[source\]](#)*Faraday's law.*

```

theorem faradayLaw (t : Time) (x : Space) (hV : ContDiff  $\mathbb{R} \infty$  V) :
  ( $\nabla \times E$  t) x = -  $\partial_t$  (fun t => B t x) t

```

2.8 Physlib.Electromagnetism.Vacuum.HarmonicWave[Physlib/Electromagnetism/Vacuum/HarmonicWave.lean](#) on GitHub.**lemma harmonicWaveX_inl_zero**[\[source\]](#)

```

@[simp]
lemma harmonicWaveX_inl_zero {d} ( $\mathcal{F}$  : FreeSpace) (k :  $\mathbb{R}$ ) (E0 : Fin d →  $\mathbb{R}$ ) (φ : Fin d →  $\mathbb{R}$ )
  (x : SpaceTime d.succ) :
  harmonicWaveX  $\mathcal{F}$  k E0 φ x (Sum.inl 0) = 0

```

lemma harmonicWaveX_inr_zero[\[source\]](#)

```

@[simp]
lemma harmonicWaveX_inr_zero {d} ( $\mathcal{F}$  : FreeSpace) (k :  $\mathbb{R}$ ) (E0 : Fin d →  $\mathbb{R}$ ) (φ : Fin d →  $\mathbb{R}$ )
  (x : SpaceTime d.succ) :
  harmonicWaveX  $\mathcal{F}$  k E0 φ x (Sum.inr 0) = 0

```

2.9 Physlib.Mathematics.DataStructures.FourTree.Basic[Physlib/Mathematics/DataStructures/FourTree/Basic.lean](#) on GitHub.

inductive Leaf[\[source\]](#)

A leaf contains has the data of a term of type α_4 .

```
inductive Leaf ( $\alpha_4$  : Type)
| leaf :  $\alpha_4 \rightarrow$  Leaf  $\alpha_4$ 
```

inductive Twig[\[source\]](#)

A twig has the data of a term of type α_3 and a multiset of type Leaf α_4 .

```
inductive Twig ( $\alpha_3$   $\alpha_4$  : Type)
| twig :  $\alpha_3 \rightarrow$  Multiset (Leaf  $\alpha_4$ )  $\rightarrow$  Twig  $\alpha_3$   $\alpha_4$ 
```

inductive Branch[\[source\]](#)

A branch has the data of a term of type α_2 and a multiset of type Twig α_3 α_4 .

```
inductive Branch ( $\alpha_2$   $\alpha_3$   $\alpha_4$  : Type)
| branch :  $\alpha_2 \rightarrow$  Multiset (Twig  $\alpha_3$   $\alpha_4$ )  $\rightarrow$  Branch  $\alpha_2$   $\alpha_3$   $\alpha_4$ 
```

inductive Trunk[\[source\]](#)

A trunk has the data of a term of type α_1 and a multiset of type Branch α_2 α_3 α_4 .

```
inductive Trunk ( $\alpha_1$   $\alpha_2$   $\alpha_3$   $\alpha_4$  : Type)
| trunk :  $\alpha_1 \rightarrow$  Multiset (Branch  $\alpha_2$   $\alpha_3$   $\alpha_4$ )  $\rightarrow$  Trunk  $\alpha_1$   $\alpha_2$   $\alpha_3$   $\alpha_4$ 
```

inductive FourTree[\[source\]](#)

A FourTree has the data of a multiset of type Trunk α_1 α_2 α_3 α_4 .

```
inductive FourTree ( $\alpha_1$   $\alpha_2$   $\alpha_3$   $\alpha_4$  : Type)
| root : Multiset (FourTree.Trunk  $\alpha_1$   $\alpha_2$   $\alpha_3$   $\alpha_4$ )  $\rightarrow$  FourTree  $\alpha_1$   $\alpha_2$   $\alpha_3$   $\alpha_4$ 
```

instance Repr (Leaf α_4)[\[source\]](#)

```
unsafe instance ( $\alpha_4$  : Type) [Repr  $\alpha_4$ ] : Repr (Leaf  $\alpha_4$ ) where
```

instance Repr (Twig α_3 α_4)[\[source\]](#)

```
unsafe instance ( $\alpha_3$   $\alpha_4$  : Type) [Repr  $\alpha_3$ ] [Repr  $\alpha_4$ ] : Repr (Twig  $\alpha_3$   $\alpha_4$ ) where
```

instance Repr (Branch α_2 α_3 α_4)[\[source\]](#)

```
unsafe instance ( $\alpha_2$   $\alpha_3$   $\alpha_4$ : Type) [Repr  $\alpha_2$ ] [Repr  $\alpha_3$ ] [Repr  $\alpha_4$ ] :
  Repr (Branch  $\alpha_2$   $\alpha_3$   $\alpha_4$ ) where
```

instance Repr (Trunk α_1 α_2 α_3 α_4)[\[source\]](#)

```
unsafe instance ( $\alpha_1$   $\alpha_2$   $\alpha_3$   $\alpha_4$ : Type) [Repr  $\alpha_1$ ] [Repr  $\alpha_2$ ] [Repr  $\alpha_3$ ] [Repr  $\alpha_4$ ] :
  Repr (Trunk  $\alpha_1$   $\alpha_2$   $\alpha_3$   $\alpha_4$ ) where
```

instance Repr (FourTree α 1 α 2 α 3 α 4)[\[source\]](#)

```
unsafe instance ( $\alpha$ 1  $\alpha$ 2  $\alpha$ 3  $\alpha$ 4 : Type) [Repr  $\alpha$ 1] [Repr  $\alpha$ 2] [Repr  $\alpha$ 3] [Repr  $\alpha$ 4] :
  Repr (FourTree  $\alpha$ 1  $\alpha$ 2  $\alpha$ 3  $\alpha$ 4) where
```

def fromMultiset[\[source\]](#)

A FourTree from a multiset of α 1 $\times$ α 2 $\times$ α 3 $\times$ α 4.

```
def fromMultiset { $\alpha$ 1  $\alpha$ 2  $\alpha$ 3  $\alpha$ 4 : Type} [DecidableEq  $\alpha$ 1]
  [DecidableEq  $\alpha$ 2] [DecidableEq  $\alpha$ 3] [DecidableEq  $\alpha$ 4]
  (l : Multiset ( $\alpha$ 1  $\times$   $\alpha$ 2  $\times$   $\alpha$ 3  $\times$   $\alpha$ 4)) : FourTree  $\alpha$ 1  $\alpha$ 2  $\alpha$ 3  $\alpha$ 4
```

def toMultiset[\[source\]](#)

A FourTree to a multiset of α 1 $\times$ α 2 $\times$ α 3 $\times$ α 4.

```
def toMultiset { $\alpha$ 1  $\alpha$ 2  $\alpha$ 3  $\alpha$ 4 : Type} (T : FourTree  $\alpha$ 1  $\alpha$ 2  $\alpha$ 3  $\alpha$ 4) : Multiset ( $\alpha$ 1  $\times$   $\alpha$ 2  $\times$   $\alpha$ 3  $\times$   $\alpha$ 4)
```

def Twig.card[\[source\]](#)

The cardinality of a Twig is the number of leafs.

```
def Twig.card { $\alpha$ 3  $\alpha$ 4 : Type} (T : Twig  $\alpha$ 3  $\alpha$ 4) : Nat
```

def Branch.card[\[source\]](#)

The cardinality of a Branch is the total number of leafs.

```
def Branch.card { $\alpha$ 2  $\alpha$ 3  $\alpha$ 4 : Type} (T : Branch  $\alpha$ 2  $\alpha$ 3  $\alpha$ 4) : Nat
```

def Trunk.card[\[source\]](#)

The cardinality of a Trunk is the total number of leafs.

```
def Trunk.card { $\alpha$ 1  $\alpha$ 2  $\alpha$ 3  $\alpha$ 4 : Type} (T : Trunk  $\alpha$ 1  $\alpha$ 2  $\alpha$ 3  $\alpha$ 4) : Nat
```

def FourTree.card[\[source\]](#)

The cardinality of a FourTree is the total number of leafs.

```
def card { $\alpha$ 1  $\alpha$ 2  $\alpha$ 3  $\alpha$ 4 : Type} (T : FourTree  $\alpha$ 1  $\alpha$ 2  $\alpha$ 3  $\alpha$ 4) : Nat
```

lemma card_eq_toMultiset_card[\[source\]](#)

```
lemma card_eq_toMultiset_card (T : FourTree  $\alpha$ 1  $\alpha$ 2  $\alpha$ 3  $\alpha$ 4s) :
  T.card = T.toMultiset.card
```

def Leaf.mem[\[source\]](#)

An element of $a : \alpha$ 4 is a member of Leaf α 4 if the underlying element of the Leaf is a .

```
def Leaf.mem { $\alpha$ 4} (T : Leaf  $\alpha$ 4) (x :  $\alpha$ 4) : Prop
```

instance Decidable (T.mem x)

[\[source\]](#)

```
instance {α4} [DecidableEq α4] (T : Leaf α4) (x : α4) : Decidable (T.mem x)
```

def Twig.mem

[\[source\]](#)

An element of $a : \alpha_3 \times \alpha_4$ is a member of $\text{Twig } \alpha_3 \alpha_4$ if the underlying α_3 element of the Twig is $a.1$ and $a.2$ is a member of one of the Leaf .

```
def Twig.mem (T : Twig α3 α4) (x : α3 × α4) : Prop
```

instance Decidable (T.mem x)

[\[source\]](#)

```
instance {α3 α4} [DecidableEq α3] [DecidableEq α4] (T : Twig α3 α4) (x : α3 × α4) :  
  Decidable (T.mem x)
```

def Branch.mem

[\[source\]](#)

An element of $a : \alpha_2 \times \alpha_3 \times \alpha_4$ is a member of $\text{Branch } \alpha_2 \alpha_3 \alpha_4$ if the underlying α_2 element of the Branch is $a.1$ and $a.2$ is a member of one of the Twig .

```
def Branch.mem (T : Branch α2 α3 α4) (x : α2 × α3 × α4) : Prop
```

instance Decidable (T.mem x)

[\[source\]](#)

```
instance [DecidableEq α2] [DecidableEq α3] [DecidableEq α4] (T : Branch α2 α3 α4)  
  (x : α2 × α3 × α4) : Decidable (T.mem x)
```

def Trunk.mem

[\[source\]](#)

An element of $a : \alpha_1 \times \alpha_2 \times \alpha_3 \times \alpha_4$ is a member of $\text{Trunk } \alpha_1 \alpha_2 \alpha_3 \alpha_4$ if the underlying α_1 element of the Trunk is $a.1$ and $a.2$ is a member of one of the Branch .

```
def Trunk.mem (T : Trunk α1 α2 α3 α4) (x : α1 × α2 × α3 × α4) : Prop
```

instance Decidable (T.mem x)

[\[source\]](#)

```
instance [DecidableEq α1] [DecidableEq α2] [DecidableEq α3] [DecidableEq α4]  
  (T : Trunk α1 α2 α3 α4) (x : α1 × α2 × α3 × α4) : Decidable (T.mem x)
```

def FourTree.mem

[\[source\]](#)

An element of $a : \alpha_1 \times \alpha_2 \times \alpha_3 \times \alpha_4$ is a member of $\text{FourTree } \alpha_1 \alpha_2 \alpha_3 \alpha_4$ if a is a member of one of the Trunk .

```
def mem (T : FourTree α1 α2 α3 α4) (x : α1 × α2 × α3 × α4) : Prop
```

instance Decidable (T.mem x)

[\[source\]](#)

```
instance [DecidableEq α1] [DecidableEq α2] [DecidableEq α3] [DecidableEq α4]  
  (T : FourTree α1 α2 α3 α4) (x : α1 × α2 × α3 × α4) : Decidable (T.mem x)
```

instance Membership ($\alpha1 \times \alpha2 \times \alpha3 \times \alpha4$) (FourTree $\alpha1 \alpha2 \alpha3 \alpha4$)

[\[source\]](#)

instance : Membership ($\alpha1 \times \alpha2 \times \alpha3 \times \alpha4$) (FourTree $\alpha1 \alpha2 \alpha3 \alpha4$) **where**

instance Decidable ($x \in T$)

[\[source\]](#)

instance [DecidableEq $\alpha1$] [DecidableEq $\alpha2$] [DecidableEq $\alpha3$] [DecidableEq $\alpha4$]
(T : FourTree $\alpha1 \alpha2 \alpha3 \alpha4$) (x : $\alpha1 \times \alpha2 \times \alpha3 \times \alpha4$) : Decidable ($x \in T$)

lemma mem_iff_mem_toMultiset

[\[source\]](#)

lemma mem_iff_mem_toMultiset (T : FourTree $\alpha1 \alpha2 \alpha3 \alpha4$) (x : $\alpha1 \times \alpha2 \times \alpha3 \times \alpha4$) :
 $x \in T \leftrightarrow x \in T.toMultiset$

lemma mem_of_parts

[\[source\]](#)

lemma mem_of_parts {T : FourTree $\alpha1 \alpha2 \alpha3 \alpha4$ } {C : $\alpha1 \times \alpha2 \times \alpha3 \times \alpha4$ }
(trunk : Trunk $\alpha1 \alpha2 \alpha3 \alpha4$)
(branch : Branch $\alpha2 \alpha3 \alpha4$)
(twig : Twig $\alpha3 \alpha4$) (leaf : Leaf $\alpha4$)
(trunk_mem : trunk $\in T.1$) (branch_mem : branch $\in$ trunk.2)
(twig_mem : twig $\in$ branch.2) (leaf_mem : leaf $\in$ twig.2)
(heq : C = (trunk.1, branch.1, twig.1, leaf.1)) :
C $\in T$

2.10 Physlib.Mathematics.DataStructures.FourTree.UniqueMap

[Physlib/Mathematics/DataStructures/FourTree/UniqueMap.lean](#) on GitHub.

def Leaf.uniqueMap4

[\[source\]](#)

Given a map $f : \alpha4 \rightarrow \alpha4$ the map from Leaf $\alpha4 \rightarrow$ Leaf $\alpha4$ mapping the underlying elements.

def Leaf.uniqueMap4 : Leaf $\alpha4 \rightarrow$ Leaf $\alpha4$
| .leaf x => .leaf (f x)

def Twig.uniqueMap4

[\[source\]](#)

Given a map $f : \alpha4 \rightarrow \alpha4$ the map from Twig $\alpha3 \alpha4 \rightarrow$ Twig $\alpha3 \alpha4$ mapping the underlying leafs and deleting any that appear in the original Twig.

def Twig.uniqueMap4 (T : Twig $\alpha3 \alpha4$) : Twig $\alpha3 \alpha4$

def Branch.uniqueMap4

[\[source\]](#)

Given a map $f : \alpha4 \rightarrow \alpha4$ the map from Branch $\alpha2 \alpha3 \alpha4 \rightarrow$ Branch $\alpha2 \alpha3 \alpha4$ mapping the underlying leafs and deleting any that appear in the original Twig.

def Branch.uniqueMap4 (T : Branch $\alpha2 \alpha3 \alpha4$) :
Branch $\alpha2 \alpha3 \alpha4$

def Trunk.uniqueMap4[\[source\]](#)

Given a map $f : \alpha_4 \rightarrow \alpha_4$ the map from $\text{Trunk } \alpha_1 \alpha_2 \alpha_3 \alpha_4 \rightarrow \text{Trunk } \alpha_1 \alpha_2 \alpha_3 \alpha_4$ mapping the underlying leafs and deleting any that appear in the original Twig.

```
def Trunk.uniqueMap4 (T : Trunk  $\alpha_1 \alpha_2 \alpha_3 \alpha_4$ ) : Trunk  $\alpha_1 \alpha_2 \alpha_3 \alpha_4$ 
```

def FourTree.uniqueMap4[\[source\]](#)

Given a map $f : \alpha_4 \rightarrow \alpha_4$ the map from $\text{FourTree } \alpha_1 \alpha_2 \alpha_3 \alpha_4 \rightarrow \text{FourTree } \alpha_1 \alpha_2 \alpha_3 \alpha_4$ mapping the underlying leafs and deleting any that appear in the original twig of that leaf.

```
def uniqueMap4 (T : FourTree  $\alpha_1 \alpha_2 \alpha_3 \alpha_4$ ) : FourTree  $\alpha_1 \alpha_2 \alpha_3 \alpha_4$ 
```

lemma map_mem_uniqueMap4[\[source\]](#)

```
lemma map_mem_uniqueMap4 {T : FourTree  $\alpha_1 \alpha_2 \alpha_3 \alpha_4$ }  
  (x :  $\alpha_1 \times \alpha_2 \times \alpha_3 \times \alpha_4$ ) (hx : x ∈ T) (f :  $\alpha_4 \rightarrow \alpha_4$ ) :  
  (x.1, x.2.1, x.2.2.1, f x.2.2.2) ∈ T.uniqueMap4 f v  
  (x.1, x.2.1, x.2.2.1, f x.2.2.2) ∈ T
```

lemma exists_of_mem_uniqueMap4[\[source\]](#)

```
lemma exists_of_mem_uniqueMap4 {T : FourTree  $\alpha_1 \alpha_2 \alpha_3 \alpha_4$ }  
  (C :  $\alpha_1 \times \alpha_2 \times \alpha_3 \times \alpha_4$ ) (h : C ∈ T.uniqueMap4 f) :  
  ∃ qHd qHu Q5 Q10, C = (qHd, qHu, Q5, f Q10) ∧ (qHd, qHu, Q5, Q10) ∈ T
```

def Twig.uniqueMap3[\[source\]](#)

Given a map $f : \alpha_3 \rightarrow \alpha_3$ the map from $\text{Twig } \alpha_3 \alpha_4 \rightarrow \text{Twig } \alpha_3 \alpha_4$ mapping the underlying first value of the twig.

```
def Twig.uniqueMap3 (T : Twig  $\alpha_3 \alpha_4$ ) : Twig  $\alpha_3 \alpha_4$ 
```

def Branch.uniqueMap3[\[source\]](#)

Given a map $f : \alpha_3 \rightarrow \alpha_3$ the map from $\text{Branch } \alpha_2 \alpha_3 \alpha_4 \rightarrow \text{Branch } \alpha_2 \alpha_3 \alpha_4$ mapping the underlying first value of the twig, and deleting any new leafs that appeared in the old branch.

```
def Branch.uniqueMap3 (T : Branch  $\alpha_2 \alpha_3 \alpha_4$ ) : Branch  $\alpha_2 \alpha_3 \alpha_4$ 
```

def Trunk.uniqueMap3[\[source\]](#)

Given a map $f : \alpha_3 \rightarrow \alpha_3$ the map from $\text{Trunk } \alpha_1 \alpha_2 \alpha_3 \alpha_4 \rightarrow \text{Trunk } \alpha_1 \alpha_2 \alpha_3 \alpha_4$ mapping the underlying first value of the twig, and deleting any new leafs that appeared in the old branch.

```
def Trunk.uniqueMap3 (T : Trunk  $\alpha_1 \alpha_2 \alpha_3 \alpha_4$ ) : Trunk  $\alpha_1 \alpha_2 \alpha_3 \alpha_4$ 
```

def FourTree.uniqueMap3[\[source\]](#)

Given a map $f : \alpha_3 \rightarrow \alpha_3$ the map from $\text{FourTree } \alpha_1 \alpha_2 \alpha_3 \alpha_4 \rightarrow \text{FourTree } \alpha_1 \alpha_2 \alpha_3 \alpha_4$ mapping the underlying first value of the twig, and deleting any new leafs that appeared in the old branch.

```
def uniqueMap3 (T : FourTree  $\alpha_1 \alpha_2 \alpha_3 \alpha_4$ ) : FourTree  $\alpha_1 \alpha_2 \alpha_3 \alpha_4$ 
```

lemma map_mem_uniqueMap3[\[source\]](#)

```
lemma map_mem_uniqueMap3 {T : FourTree  $\alpha_1 \alpha_2 \alpha_3 \alpha_4$ }
  (x :  $\alpha_1 \times \alpha_2 \times \alpha_3 \times \alpha_4$ ) (hx : x  $\in$  T) (f :  $\alpha_3 \rightarrow \alpha_3$ ) :
  (x.1, x.2.1, f x.2.2.1, x.2.2.2)  $\in$  T.uniqueMap3 f v
  (x.1, x.2.1, f x.2.2.1, x.2.2.2)  $\in$  T
```

lemma exists_of_mem_uniqueMap3[\[source\]](#)

```
lemma exists_of_mem_uniqueMap3 {T : FourTree  $\alpha_1 \alpha_2 \alpha_3 \alpha_4$ }
  (C :  $\alpha_1 \times \alpha_2 \times \alpha_3 \times \alpha_4$ ) (h : C  $\in$  T.uniqueMap3 f) :
   $\exists$  qHd qHu Q5 Q10, C = (qHd, qHu, f Q5, Q10)  $\wedge$ 
  (qHd, qHu, Q5, Q10)  $\in$  T
```

2.11 Physlib.Mathematics.Distribution.Basic

[Physlib/Mathematics/Distribution/Basic.lean](#) on GitHub.

abbrev Distribution[\[source\]](#)

An F -valued distribution on E (where E is a normed vector space over $\mathbb{R}$ and F is a normed vector space over $\mathbb{k}$) is a continuous linear map $\mathcal{S}(E, \mathbb{k}) \rightarrow L[\mathbb{k}] F$ where $\mathcal{S}(E, \mathbb{k})$ is the Schwartz space of smooth functions $E \rightarrow \mathbb{k}$ with rapidly decreasing iterated derivatives. This is notated as $E \rightarrow d[\mathbb{k}] F$.

This should be seen as a generalisation of functions $E \rightarrow F$.

```
abbrev Distribution ( $\mathbb{k} E F$  : Type) [RCLike  $\mathbb{k}$ ] [NormedAddCommGroup E] [NormedAddCommGroup F]
  [NormedSpace  $\mathbb{R} E$ ] [NormedSpace  $\mathbb{k} F$ ] : Type
```

def ofLinear[\[source\]](#)

The construction of a distribution from the following data: 1. We take a finite set s of pairs $(k, n) \in \mathbb{N} \times \mathbb{N}$ that will be explained later. 2. We take a linear map u that evaluates the given Schwartz function η . At this stage we don't need u to be continuous. 3. Recall that a Schwartz function η satisfies a bound $\|x\|^k * \|(d^n/dx^n) \eta\| < M_{nk}$ where $M_{nk} : \mathbb{R}$ only depends on $(k, n) : \mathbb{N} \times \mathbb{N}$. 4. This step is where s is used: for each test function η , the norm $\|u \eta\|$ is required to be bounded by $C * (\|x\|^k * \|(d^n/dx^n) \eta\|)$ for some $x : \mathbb{R}$ and for some $(k, n) \in s$, where $C \geq 0$ is a global scalar.

```
def ofLinear (s : Finset ( $\mathbb{N} \times \mathbb{N}$ )) (u :  $\mathcal{S}(E, \mathbb{k}) \rightarrow_l [\mathbb{k}] F$ )
  (hu :  $\exists C : \mathbb{R}, 0 \leq C \wedge \forall \eta : \mathcal{S}(E, \mathbb{k}), \exists (k : \mathbb{N}) (n : \mathbb{N}) (x : E), (k, n) \in s \wedge$ 
     $\|u \eta\| \leq C * (\|x\| ^ k * \|\text{iteratedFDeriv } \mathbb{R} n \eta x\|))$ ) :  $E \rightarrow d[\mathbb{k}] F$ 
```


def fderivD[\[source\]](#)

The Fréchet derivative of a distribution.

Informally, for a distribution $u : E \rightarrow_d[\mathbb{k}] F$, the Fréchet derivative $fderiv\ u\ x\ v$ corresponds to the derivative of u at the point x in the direction v . For example, if $F = \mathbb{R}^3$ then $fderiv\ u\ x\ v$ is a vector in $\mathbb{R}^3$ corresponding to $(v_1\ \partial u_1/\partial x_1 + v_2\ \partial u_1/\partial x_2 + v_3\ \partial u_1/\partial x_3, v_1\ \partial u_2/\partial x_1 + v_2\ \partial u_2/\partial x_2 + v_3\ \partial u_2/\partial x_3, \dots)$.

Formally, for a distribution $u : E \rightarrow_d[\mathbb{k}] F$, this is actually defined the distribution which takes test function $\eta : E \rightarrow \mathbb{k}$ to $- u\ (\text{SchwartzMap.evalCLM } v\ (\text{SchwartzMap.fderivCLM } \mathbb{k}\ \eta))$.

Note that, unlike for functions, the Fréchet derivative of a distribution always exists.

```
def fderivD [FiniteDimensional ℝ E] : (E →d[ℝ] F) →l[ℝ] (E →d[ℝ] (E →L[ℝ] F)) where
```

lemma fderivD_apply[\[source\]](#)

```
lemma fderivD_apply [FiniteDimensional ℝ E] (u : E →d[ℝ] F) (η : S(E, ℝ)) (v : E) :
  fderivD ℝ u η v = - u (SchwartzMap.evalCLM (ℝ := ℝ) E ℝ v (SchwartzMap.fderivCLM ℝ E ℝ η))
```

def fourierTransform[\[source\]](#)

Definition of Fourier transform of distribution: Let u be a distribution. Then its Fourier transform is $F\{u\}$ where given a test function η , $F\{u\}(\eta) := u(F\{\eta\})$.

```
def fourierTransform : (E →d[ℂ] F) →l[ℂ] (E →d[ℂ] F) where
```

def const[\[source\]](#)

The constant distribution $E \rightarrow_d[\mathbb{k}] F$, for a given $c : F$ this corresponds to the integral $\int x, \eta\ x \cdot c\ \partial\text{MeasureTheory.volume}$.

```
def const [hμ : Measure.HasTemperateGrowth (volume (α := E))] (c : F) : E →d[ℝ] F
```

lemma const_apply[\[source\]](#)

```
lemma const_apply [hμ : Measure.HasTemperateGrowth (volume (α := E))] (c : F)
  (η : S(E, ℝ)) :
  const ℝ E c η = ∫ x, η x • c ∂MeasureTheory.volume
```

lemma fderivD_const[\[source\]](#)

```
@[simp]
lemma fderivD_const [hμ : Measure.IsAddHaarMeasure (volume (α := E))]
  [FiniteDimensional ℝ E] (c : F) :
  fderivD ℝ (const ℝ E c) = 0
```

def diracDelta[\[source\]](#)

Dirac delta distribution $diracDelta\ \mathbb{k}\ a : E \rightarrow_d[\mathbb{k}] \mathbb{k}$ takes in a test function $\eta : S(E, \mathbb{k})$ and outputs $\eta\ a$. Intuitively this is an infinite density at a single point a .

```
def diracDelta (a : E) : E →d[ℝ] ℝ
```

def diracDelta'[\[source\]](#)

Dirac delta in a given direction $v : F$. `diracDelta'  $\mathbb{k}$  a v` takes in a test function $\eta : \mathcal{S}(E, \mathbb{k})$ and outputs $\eta \ a \cdot v$. Intuitively this is an infinitely intense vector field at a single point a pointing at the direction v .

```
def diracDelta' (a : E) (v : F) : E →d[ $\mathbb{k}$ ] F
```

def heavisideStep[\[source\]](#)

The Heaviside step distribution defined on $(\text{EuclideanSpace } \mathbb{R} \ (\text{Fin } d.\text{succ}))$ equal to 1 in the positive z-direction and 0 in the negative z-direction.

```
def heavisideStep (d :  $\mathbb{N}$ ) : ( $\text{EuclideanSpace } \mathbb{R} \ (\text{Fin } d.\text{succ})$ ) →d[ $\mathbb{R}$ ]  $\mathbb{R}$ 
```

lemma heavisideStep_apply[\[source\]](#)

```
lemma heavisideStep_apply (d :  $\mathbb{N}$ ) ( $\eta : \mathcal{S}(\text{EuclideanSpace } \mathbb{R} \ (\text{Fin } d.\text{succ}), \mathbb{R})$ ) :  
  heavisideStep d  $\eta = \int x \text{ in } \{x : \text{EuclideanSpace } \mathbb{R} \ (\text{Fin } d.\text{succ}) \mid 0 < x \ (\text{Fin.last } d)\},$   
   $\eta \ x \ \partial \text{MeasureTheory.volume}$ 
```

2.12 Physlib.Mathematics.Distribution.PowMul

[Physlib/Mathematics/Distribution/PowMul.lean](#) on GitHub.

lemma norm_iteratedFDeriv_ofRealCLM[\[source\]](#)

```
lemma norm_iteratedFDeriv_ofRealCLM {x} (i :  $\mathbb{N}$ ) :  
  ||iteratedFDeriv  $\mathbb{R}$  i (RCLike.ofRealCLM (K :=  $\mathbb{k}$ )) x|| =  
    if i = 0 then |x| else if i = 1 then 1 else 0
```

def powOneMul[\[source\]](#)

*The continuous linear map $\mathcal{S}(\mathbb{R}, \mathbb{k}) \rightarrow L[\mathbb{k}] \mathcal{S}(\mathbb{R}, \mathbb{k})$ taking a Schwartz map η to $x * \eta$.*

```
def powOneMul :  $\mathcal{S}(\mathbb{R}, \mathbb{k}) \rightarrow L[\mathbb{k}] \mathcal{S}(\mathbb{R}, \mathbb{k})$ 
```

lemma powOneMul_apply[\[source\]](#)

```
lemma powOneMul_apply ( $\psi : \mathcal{S}(\mathbb{R}, \mathbb{k})$ ) (x :  $\mathbb{R}$ ) :  
  powOneMul  $\mathbb{k}$   $\psi \ x = x * \psi \ x$ 
```

2.13 Physlib.Mathematics.Fin

[Physlib/Mathematics/Fin.lean](#) on GitHub.

def predAboveI[\[source\]](#)

Given a i and x in $\text{Fin } n.\text{succ}.\text{succ}$ returns an element of $\text{Fin } n.\text{succ}$ subtracting 1 if $i.\text{val} \leq x.\text{val}$ else casting x .

```
def predAboveI (i x :  $\text{Fin } n.\text{succ}.\text{succ}$ ) :  $\text{Fin } n.\text{succ}$ 
```

lemma predAboveI_self[\[source\]](#)

```
lemma predAboveI_self (i : Fin n.succ.succ) : predAboveI i i = {i.val - 1, by omega}
```

lemma predAboveI_succAbove[\[source\]](#)

```
@[simp]
```

```
lemma predAboveI_succAbove (i : Fin n.succ.succ) (x : Fin n.succ) :  
  predAboveI i (Fin.succAbove i x) = x
```

lemma succsAbove_predAboveI[\[source\]](#)

```
lemma succsAbove_predAboveI {i x : Fin n.succ.succ} (h : i ≠ x) :  
  Fin.succAbove i (predAboveI i x) = x
```

lemma predAboveI_eq_iff[\[source\]](#)

```
lemma predAboveI_eq_iff {i x : Fin n.succ.succ} (h : i ≠ x) (y : Fin n.succ) :  
  y = predAboveI i x ↔ i.succAbove y = x
```

lemma predAboveI_lt[\[source\]](#)

```
lemma predAboveI_lt {i x : Fin n.succ.succ} (h : x.val < i.val) :  
  predAboveI i x = {x.val, by omega}
```

lemma predAboveI_ge[\[source\]](#)

```
lemma predAboveI_ge {i x : Fin n.succ.succ} (h : i.val < x.val) :  
  predAboveI i x = {x.val - 1, by omega}
```

lemma succAbove_succAbove_predAboveI[\[source\]](#)

```
lemma succAbove_succAbove_predAboveI (i : Fin n.succ.succ) (j : Fin n.succ) (x : Fin n) :  
  i.succAbove (j.succAbove x) =  
  (i.succAbove j).succAbove ((predAboveI (i.succAbove j) i).succAbove x)
```

def finExtractOne[\[source\]](#)

The equivalence between $\text{Fin } n.\text{succ}$ and $\text{Fin } 1 \oplus \text{Fin } n$ extracting the i th component.

```
def finExtractOne {n : ℕ} (i : Fin (n + 1)) : Fin (n + 1) ≈ Fin 1 ⊕ Fin n
```

lemma finExtractOne_apply_eq[\[source\]](#)

```
@[simp]
```

```
lemma finExtractOne_apply_eq {n : ℕ} (i : Fin n.succ) :  
  finExtractOne i i = Sum.inl 0
```

lemma finExtractOne_symm_inr[\[source\]](#)

```
lemma finExtractOne_symm_inr {n : ℕ} (i : Fin n.succ) :  
  (finExtractOne i).symm ∘ Sum.inr = i.succAbove
```

lemma finExtractOne_symm_inr_apply[\[source\]](#)

@[simp]

```
lemma finExtractOne_symm_inr_apply {n : ℕ} (i : Fin n.succ) (x : Fin n) :
  (finExtractOne i).symm (Sum.inr x) = i.succAbove x
```

lemma finExtractOne_symm_inl_apply[\[source\]](#)

@[simp]

```
lemma finExtractOne_symm_inl_apply {n : ℕ} (i : Fin n.succ) :
  (finExtractOne i).symm (Sum.inl 0) = i
```

lemma finExtractOne_apply_neq[\[source\]](#)

```
lemma finExtractOne_apply_neq {n : ℕ} (i j : Fin (n + 1 + 1)) (hij : i ≠ j) :
  finExtractOne i j = Sum.inr (predAboveI i j)
```

def finExtractOnPermHom[\[source\]](#)

Given an equivalence $\text{Fin } n.\text{succ}.\text{succ} \approx \text{Fin } n.\text{succ}.\text{succ}$, and an $i : \text{Fin } n.\text{succ}.\text{succ}$, the map $\text{Fin } n.\text{succ} \rightarrow \text{Fin } n.\text{succ}$ obtained by dropping i and its image.

```
def finExtractOnPermHom {m : ℕ} (i : Fin n.succ.succ) (σ : Fin n.succ.succ ≈ Fin m.succ.succ) :
  Fin n.succ → Fin m.succ
```

lemma finExtractOnPermHom_inv[\[source\]](#)

```
lemma finExtractOnPermHom_inv {m : ℕ} (i : Fin n.succ.succ)
  (σ : Fin n.succ.succ ≈ Fin m.succ.succ) :
  (finExtractOnPermHom (σ i) σ.symm) ∘ (finExtractOnPermHom i σ) = id
```

def finExtractOnePerm[\[source\]](#)

Given an equivalence $\text{Fin } n.\text{succ}.\text{succ} \approx \text{Fin } n.\text{succ}.\text{succ}$, and an $i : \text{Fin } n.\text{succ}.\text{succ}$, the equivalence $\text{Fin } n.\text{succ} \approx \text{Fin } n.\text{succ}$ obtained by dropping i and its image.

```
def finExtractOnePerm {m : ℕ} (i : Fin n.succ.succ) (σ : Fin n.succ.succ ≈ Fin m.succ.succ) :
  Fin n.succ ≈ Fin m.succ where
```

lemma finExtractOnePerm_equiv[\[source\]](#)

```
lemma finExtractOnePerm_equiv {n m : ℕ} (e : Fin n.succ.succ ≈ Fin m.succ.succ)
  (i : Fin n.succ.succ) :
  e ∘ i.succAbove = (e i).succAbove ∘ finExtractOnePerm i e
```

lemma finExtractOnePerm_apply[\[source\]](#)

@[simp]

```
lemma finExtractOnePerm_apply (i : Fin n.succ.succ) (σ : Fin n.succ.succ ≈ Fin n.succ.succ)
  (x : Fin n.succ) : finExtractOnePerm i σ x = predAboveI (σ i)
  (σ ((finExtractOne i).symm (Sum.inr x)))
```

lemma finExtractOnePerm_symm_apply[\[source\]](#)

```
@[simp]
lemma finExtractOnePerm_symm_apply (i : Fin n.succ.succ) (σ : Fin n.succ.succ ≈ Fin n.succ.succ)
  (x : Fin n.succ) : (finExtractOnePerm i σ).symm x = predAboveI (σ.symm (σ i))
  (σ.symm ((finExtractOne (σ i)).symm (Sum.inr x)))
```

def finExtractTwo[\[source\]](#)

The equivalence of types $\text{Fin } n.\text{succ.succ} \approx (\text{Fin } 1 \oplus \text{Fin } 1) \oplus \text{Fin } n$ extracting the i and $(i.\text{succAbove } j)$.

```
def finExtractTwo {n : ℕ} (i : Fin n.succ.succ) (j : Fin n.succ) :
  Fin n.succ.succ ≈ (Fin 1 ⊕ Fin 1) ⊕ Fin n
```

lemma finExtractTwo_apply_fst[\[source\]](#)

```
@[simp]
lemma finExtractTwo_apply_fst {n : ℕ} (i : Fin n.succ.succ) (j : Fin n.succ) :
  finExtractTwo i j i = Sum.inl (Sum.inl 0)
```

lemma finExtractTwo_symm_inr[\[source\]](#)

```
lemma finExtractTwo_symm_inr {n : ℕ} (i : Fin n.succ.succ) (j : Fin n.succ) :
  (finExtractTwo i j).symm ∘ Sum.inr = i.succAbove ∘ j.succAbove
```

lemma finExtractTwo_symm_inr_apply[\[source\]](#)

```
@[simp]
lemma finExtractTwo_symm_inr_apply {n : ℕ} (i : Fin n.succ.succ) (j : Fin n.succ) (x : Fin n) :
  (finExtractTwo i j).symm (Sum.inr x) = i.succAbove (j.succAbove x)
```

lemma finExtractTwo_symm_inl_inr_apply[\[source\]](#)

```
@[simp]
lemma finExtractTwo_symm_inl_inr_apply {n : ℕ} (i : Fin n.succ.succ) (j : Fin n.succ) :
  (finExtractTwo i j).symm (Sum.inl (Sum.inr 0)) = i.succAbove j
```

lemma finExtractTwo_symm_inl_inl_apply[\[source\]](#)

```
@[simp]
lemma finExtractTwo_symm_inl_inl_apply {n : ℕ} (i : Fin n.succ.succ) (j : Fin n.succ) :
  (finExtractTwo i j).symm (Sum.inl (Sum.inl 0)) = i
```

lemma finExtractTwo_apply_snd[\[source\]](#)

```
@[simp]
lemma finExtractTwo_apply_snd {n : ℕ} (i : Fin n.succ.succ) (j : Fin n.succ) :
  finExtractTwo i j (i.succAbove j) = Sum.inl (Sum.inr 0)
```

def finMapToEquiv[\[source\]](#)

Takes two maps $\text{Fin } n \rightarrow \text{Fin } m$ and returns the equivalence they form.

```
def finMapToEquiv (f1 : Fin n → Fin m) (f2 : Fin m → Fin n)
  (h : ∀ x, f1 (f2 x) = x := by decide)
  (h' : ∀ x, f2 (f1 x) = x := by decide) : Fin n ≈ Fin m where
```

lemma finMapToEquiv_apply[\[source\]](#)

```
@[simp]
lemma finMapToEquiv_apply {f1 : Fin n → Fin m} {f2 : Fin m → Fin n}
  {h : ∀ x, f1 (f2 x) = x} {h' : ∀ x, f2 (f1 x) = x} (x : Fin n) :
  finMapToEquiv f1 f2 h h' x = f1 x
```

lemma finMapToEquiv_symm_apply[\[source\]](#)

```
@[simp]
lemma finMapToEquiv_symm_apply {f1 : Fin n → Fin m} {f2 : Fin m → Fin n}
  {h : ∀ x, f1 (f2 x) = x} {h' : ∀ x, f2 (f1 x) = x} (x : Fin m) :
  (finMapToEquiv f1 f2 h h').symm x = f2 x
```

lemma finMapToEquiv_symm_eq[\[source\]](#)

```
lemma finMapToEquiv_symm_eq {f1 : Fin n → Fin m} {f2 : Fin m → Fin n}
  {h : ∀ x, f1 (f2 x) = x} {h' : ∀ x, f2 (f1 x) = x} :
  (finMapToEquiv f1 f2 h h').symm = finMapToEquiv f2 f1 h' h
```

def equivCons[\[source\]](#)

Given an equivalence between $\text{Fin } n$ and $\text{Fin } m$, the induced equivalence between $\text{Fin } n.\text{succ}$ and $\text{Fin } m.\text{succ}$ derived by $\text{Fin}.\text{cons}$.

```
def equivCons {n m : ℕ} (e : Fin n ≈ Fin m) : Fin n.succ ≈ Fin m.succ where
```

lemma equivCons_zero[\[source\]](#)

```
@[simp]
lemma equivCons_zero {n m : ℕ} (e : Fin n ≈ Fin m) :
  equivCons e 0 = 0
```

lemma equivCons_trans[\[source\]](#)

```
@[simp]
lemma equivCons_trans {n m k : ℕ} (e : Fin n ≈ Fin m) (f : Fin m ≈ Fin k) :
  Fin.equivCons (e.trans f) = (Fin.equivCons e).trans (Fin.equivCons f)
```

lemma equivCons_castOrderIso[\[source\]](#)

```
@[simp]
lemma equivCons_castOrderIso {n m : ℕ} (h : n = m) :
  (Fin.equivCons (Fin.castOrderIso h).toEquiv) = (Fin.castOrderIso (by simp [h])).toEquiv
```

lemma equivCons_symm_succ[\[source\]](#)

@[simp]

```
lemma equivCons_symm_succ {n m : ℕ} (e : Fin n ≈ Fin m) (i : ℕ) (hi : i + 1 < m.succ) :
  (Fin.equivCons e).symm (i + 1, hi) = (e.symm (i, Nat.succ_lt_succ_iff.mp hi)).succ
```

lemma equivCons_succ[\[source\]](#)

@[simp]

```
lemma equivCons_succ {n m : ℕ} (e : Fin n ≈ Fin m) (i : ℕ) (hi : i + 1 < n.succ) :
  (Fin.equivCons e) (i + 1, hi) = (e (i, Nat.succ_lt_succ_iff.mp hi)).succ
```

2.14 Physlib.Mathematics.Fin.Involutions[Physlib/Mathematics/Fin/Involutions.lean](#) on GitHub.**def involutionCons**[\[source\]](#)

There is an equivalence between involutions of $\text{Fin } n.\text{succ}$ and involutions of $\text{Fin } n$ and an optional valid choice of an element in $\text{Fin } n$ (which is where 0 in $\text{Fin } n.\text{succ}$ will be sent).

```
def involutionCons (n : ℕ) : {f : Fin n.succ → Fin n.succ // Function.Involutive f} ≈
  (f : {f : Fin n → Fin n // Function.Involutive f}) × {i : Option (Fin n) //
    ∀ (h : i.isSome), f.1 (Option.get i h) = (Option.get i h)} where
```

lemma involutionCons_ext[\[source\]](#)

```
lemma involutionCons_ext {n : ℕ} {f1 f2 : (f : {f : Fin n → Fin n // Function.Involutive f}) ×
  {i : Option (Fin n) // ∀ (h : i.isSome), f.1 (Option.get i h) = (Option.get i h)}}
  (h1 : f1.1 = f2.1) (h2 : f1.2 = Equiv.subtypeEquivRight (by rw [h1]; simp) f2.2) : f1 = f2
```

def involutionAddEquiv[\[source\]](#)

Given an involution of $\text{Fin } n$, the optional choice of an element in $\text{Fin } n$ which maps to itself is equivalent to the optional choice of an element in Fin ($\text{Finset.univ.filter fun i => f.1 i = i}$).card.

```
def involutionAddEquiv {n : ℕ} (f : {f : Fin n → Fin n // Function.Involutive f}) :
  {i : Option (Fin n) // ∀ (h : i.isSome), f.1 (Option.get i h) = (Option.get i h)} ≈
  Option (Fin (Finset.univ.filter fun i => f.1 i = i).card)
```

lemma involutionAddEquiv_none_image_zero[\[source\]](#)

```
lemma involutionAddEquiv_none_image_zero {n : ℕ} :
  {f : {f : Fin n.succ → Fin n.succ // Function.Involutive f}}
  → involutionAddEquiv (involutionCons n f).1 (involutionCons n f).2 = none
  → f.1 {0, Nat.zero_lt_succ n} = {0, Nat.zero_lt_succ n}
```

lemma involutionAddEquiv_cast[\[source\]](#)

```
lemma involutionAddEquiv_cast {n : ℕ} {f1 f2 : {f : Fin n → Fin n // Function.Involutive f}}
  (hf : f1 = f2) :
  involutionAddEquiv f1 = (Equiv.subtypeEquivRight (by rw [hf]; simp)).trans
```

```
((involutionAddEquiv f2).trans (Equiv.optionCongr (finCongr (by rw [hf]))))
```

lemma involutionAddEquiv_cast'

[\[source\]](#)

```
lemma involutionAddEquiv_cast' {m : ℕ} {f1 f2 : {f : Fin m → Fin m // Function.Involutive f}}
  {N : ℕ} (hf : f1 = f2) (n : Option (Fin N))
  (hn1 : N = (Finset.filter (fun i => f1.1 i = i) Finset.univ).card)
  (hn2 : N = (Finset.filter (fun i => f2.1 i = i) Finset.univ).card) :
  HEq ((involutionAddEquiv f1).symm (Option.map (finCongr hn1) n))
  ((involutionAddEquiv f2).symm (Option.map (finCongr hn2) n))
```

lemma involutionAddEquiv_none_succ

[\[source\]](#)

```
lemma involutionAddEquiv_none_succ {n : ℕ}
  {f : {f : Fin n.succ → Fin n.succ // Function.Involutive f}}
  (h : involutionAddEquiv (involutionCons n f).1 (involutionCons n f).2 = none)
  (x : Fin n) : f.1 x.succ = x.succ ↔ (involutionCons n f).1.1 x = x
```

def involutionNoFixedEquivSum

[\[source\]](#)

Fixed point free involutions of $\text{Fin } n.\text{succ}$ can be separated based on where they sent $\emptyset$.

```
def involutionNoFixedEquivSum {n : ℕ} :
  {f : Fin n.succ → Fin n.succ // Function.Involutive f}
  ∧ ∀ i, f i ≠ i} ≈ Σ (k : Fin n), {f : Fin n.succ → Fin n.succ // Function.Involutive f
  ∧ (∀ i, f i ≠ i) ∧ f ∅ = k.succ} where
```

def involutionNoFixedZeroSetEquivEquiv

[\[source\]](#)

The condition on fixed point free involutions of $\text{Fin } n.\text{succ}$ for a fixed value of $f \emptyset$, can be modified by conjugation with an equivalence.

```
def involutionNoFixedZeroSetEquivEquiv {n : ℕ}
  (k : Fin n) (e : Fin n.succ ≈ Fin n.succ) :
  {f : Fin n.succ → Fin n.succ // Function.Involutive f ∧ (∀ i, f i ≠ i) ∧ f ∅ = k.succ} ≈
  {f : Fin n.succ → Fin n.succ // Function.Involutive (e.symm ∘ f ∘ e) ∧
  (∀ i, (e.symm ∘ f ∘ e) i ≠ i) ∧ (e.symm ∘ f ∘ e) ∅ = k.succ} where
```

def involutionNoFixedZeroSetEquivSetEquiv

[\[source\]](#)

The condition on fixed point free involutions of $\text{Fin } n.\text{succ}$ for a fixed value of $f \emptyset$ given an equivalence e , can be modified so that only the condition on $f \emptyset$ is up-to the equivalence e .

```
def involutionNoFixedZeroSetEquivSetEquiv {n : ℕ} (k : Fin n)
  (e : Fin n.succ ≈ Fin n.succ) :
  {f : Fin n.succ → Fin n.succ // Function.Involutive (e.symm ∘ f ∘ e) ∧
  (∀ i, (e.symm ∘ f ∘ e) i ≠ i) ∧ (e.symm ∘ f ∘ e) ∅ = k.succ} ≈
  {f : Fin n.succ → Fin n.succ // Function.Involutive f ∧
  (∀ i, f i ≠ i) ∧ (e.symm ∘ f ∘ e) ∅ = k.succ}
```


def involutionNoFixedZeroSetEquivEquiv'[\[source\]](#)

Fixed point free involutions of $\text{Fin } n.\text{succ}$ fixing $(e.\text{symm} \circ f \circ e) = k.\text{succ}$ for a given e are equivalent to fixing $f(e \ 0) = e \ k.\text{succ}$.

```
def involutionNoFixedZeroSetEquivEquiv' {n : ℕ} (k : Fin n) (e : Fin n.succ ≈ Fin n.succ) :
  {f : Fin n.succ → Fin n.succ // Function.Involutive f ∧
    (∀ i, f i ≠ i) ∧ (e.symm ∘ f ∘ e) 0 = k.succ} ≈
  {f : Fin n.succ → Fin n.succ // Function.Involutive f ∧
    (∀ i, f i ≠ i) ∧ f (e 0) = e k.succ}
```

def involutionNoFixedZeroSetEquivSetOne[\[source\]](#)

Fixed point involutions of $\text{Fin } n.\text{succ}.\text{succ}$ with $f \ 0 = k.\text{succ}$ are equivalent to fixed point involutions with $f \ 0 = 1$.

```
def involutionNoFixedZeroSetEquivSetOne {n : ℕ} (k : Fin n.succ) :
  {f : Fin n.succ.succ → Fin n.succ.succ // Function.Involutive f ∧
    (∀ i, f i ≠ i) ∧ f 0 = k.succ}
  ≈ {f : Fin n.succ.succ → Fin n.succ.succ // Function.Involutive f ∧
    (∀ i, f i ≠ i) ∧ f 0 = 1}
```

def involutionNoFixedSetOne[\[source\]](#)

Fixed point involutions of $\text{Fin } n.\text{succ}.\text{succ}$ fixing $f \ 0 = 1$ are equivalent to fixed point involutions of $\text{Fin } n$.

```
def involutionNoFixedSetOne {n : ℕ} :
  {f : Fin n.succ.succ → Fin n.succ.succ // Function.Involutive f ∧
    (∀ i, f i ≠ i) ∧ f 0 = 1} ≈ {f : Fin n → Fin n // Function.Involutive f ∧
    (∀ i, f i ≠ i)} where
```

def involutionNoFixedZeroSetEquiv[\[source\]](#)

Fixed point involutions of $\text{Fin } n.\text{succ}.\text{succ}$ for fixed $f \ 0 = k.\text{succ}$ are equivalent to fixed point involutions of $\text{Fin } n$.

```
def involutionNoFixedZeroSetEquiv {n : ℕ} (k : Fin n.succ) :
  {f : Fin n.succ.succ → Fin n.succ.succ // Function.Involutive f ∧
    (∀ i, f i ≠ i) ∧ f 0 = k.succ}
  ≈ {f : Fin n → Fin n // Function.Involutive f ∧ (∀ i, f i ≠ i)}
```

def involutionNoFixedEquivSumSame[\[source\]](#)

The type of fixed point free involutions of $\text{Fin } n.\text{succ}.\text{succ}$ is equivalent to the sum of $\text{Fin } n.\text{succ}$ copies of fixed point involutions of $\text{Fin } n$.

```
def involutionNoFixedEquivSumSame {n : ℕ} :
  {f : Fin n.succ.succ → Fin n.succ.succ // Function.Involutive f ∧ (∀ i, f i ≠ i)}
  ≈ Σ ( _ : Fin n.succ), {f : Fin n → Fin n // Function.Involutive f ∧ (∀ i, f i ≠ i)}
```

def involutionNoFixedZeroEquivProd[\[source\]](#)

Every fixed-point free involutions of $\text{Fin } n.\text{succ}.\text{succ}$ can be decomposed into a element of $\text{Fin } n.\text{succ}$ (where 0 is sent) and a fixed-point free involution of $\text{Fin } n$.

```
def involutionNoFixedZeroEquivProd {n : ℕ} :
  {f : Fin n.succ.succ → Fin n.succ.succ // Function.Involutive f ∧ (∀ i, f i ≠ i)}
  ≈ Fin n.succ × {f : Fin n → Fin n // Function.Involutive f ∧ (∀ i, f i ≠ i)}
```

```
instance Fintype { f // Function.Involutive f ∧ ∀ (i : Fin n), f i ≠ i }
```

[\[source\]](#)

The type of fixed-point free involutions of $\text{Fin } n$ is finite.

```
instance {n : ℕ} : Fintype { f // Function.Involutive f ∧ ∀ (i : Fin n), f i ≠ i }
```

```
lemma involutionNoFixed_card_succ
```

[\[source\]](#)

```
lemma involutionNoFixed_card_succ {n : ℕ} :
  Fintype.card
  {f : Fin n.succ.succ → Fin n.succ.succ // Function.Involutive f ∧ (∀ i, f i ≠ i)}
  = n.succ *
  Fintype.card {f : Fin n → Fin n // Function.Involutive f ∧ (∀ i, f i ≠ i)}
```

```
lemma involutionNoFixed_card_mul_two
```

[\[source\]](#)

```
lemma involutionNoFixed_card_mul_two : (n : ℕ) →
  Fintype.card {f : Fin (2 * n) → Fin (2 * n) // Function.Involutive f ∧ (∀ i, f i ≠ i)}
  = (2 * n - 1)!!
| 0 => rfl
| Nat.succ n => by
  erw [involutionNoFixed_card_succ, involutionNoFixed_card_mul_two n]
  exact (Nat.doubleFactorial_add_one (Nat.mul 2 n)).symm
```

```
lemma involutionNoFixed_card_mul_two_plus_one
```

[\[source\]](#)

```
lemma involutionNoFixed_card_mul_two_plus_one : (n : ℕ) →
  Fintype.card {f : Fin (2 * n + 1) → Fin (2 * n + 1) // Function.Involutive f ∧ (∀ i, f i ≠ i)}
  = 0
| 0 => rfl
| Nat.succ n => by
  erw [involutionNoFixed_card_succ, involutionNoFixed_card_mul_two_plus_one n]
  ring
```

```
lemma involutionNoFixed_card_even
```

[\[source\]](#)

```
lemma involutionNoFixed_card_even : (n : ℕ) → (he : Even n) →
  Fintype.card {f : Fin n → Fin n // Function.Involutive f ∧ (∀ i, f i ≠ i)} = (n - 1)!!
```

```
lemma involutionNoFixed_card_odd
```

[\[source\]](#)

```
lemma involutionNoFixed_card_odd : (n : ℕ) → (ho : Odd n) →
  Fintype.card {f : Fin n → Fin n // Function.Involutive f ∧ (∀ i, f i ≠ i)} = 0
```

2.15 Physlib.Mathematics.List

[Physlib/Mathematics/List.lean](#) on GitHub.

lemma takeWhile_eraseIdx[\[source\]](#)

```

lemma takeWhile_eraseIdx {I : Type} (P : I → Prop) [DecidablePred P] :
  (l : List I) → (i : ℕ) → (hi : ∀ (j : Fin l.length), i < j → P (l.get j) → P (l.get i)) →
  List.takeWhile P (List.eraseIdx l i) = (List.takeWhile P l).eraseIdx i
| [], _, h => by
  rfl
| a :: [], 0, h => by
  simp only [List.takeWhile, List.eraseIdx_zero]
  split
  · rfl
  · rfl
| a :: [], Nat.succ n, h => by
  simp only [Nat.succ_eq_add_one, List.eraseIdx_cons_succ, List.eraseIdx_nil]
  rw [List.eraseIdx_of_length_le]
  have h1 : (List.takeWhile P [a]).length ≤ [a].length

```

lemma dropWhile_eraseIdx[\[source\]](#)

```

lemma dropWhile_eraseIdx {I : Type} (P : I → Prop) [DecidablePred P] :
  (l : List I) → (i : ℕ) → (hi : ∀ (j : Fin l.length), i < j → P (l.get j) → P (l.get i)) →
  List.dropWhile P (List.eraseIdx l i) =
    if (List.takeWhile P l).length ≤ i then
      (List.dropWhile P l).eraseIdx (i - (List.takeWhile P l).length)
    else (List.dropWhile P l)
| [], _, h => by
  simp
| a :: [], 0, h => by
  simp only [List.eraseIdx_zero, nonpos_iff_eq_zero, List.length_eq_zero_iff,
    List.takeWhile_eq_nil_iff, List.length_singleton, zero_lt_one, Fin.zero_eta, Fin.isValue,
    List.get_eq_getElem, Fin.val_eq_zero, List.getElem_cons_zero, decide_eq_true_eq,
    forall_const,
    List.dropWhile, zero_le, Nat.sub_eq_zero_of_le, ite_not]
  simp_all only [List.length_cons, List.length_nil, List.get_eq_getElem, Fin.val_eq_zero,
    List.getElem_cons_zero, implies_true, List.tail_cons, List.dropWhile_nil, decide_true,
    decide_false, ite_self]
| a :: [], Nat.succ n, h => by
  simp only [List.dropWhile, List.takeWhile, Nat.succ_eq_add_one]
  rw [List.eraseIdx_of_length_le]
  simp_all only [List.length_singleton, List.get_eq_getElem, Fin.val_eq_zero,
    List.getElem_cons_zero, implies_true]
  split
  next x heq =>
    simp_all only [decide_eq_true_eq, decide_true, List.dropWhile_cons_of_pos, List.
      dropWhile_nil,
      List.length_singleton, le_add_iff_nonneg_left, zero_le, ↗reduceIte,
      add_tsub_cancel_right, List.eraseIdx_nil]
  next x heq =>
    simp_all only [decide_eq_false_iff_not, decide_false, Bool.false_eq_true,
      not_false_eq_true,
      List.dropWhile_cons_of_neg, List.length_nil, le_add_iff_nonneg_left,
      zero_le, ↗reduceIte, tsub_zero, List.eraseIdx_cons_succ, List.eraseIdx_nil]
  exact Nat.le_add_left [a].length n
| a :: b :: l, 0, h => by
  simp only [List.dropWhile, List.takeWhile, nonpos_iff_eq_zero, List.length_eq_zero_iff,
    zero_le,
    Nat.sub_eq_zero_of_le, List.eraseIdx_zero]
  by_cases hPb : P b

```

• **have** hPa : P a

lemma insertionSort_length

[\[source\]](#)

```
lemma insertionSort_length {I : Type} (le1 : I → I → Prop) [DecidableRel le1] (l : List I) :
  (List.insertionSort le1 l).length = l.length
```

def orderedInsertPos

[\[source\]](#)

The position r0 ends up in r on adding it via List.orderedInsert _ r0 r.

@[expose]

```
def orderedInsertPos {I : Type} (le1 : I → I → Prop) [DecidableRel le1] (r : List I) (r0 : I) :
  Fin (List.orderedInsert le1 r0 r).length
```

lemma orderedInsertPos_lt_length

[\[source\]](#)

```
lemma orderedInsertPos_lt_length {I : Type} (le1 : I → I → Prop) [DecidableRel le1] (r : List I
)
  (r0 : I) : orderedInsertPos le1 r r0 < (r0 :: r).length
```

lemma orderedInsert_get_orderedInsertPos

[\[source\]](#)

@[simp]

```
lemma orderedInsert_get_orderedInsertPos {I : Type} (le1 : I → I → Prop) [DecidableRel le1]
  (r : List I) (r0 : I) :
  (List.orderedInsert le1 r0 r)[(orderedInsertPos le1 r r0).val] = r0
```

lemma orderedInsert_eraseIdx_orderedInsertPos

[\[source\]](#)

@[simp]

```
lemma orderedInsert_eraseIdx_orderedInsertPos {I : Type} (le1 : I → I → Prop) [DecidableRel le1]
]
  (r : List I) (r0 : I) :
  (List.orderedInsert le1 r0 r).eraseIdx ↑(orderedInsertPos le1 r r0) = r
```

lemma orderedInsertPos_cons

[\[source\]](#)

```
lemma orderedInsertPos_cons {I : Type} (le1 : I → I → Prop) [DecidableRel le1]
  (r : List I) (r0 r1 : I) :
  (orderedInsertPos le1 (r1 :: r) r0).val =
  if le1 r0 r1 then {0, by simp} else (Fin.succ (orderedInsertPos le1 r r0))
```

lemma orderedInsertPos_sigma

[\[source\]](#)

```
lemma orderedInsertPos_sigma {I : Type} {f : I → Type}
  (le1 : I → I → Prop) [DecidableRel le1] (l : List (Σ i, f i))
  (k : I) (a : f k) :
  (orderedInsertPos (fun (i j : Σ i, f i) => le1 i.1 j.1) l (k, a)).1 =
  (orderedInsertPos le1 (List.map (fun (i : Σ i, f i) => i.1) l) k).1
```

lemma orderedInsert_get_lt[\[source\]](#)

```

lemma orderedInsert_get_lt {I : Type} (le1 : I → I → Prop) [DecidableRel le1]
  (r : List I) (r0 : I) (i : ℕ)
  (hi : i < orderedInsertPos le1 r r0) :
  (List.orderedInsert le1 r0 r)[i] = r.get (i, by
    simp only [orderedInsertPos] at hi
    have h1 : (List.takeWhile (fun b => decide ¬le1 r0 b) r).length ≤ r.length :=
      List.Sublist.length_le (List.takeWhile_sublist fun b => decide ¬le1 r0 b)
      omega)

```

lemma orderedInsertPos_take_orderedInsert[\[source\]](#)

```

lemma orderedInsertPos_take_orderedInsert {I : Type} (le1 : I → I → Prop) [DecidableRel le1]
  (r : List I) (r0 : I) :
  (List.take (orderedInsertPos le1 r r0) (List.orderedInsert le1 r0 r)) =
  List.takeWhile (fun b => decide ¬le1 r0 b) r

```

lemma orderedInsertPos_take_eq_orderedInsert[\[source\]](#)

```

lemma orderedInsertPos_take_eq_orderedInsert {I : Type} (le1 : I → I → Prop) [DecidableRel le1]
  (r : List I) (r0 : I) :
  List.take (orderedInsertPos le1 r r0) r =
  List.take (orderedInsertPos le1 r r0) (List.orderedInsert le1 r0 r)

```

lemma orderedInsertPos_drop_eq_orderedInsert[\[source\]](#)

```

lemma orderedInsertPos_drop_eq_orderedInsert {I : Type} (le1 : I → I → Prop) [DecidableRel le1]
  (r : List I) (r0 : I) :
  List.drop (orderedInsertPos le1 r r0) r =
  List.drop (orderedInsertPos le1 r r0).succ (List.orderedInsert le1 r0 r)

```

lemma orderedInsertPos_take[\[source\]](#)

```

lemma orderedInsertPos_take {I : Type} (le1 : I → I → Prop) [DecidableRel le1]
  (r : List I) (r0 : I) :
  List.take (orderedInsertPos le1 r r0) r = List.takeWhile (fun b => decide ¬le1 r0 b) r

```

lemma orderedInsertPos_drop[\[source\]](#)

```

lemma orderedInsertPos_drop {I : Type} (le1 : I → I → Prop) [DecidableRel le1]
  (r : List I) (r0 : I) :
  List.drop (orderedInsertPos le1 r r0) r = List.dropWhile (fun b => decide ¬le1 r0 b) r

```

lemma orderedInsertPos_succ_take_orderedInsert[\[source\]](#)

```

lemma orderedInsertPos_succ_take_orderedInsert {I : Type} (le1 : I → I → Prop) [DecidableRel
  le1]
  (r : List I) (r0 : I) :
  (List.take (orderedInsertPos le1 r r0).succ (List.orderedInsert le1 r0 r)) =
  List.takeWhile (fun b => decide ¬le1 r0 b) r ++ [r0]

```

lemma lt_orderedInsertPos_rel[\[source\]](#)

```

lemma lt_orderedInsertPos_rel {I : Type} (le1 : I → I → Prop) [DecidableRel le1]
  (r0 : I) (r : List I) (n : Fin r.length)
  (hn : n.val < (orderedInsertPos le1 r r0).val) : ¬ le1 r0 (r.get n)

```

lemma lt_orderedInsertPos_rel_fin[\[source\]](#)

```

lemma lt_orderedInsertPos_rel_fin {I : Type} (le1 : I → I → Prop) [DecidableRel le1]
  (r0 : I) (r : List I) (n : Fin (List.orderedInsert le1 r0 r).length)
  (hn : n < (orderedInsertPos le1 r r0)) : ¬ le1 r0 ((List.orderedInsert le1 r0 r).get n)

```

lemma gt_orderedInsertPos_rel[\[source\]](#)

```

lemma gt_orderedInsertPos_rel {I : Type} (le1 : I → I → Prop) [DecidableRel le1]
  [Std.Total le1] [IsTrans I le1] (r0 : I) (r : List I) (hs : List.Pairwise le1 r)
  (n : Fin r.length)
  (hn : ¬ n.val < (orderedInsertPos le1 r r0).val) : le1 r0 (r.get n)

```

lemma orderedInsert_eraseIdx_lt_orderedInsertPos[\[source\]](#)

```

lemma orderedInsert_eraseIdx_lt_orderedInsertPos {I : Type} (le1 : I → I → Prop) [DecidableRel
  le1]
  (r : List I) (r0 : I) (i : ℕ)
  (hi : i < orderedInsertPos le1 r r0)
  (hr : ∀ (i j : Fin r.length), i < j → ¬le1 r0 (r.get j) → ¬le1 r0 (r.get i)) :
  (List.orderedInsert le1 r0 r).eraseIdx i = List.orderedInsert le1 r0 (r.eraseIdx i)

```

lemma orderedInsert_eraseIdx_orderedInsertPos_le[\[source\]](#)

```

lemma orderedInsert_eraseIdx_orderedInsertPos_le {I : Type} (le1 : I → I → Prop) [DecidableRel
  le1]
  (r : List I) (r0 : I) (i : ℕ)
  (hi : orderedInsertPos le1 r r0 ≤ i)
  (hr : ∀ (i j : Fin r.length), i < j → ¬le1 r0 (r.get j) → ¬le1 r0 (r.get i)) :
  (List.orderedInsert le1 r0 r).eraseIdx (Nat.succ i) =
  List.orderedInsert le1 r0 (r.eraseIdx i)

```

def orderedInsertEquiv[\[source\]](#)

The equivalence between $\text{Fin } (r0 :: r).length$ and $\text{Fin } (\text{List.orderedInsert } le1 \ r0 \ r).length$ according to where the elements map, i.e. θ is taken to $\text{orderedInsertPos } le1 \ r \ r0$.

```

@[expose]

```

```

def orderedInsertEquiv {I : Type} (le1 : I → I → Prop) [DecidableRel le1] (r : List I) (r0 : I)
  :
  Fin (r.length + 1) ≈ Fin (List.orderedInsert le1 r0 r).length

```

lemma orderedInsertEquiv_zero[\[source\]](#)

```

@[simp]
lemma orderedInsertEquiv_zero {I : Type} (le1 : I → I → Prop) [DecidableRel le1] (r : List I)
  (r0 : I) : orderedInsertEquiv le1 r r0 0 = orderedInsertPos le1 r r0

```

lemma orderedInsertEquiv_succ[\[source\]](#)

```

lemma orderedInsertEquiv_succ {I : Type} (le1 : I → I → Prop) [DecidableRel le1] (r : List I)
  (r0 : I) (n : ℕ) (hn : Nat.succ n < (r0 :: r).length) :
  orderedInsertEquiv le1 r r0 (Nat.succ n, hn) =
  Fin.cast (List.orderedInsert_length le1 r r0).symm
  ((Fin.succAbove ((orderedInsertPos le1 r r0), orderedInsertPos_lt_length le1 r r0))
  (n, Nat.succ_lt_succ_iff.mp hn))

```

lemma orderedInsertEquiv_fin_succ[\[source\]](#)

```

lemma orderedInsertEquiv_fin_succ {I : Type} (le1 : I → I → Prop) [DecidableRel le1] (r : List
  I)
  (r0 : I) (n : Fin r.length) :
  orderedInsertEquiv le1 r r0 n.succ = Fin.cast (List.orderedInsert_length le1 r r0).symm
  ((Fin.succAbove ((orderedInsertPos le1 r r0), orderedInsertPos_lt_length le1 r r0))
  (n, n.isLt))

```

lemma orderedInsertEquiv_monotone_fin_succ[\[source\]](#)

```

lemma orderedInsertEquiv_monotone_fin_succ {I : Type}
  (le1 : I → I → Prop) [DecidableRel le1] (r : List I)
  (r0 : I) (n m : Fin r.length)
  (hx : orderedInsertEquiv le1 r r0 n.succ < orderedInsertEquiv le1 r r0 m.succ) :
  n < m

```

lemma orderedInsertEquiv_congr[\[source\]](#)

```

lemma orderedInsertEquiv_congr {α : Type} {r : α → α → Prop} [DecidableRel r] (a : α)
  (l l' : List α) (h : l = l') :
  orderedInsertEquiv r l a = (Fin.castOrderIso (by simp [h])).toEquiv.trans
  ((orderedInsertEquiv r l' a).trans (Fin.castOrderIso (by simp [h])).toEquiv)

```

lemma get_eq_orderedInsertEquiv[\[source\]](#)

```

lemma get_eq_orderedInsertEquiv {I : Type} (le1 : I → I → Prop) [DecidableRel le1] (r : List I)
  (r0 : I) :
  (r0 :: r).get = (List.orderedInsert le1 r0 r).get ◦ (orderedInsertEquiv le1 r r0)

```

lemma orderedInsertEquiv_get[\[source\]](#)

```

lemma orderedInsertEquiv_get {I : Type} (le1 : I → I → Prop) [DecidableRel le1] (r : List I)
  (r0 : I) :
  (r0 :: r).get ◦ (orderedInsertEquiv le1 r r0).symm = (List.orderedInsert le1 r0 r).get

```

lemma orderedInsert_eraseIdx_orderedInsertEquiv_zero[\[source\]](#)

```

lemma orderedInsert_eraseIdx_orderedInsertEquiv_zero
  {I : Type} (le1 : I → I → Prop) [DecidableRel le1] (r : List I) (r0 : I) :
  (List.orderedInsert le1 r0 r).eraseIdx (orderedInsertEquiv le1 r r0 (0, by simp)) = r

```

lemma orderedInsert_eraseIdx_orderedInsertEquiv_succ[\[source\]](#)

```

lemma orderedInsert_eraseIdx_orderedInsertEquiv_succ
  {I : Type} (le1 : I → I → Prop) [DecidableRel le1] (r : List I) (r0 : I) (n : ℕ)
  (hn : Nat.succ n < (r0 :: r).length)
  (hr : ∀ (i j : Fin r.length), i < j → ¬le1 r0 (r.get j) → ¬le1 r0 (r.get i)) :
  (List.orderedInsert le1 r0 r).eraseIdx (orderedInsertEquiv le1 r r0 (Nat.succ n, hn)) =
  (List.orderedInsert le1 r0 (r.eraseIdx n))

```

lemma orderedInsert_eraseIdx_orderedInsertEquiv_fin_succ[\[source\]](#)

```

lemma orderedInsert_eraseIdx_orderedInsertEquiv_fin_succ
  {I : Type} (le1 : I → I → Prop) [DecidableRel le1] (r : List I) (r0 : I) (n : Fin r.length)
  (hr : ∀ (i j : Fin r.length), i < j → ¬le1 r0 (r.get j) → ¬le1 r0 (r.get i)) :
  (List.orderedInsert le1 r0 r).eraseIdx (orderedInsertEquiv le1 r r0 n.succ) =
  (List.orderedInsert le1 r0 (r.eraseIdx n))

```

lemma orderedInsertEquiv_sigma[\[source\]](#)

```

lemma orderedInsertEquiv_sigma {I : Type} {f : I → Type}
  (le1 : I → I → Prop) [DecidableRel le1] (l : List (Σ i, f i))
  (i : I) (a : f i) :
  (orderedInsertEquiv (fun i j => le1 i.fst j.fst) l (i, a)) =
  (Fin.castOrderIso (by simp)).toEquiv.trans
  ((orderedInsertEquiv le1 (List.map (fun i => i.1) l) i).trans
  (Fin.castOrderIso (by simp [List.orderedInsert_length]))).toEquiv)

```

lemma orderedInsert_eq_insertIdx_orderedInsertPos[\[source\]](#)

```

lemma orderedInsert_eq_insertIdx_orderedInsertPos {I : Type} (le1 : I → I → Prop) [DecidableRel
  le1]
  (r : List I) (r0 : I) :
  List.orderedInsert le1 r0 r = List.insertIdx r (orderedInsertPos le1 r r0).1 r0

```

def insertionSortEquiv[\[source\]](#)

The equivalence between $\text{Fin } l.\text{length} \approx \text{Fin } (\text{List.insertionSort } r \ l).\text{length}$ induced by the sorting algorithm.

```

def insertionSortEquiv {α : Type} (r : α → α → Prop) [DecidableRel r] : (l : List α) →
  Fin l.length ≈ Fin (List.insertionSort r l).length
| [] => Equiv.refl _
| a :: l =>
  (Fin.equivCons (insertionSortEquiv r l)).trans (orderedInsertEquiv r (List.insertionSort r
  l) a)

```

lemma insertionSortEquiv_get[\[source\]](#)

```

lemma insertionSortEquiv_get {α : Type} {r : α → α → Prop} [DecidableRel r] : (l : List α) →
  l.get ◦ (insertionSortEquiv r l).symm = (List.insertionSort r l).get
| [] => by rfl
| a :: l => by
  rw [insertionSortEquiv]
  change ((a :: l).get ◦ ((Fin.equivCons (insertionSortEquiv r l))).symm) ◦
  (orderedInsertEquiv r (List.insertionSort r l) a).symm = _

```



```

have hl : (a :: l).get ◦ ((Fin.equivCons (insertionSortEquiv r l))).symm =
  (a :: List.insertionSort r l).get

```

lemma insertionSortEquiv_congr

[\[source\]](#)

```

lemma insertionSortEquiv_congr {α : Type} {r : α → α → Prop} [DecidableRel r] (l l' : List α)
  (h : l = l') : insertionSortEquiv r l = (Fin.castOrderIso (by simp [h])).toEquiv.trans
  ((insertionSortEquiv r l').trans (Fin.castOrderIso (by simp [h])).toEquiv)

```

lemma insertionSortEquiv_congr_apply

[\[source\]](#)

```

lemma insertionSortEquiv_congr_apply {α : Type} {r : α → α → Prop} [DecidableRel r] (l l' :
  List α)
  (h : l = l') (i : Fin l.length) :
  insertionSortEquiv r l i =
  Fin.cast (by simp [h])
  ((insertionSortEquiv r l') (Fin.cast (by simp [h]) i))

```

lemma insertionSort_get_comp_insertionSortEquiv

[\[source\]](#)

```

lemma insertionSort_get_comp_insertionSortEquiv {α : Type} {r : α → α → Prop} [DecidableRel r]
  (l : List α) : (List.insertionSort r l).get ◦ (insertionSortEquiv r l) = l.get

```

lemma insertionSort_eq_ofFn

[\[source\]](#)

```

lemma insertionSort_eq_ofFn {α : Type} {r : α → α → Prop} [DecidableRel r] (l : List α) :
  List.insertionSort r l = List.ofFn (l.get ◦ (insertionSortEquiv r l).symm)

```

lemma insertionSortEquiv_order

[\[source\]](#)

```

lemma insertionSortEquiv_order {α : Type} {r : α → α → Prop} [DecidableRel r] :
  (l : List α) → (i : Fin l.length) → (j : Fin l.length) → (hij : i < j)
  → (hij' : insertionSortEquiv r l j < insertionSortEquiv r l i) →
  ¬ r l[i] l[j]
| [], i, _, _, _ => Fin.elim0 i
| a :: as, {0, hi}, {j + 1, hj}, hij, hij' => by
  simp only [List.length_cons, Fin.zero_eta, Fin.getElem_fin, Fin.val_zero,
    List.getElem_cons_zero, List.getElem_cons_succ]
  nth_rewrite 2 [insertionSortEquiv] at hij'
  simp only [List.length_cons, Nat.succ_eq_add_one, Fin.zero_eta,
    Equiv.trans_apply, equivCons_zero] at hij'
  have hx

```

lemma eraseIdx_length'

[\[source\]](#)

```

lemma eraseIdx_length' {I : Type} (l : List I) (i : Fin l.length) :
  (List.eraseIdx l i).length = l.length - 1

```

lemma eraseIdx_length

[\[source\]](#)

```

lemma eraseIdx_length {I : Type} (l : List I) (i : Fin l.length) :
  (List.eraseIdx l i).length + 1 = l.length

```

lemma eraseIdx_length_succ[\[source\]](#)

```
lemma eraseIdx_length_succ {I : Type} (l : List I) (i : Fin l.length) :
  (List.eraseIdx l i).length.succ = l.length
```

lemma eraseIdx_cons_length[\[source\]](#)

```
lemma eraseIdx_cons_length {I : Type} (a : I) (l : List I) (i : Fin (a :: l).length) :
  (List.eraseIdx (a :: l) i).length = l.length
```

lemma eraseIdx_get[\[source\]](#)

```
lemma eraseIdx_get {I : Type} (l : List I) (i : Fin l.length) :
  (List.eraseIdx l i).get = l.get ◦ (Fin.cast (eraseIdx_length l i)) ◦
  (Fin.cast (eraseIdx_length l i).symm i).succAbove
```

lemma eraseIdx_insertionSort[\[source\]](#)

```
lemma eraseIdx_insertionSort {I : Type} (le1 : I → I → Prop) [DecidableRel le1]
  [Std.Total le1] [IsTrans I le1] :
  (n : ℕ) → (r : List I) → (hn : n < r.length) →
  (List.insertionSort le1 r).eraseIdx ↑((insertionSortEquiv le1 r) {n, hn})
  = List.insertionSort le1 (r.eraseIdx n)
| 0, [], _ => by rfl
| 0, (r0 :: r), hn => by
  simp only [List.insertionSort, List.foldr_cons, List.length_cons, insertionSortEquiv,
    Nat.succ_eq_add_one, Fin.zero_eta, Equiv.trans_apply, equivCons_zero,
    orderedInsertEquiv_zero, orderedInsert_eraseIdx_orderedInsertPos, List.eraseIdx_zero, List.tail_cons]
| Nat.succ n, [], hn => by rfl
| Nat.succ n, (r0 :: r), hn => by
  simp only [List.insertionSort, List.length_cons, insertionSortEquiv, Nat.succ_eq_add_one,
    Equiv.trans_apply, equivCons_succ]
  have h0r
```

lemma eraseIdx_insertionSort_fin[\[source\]](#)

```
lemma eraseIdx_insertionSort_fin {I : Type} (le1 : I → I → Prop) [DecidableRel le1]
  [Std.Total le1] [IsTrans I le1] (r : List I) (n : Fin r.length) :
  (List.insertionSort le1 r).eraseIdx ↑((Physlib.List.insertionSortEquiv le1 r) n)
  = List.insertionSort le1 (r.eraseIdx n)
```

def insertionSortMinPos[\[source\]](#)

Given a list $i :: l$ the left-most minimal position a of $i :: l$ wrt r . That is the first position of l such that for every element $(i :: l)[b]$ before that position $r ((i :: l)[b]) ((i :: l)[a])$ is not true. The use of $i :: l$ here rather than just l is to ensure that such a position exists. .

```
@[expose]
```

```
def insertionSortMinPos {α : Type} (r : α → α → Prop) [DecidableRel r] (i : α) (l : List α) :
  Fin (i :: l).length
```

lemma insertionSortMin_eq_insertionSort_head[\[source\]](#)

```

lemma insertionSortMin_eq_insertionSort_head {α : Type} (r : α → α → Prop) [DecidableRel r]
  (i : α) (l : List α) :
  insertionSortMin r i l = (List.insertionSort r (i :: l)).head (by
  refine List.ne_nil_of_length_pos ?_
  rw [insertionSort_length]
  exact Nat.zero_lt_succ l.length)

```

def insertionSortDropMinPos[\[source\]](#)

The list remaining after dropping the element at the position determined by `insertionSortMinPos`.

```

@[expose]
def insertionSortDropMinPos {α : Type} (r : α → α → Prop) [DecidableRel r] (i : α) (l : List α)
  :
  List α

```

lemma insertionSort_eq_insertionSortMin_cons[\[source\]](#)

```

lemma insertionSort_eq_insertionSortMin_cons {α : Type} (r : α → α → Prop) [DecidableRel r]
  [Std.Total r] [IsTrans α r] (i : α) (l : List α) :
  List.insertionSort r (i :: l) =
  (insertionSortMin r i l) :: List.insertionSort r (insertionSortDropMinPos r i l)

```

lemma optionEraseZ_some_length[\[source\]](#)

```

@[simp]
lemma optionEraseZ_some_length {I : Type} (l : List I) (a : I) (i : (Fin l.length)) :
  (optionEraseZ l a (some i)).length = l.length - 1

```

lemma optionEraseZ_ext[\[source\]](#)

```

lemma optionEraseZ_ext {I : Type} {l l' : List I} {a a' : I} {i : Option (Fin l.length)}
  {i' : Option (Fin l'.length)} (hl : l = l') (ha : a = a')
  (hi : Option.map (Fin.cast (by rw [hl])) i = i') :
  optionEraseZ l a i = optionEraseZ l' a' i'

```

lemma mem_take_finrange[\[source\]](#)

```

lemma mem_take_finrange : (n m : ℕ) → (a : Fin n) → a ∈ List.take m (List.finRange n) ↔ a.val <
  m
| 0, m, a => Fin.elim0 a
| n+1, 0, a => by
  simp
| n +1, m + 1, ⟨0, h⟩ => by
  simp [List.finRange_succ]
| n +1, m + 1, ⟨i + 1, h⟩ => by
  simp only [List.finRange_succ, List.take_succ_cons, List.mem_cons, Fin.ext_iff, Fin.
  val_zero,
  AddLeftCancelMonoid.add_eq_zero, one_ne_zero, and_false, false_or, add_lt_add_iff_right]
  rw [← List.map_take, @List.mem_map]
  apply Iff.intro
  · intro h
    obtain ⟨a, ha⟩

```

2.16 Physlib.Mathematics.List.InsertIdx

[Physlib/Mathematics/List/InsertIdx.lean](#) on GitHub.

lemma insertIdx_map

[\[source\]](#)

```
lemma insertIdx_map {I J : Type} (f : I → J) : (i : ℕ) → (r : List I) → (r0 : I) →
  (List.insertIdx r i r0).map f = List.insertIdx (r.map f) i (f r0)
| 0, [], r0 => by simp
| n+1, [], r0 => by simp
| 0, a::as, r0 => by simp
| n+1, a::as, r0 => by
  simp only [List.insertIdx_succ_cons, List.map_cons, List.cons.injEq, true_and]
  exact insertIdx_map f n as r0
```

lemma eraseIdx_sorted

[\[source\]](#)

```
lemma eraseIdx_sorted {I : Type} (le : I → I → Prop) :
  (r : List I) → (n : ℕ) →
  List.Pairwise le r → List.Pairwise le (r.eraseIdx n)
| [], _, _ => by simp
| a::as, 0, h => by
  simp only [List.eraseIdx]
  simp only [List.pairwise_cons] at h
  exact h.2
| a::as, n+1, h => by
  simp only [List.eraseIdx_cons_succ, List.pairwise_cons]
  simp only [List.pairwise_cons] at h
  refine And.intro ?_ (eraseIdx_sorted le as n h.2)
  intro b hb
  refine h.1 _ ?_
  exact List.mem_of_mem_eraseIdx hb
```

lemma mem_eraseIdx_nodup

[\[source\]](#)

```
lemma mem_eraseIdx_nodup {I : Type} (i : I) :
  (l : List I) → (n : ℕ) → (hn : n < l.length) → (h : List.Nodup l) →
  i ∈ l.eraseIdx n ↔ i ∈ l ∧ i ≠ l[n]
| [], _, _, _ => by simp
| a1 :: as, 0, _, h => by
  simp only [List.eraseIdx_zero, List.tail_cons, List.mem_cons, List.getElem_cons_zero, ne_eq]
  by_cases hi : i = a1
  · subst hi
    simp only [List.nodup_cons] at h
    simp [h]
  · simp [hi]
| a1 :: as, n+1, hn, h => by
  simp only [List.eraseIdx_cons_succ, List.mem_cons, List.getElem_cons_succ, ne_eq]
  simp only [List.nodup_cons] at h
  rw [mem_eraseIdx_nodup i as n (Nat.succ_lt_succ_iff.mp hn) h.2]
  simp_all only [ne_eq]
  obtain (left, right)
```

lemma insertIdx_eq_take_drop

[\[source\]](#)

```

lemma insertIdx_eq_take_drop {I : Type} (i : I) : (r : List I) → (n : Fin r.length.succ) →
  List.insertIdx r n i = List.take n r ++ i :: r.drop n
| [], 0 => by simp
| a :: as, 0 => by
  simp
| a :: as, (n + 1, h) => by
  simp only [List.insertIdx_succ_cons, List.take_succ_cons, List.drop_succ_cons, List.
    cons_append,
    List.cons.injEq, true_and]
  exact insertIdx_eq_take_drop i as (n, Nat.succ_lt_succ_iff.mp h)

```

lemma insertIdx_length_fin

[\[source\]](#)

```

@[simp]
lemma insertIdx_length_fin {I : Type} (i : I) :
  (r : List I) → (n : Fin r.length.succ) →
  (List.insertIdx r n i).length = r.length.succ
| [], 0 => by simp
| a :: as, 0 => by simp
| a :: as, (n + 1, h) => by
  simp only [List.insertIdx_succ_cons, List.length_cons, Nat.succ_eq_add_one, add_left_inj]
  exact insertIdx_length_fin i as (n, Nat.succ_lt_succ_iff.mp h)

```

lemma insertIdx_getElem_fin

[\[source\]](#)

```

@[simp]
lemma insertIdx_getElem_fin {I : Type} (i : I) :
  (r : List I) → (k : Fin r.length.succ) → (m : Fin r.length) →
  (List.insertIdx r k i)[(k.succAbove m).val] = r[m.val]
| [], 0, m => by exact Fin.elim0 m
| a :: as, 0, m => by simp
| a :: as, (n + 1, h), (0, h0) => by
  simp [Fin.succAbove, Fin.lt_def]
| a :: as, (n + 1, h), (m+1, hm) => by
  simp only [List.insertIdx_succ_cons, List.length_cons, Nat.succ_eq_add_one,
    List.getElem_cons_succ]
  conv_rhs => rw [← insertIdx_getElem_fin i as (n, Nat.succ_lt_succ_iff.mp h)
    (m, Nat.lt_of_succ_lt_succ hm)]
  simp only [Fin.succAbove, Fin.castSucc_mk, Fin.lt_def, add_lt_add_iff_right, Fin.succ_mk,
    Nat.succ_eq_add_one]
  split
  · simp_all only [List.getElem_cons_succ]
  · simp_all only [List.getElem_cons_succ]

```

lemma insertIdx_eraseIdx_fin

[\[source\]](#)

```

lemma insertIdx_eraseIdx_fin {I : Type} :
  (r : List I) → (k : Fin r.length) →
  (List.eraseIdx r k).insertIdx k r[k] = r
| [], k => by exact Fin.elim0 k
| a :: as, (0, h) => by simp
| a :: as, (n + 1, h) => by
  simp only [List.length_cons, Fin.getElem_fin, List.getElem_cons_succ, List.
    eraseIdx_cons_succ,
    List.insertIdx_succ_cons, List.cons.injEq, true_and]

```

```
exact insertIdx_eraseIdx_fin as ⟨n, Nat.lt_of_succ_lt_succ h⟩
```

lemma insertIdx_length_fst_append[\[source\]](#)

```
lemma insertIdx_length_fst_append {I : Type} (φ : I) : (φs φs' : List I) →
  List.insertIdx (φs ++ φs') φs.length φ = (φs ++ φ :: φs')
| [], φs' => by simp
| φ' :: φs, φs' => by
  simp only [List.length_cons, List.cons_append, List.insertIdx_succ_cons, List.cons.injEq,
    true_and]
  exact insertIdx_length_fst_append φ φs φs'
```

lemma get_eq_insertIdx_succAbove[\[source\]](#)

```
lemma get_eq_insertIdx_succAbove {I : Type} (i : I) (r : List I) (k : Fin r.length.succ) :
  r.get = (List.insertIdx r k i).get ◦
  (finCongr (insertIdx_length_fin i r k).symm) ◦ k.succAbove
```

lemma take_insert_same[\[source\]](#)

```
lemma take_insert_same {I : Type} (i : I) :
  (n : ℕ) → (r : List I) →
  List.take n (List.insertIdx r n i) = List.take n r
| 0, _ => by simp
| _+1, [] => by simp
| n+1, a::as => by
  simp only [List.insertIdx_succ_cons, List.take_succ_cons, List.cons.injEq, true_and]
  exact take_insert_same i n as
```

lemma take_eraseIdx_same[\[source\]](#)

```
lemma take_eraseIdx_same {I : Type} :
  (n : ℕ) → (r : List I) →
  List.take n (List.eraseIdx r n) = List.take n r
| 0, _ => by simp
| _+1, [] => by simp
| n+1, a::as => by
  simp only [List.eraseIdx_cons_succ, List.take_succ_cons, List.cons.injEq, true_and]
  exact take_eraseIdx_same n as
```

lemma drop_eraseIdx_succ[\[source\]](#)

```
lemma drop_eraseIdx_succ {I : Type} :
  (n : ℕ) → (r : List I) → (hn : n < r.length) →
  r[n] :: List.drop n (List.eraseIdx r n) = List.drop n r
| 0, _, _ => by
  simp only [List.eraseIdx_zero, List.drop_tail, zero_add, List.drop_one, List.drop_zero]
  rw [List.getElem_zero]
  exact List.cons_head_tail _
| n+1, [], hn => by simp at hn
| n+1, a::as, hn => by
  simp only [List.getElem_cons_succ, List.eraseIdx_cons_succ, List.drop_succ_cons]
  refine drop_eraseIdx_succ n as _
```

lemma take_insert_gt[\[source\]](#)

```

lemma take_insert_gt {I : Type} (i : I) :
  (n m : ℕ) → (h : n < m) → (r : List I) →
  List.take n (List.insertIdx r m i) = List.take n r
| 0, 0, _, _ => by simp
| 0, m + 1, _, _ => by simp
| n + 1, m + 1, _, [] => by simp
| n + 1, m + 1, h, a::as => by
  simp only [List.insertIdx_succ_cons, List.take_succ_cons, List.cons.injEq, true_and]
  refine take_insert_gt i n m (Nat.succ_lt_succ_iff.mp h) as

```

lemma take_insert_let[\[source\]](#)

```

lemma take_insert_let {I : Type} (i : I) :
  (n m : ℕ) → (h : m ≤ n) → (r : List I) → (hm : m ≤ r.length) →
  (List.take (n + 1) (List.insertIdx r m i)).Perm (i :: List.take n r)
| 0, 0, h, _, _ => by simp
| m + 1, 0, h, r, _ => by simp
| n + 1, m + 1, h, [], hm => by
  simp at hm
| n + 1, m + 1, h, a::as, hm => by
  simp only [List.insertIdx_succ_cons, List.take_succ_cons]
  have hp : (i :: a :: List.take n as).Perm (a :: i :: List.take n as)

```

2.17 Physlib.Mathematics.List.InsertionSort

[Physlib/Mathematics/List/InsertionSort.lean](#) on GitHub.

lemma insertionSortMin_lt_length_succ[\[source\]](#)

```

lemma insertionSortMin_lt_length_succ {α : Type} (r : α → α → Prop) [DecidableRel r]
  (i : α) (l : List α) :
  insertionSortMinPos r i l < (insertionSortDropMinPos r i l).length.succ

```

def insertionSortMinPosFin[\[source\]](#)

Given a list $i :: l$ the left-most minimal position a of $i :: l$ wrt r as an element of $\text{Fin } (\text{insertionSortDropMinPos } r \ i \ l).length.succ$.

```

def insertionSortMinPosFin {α : Type} (r : α → α → Prop) [DecidableRel r] (i : α) (l : List α) :
  :
  Fin (insertionSortDropMinPos r i l).length.succ

```

lemma insertionSortMin_lt_mem_insertionSortDropMinPos[\[source\]](#)

```

lemma insertionSortMin_lt_mem_insertionSortDropMinPos {α : Type} (r : α → α → Prop) [
  DecidableRel r]
  [Std.Total r] [IsTrans α r] (a : α) (l : List α)
  (i : Fin (insertionSortDropMinPos r a l).length) :
  r (insertionSortMin r a l) ((insertionSortDropMinPos r a l)[i])

```

lemma insertionSortMinPos_insertionSortEquiv[\[source\]](#)

```
lemma insertionSortMinPos_insertionSortEquiv {α : Type} (r : α → α → Prop) [DecidableRel r]
  (a : α) (l : List α) :
  insertionSortEquiv r (a :: l) (insertionSortMinPos r a l) =
  ⟨0, by simp [List.orderedInsert_length]⟩
```

lemma insertionSortEquiv_gt_zero_of_ne_insertionSortMinPos

[\[source\]](#)

```
lemma insertionSortEquiv_gt_zero_of_ne_insertionSortMinPos {α : Type} (r : α → α → Prop)
  [DecidableRel r] (a : α) (l : List α) (k : Fin (a :: l).length)
  (hk : k ≠ insertionSortMinPos r a l) :
  ⟨0, by simp [List.orderedInsert_length]⟩ < insertionSortEquiv r (a :: l) k
```

lemma insertionSortMin_lt_mem_insertionSortDropMinPos_of_lt

[\[source\]](#)

```
lemma insertionSortMin_lt_mem_insertionSortDropMinPos_of_lt {α : Type} (r : α → α → Prop)
  [DecidableRel r] (a : α) (l : List α)
  (i : Fin (insertionSortDropMinPos r a l).length)
  (h : (insertionSortMinPosFin r a l).succAbove i < insertionSortMinPosFin r a l) :
  ¬ r ((insertionSortDropMinPos r a l)[i]) (insertionSortMin r a l)
```

lemma insertionSort_insertionSort

[\[source\]](#)

```
lemma insertionSort_insertionSort {α : Type} (r : α → α → Prop) [DecidableRel r]
  [Std.Total r] [IsTrans α r] (l1 : List α) :
  List.insertionSort r (List.insertionSort r l1) = List.insertionSort r l1
```

lemma orderedInsert_commute

[\[source\]](#)

```
lemma orderedInsert_commute {α : Type} (r : α → α → Prop) [DecidableRel r]
  [Std.Total r] [IsTrans α r] (a b : α) (hr : ¬ r a b) : (l : List α) →
  List.orderedInsert r a (List.orderedInsert r b l) =
  List.orderedInsert r b (List.orderedInsert r a l)
| [] => by
  have hrb : r b a
```

lemma insertionSort_orderedInsert_append

[\[source\]](#)

```
lemma insertionSort_orderedInsert_append {α : Type} (r : α → α → Prop) [DecidableRel r]
  [Std.Total r] [IsTrans α r] (a : α) : (l1 l2 : List α) →
  List.insertionSort r (List.orderedInsert r a l1 ++ l2) = List.insertionSort r (a :: l1 ++
  l2)
| [], l2 => by
  simp
| b :: l1, l2 => by
  conv_lhs => simp
  by_cases h : r a b
  · simp [h]
  conv_lhs => simp [h]
  rw [insertionSort_orderedInsert_append r a l1 l2]
  simp only [List.cons_append, List.insertionSort_cons]
  rw [orderedInsert_commute r a b h]
```


lemma insertionSort_insertionSort_append[\[source\]](#)

```

lemma insertionSort_insertionSort_append {α : Type} (r : α → α → Prop) [DecidableRel r]
  [Std.Total r] [IsTrans α r] : (l1 l2 : List α) →
  List.insertionSort r (List.insertionSort r l1 ++ l2) = List.insertionSort r (l1 ++ l2)
| [], l2 => by
  simp
| a :: l1, l2 => by
  conv_lhs => simp
  rw [insertionSort_orderedInsert_append]
  simp only [List.cons_append, List.insertionSort_cons]
  rw [insertionSort_insertionSort_append r l1 l2]

```

lemma insertionSort_append_insertionSort_append[\[source\]](#)

```

lemma insertionSort_append_insertionSort_append {α : Type} (r : α → α → Prop) [DecidableRel r]
  [Std.Total r] [IsTrans α r] : (l1 l2 l3 : List α) →
  List.insertionSort r (l1 ++ List.insertionSort r l2 ++ l3) =
  List.insertionSort r (l1 ++ l2 ++ l3)
| [], l2, l3 => by
  simp only [List.nil_append]
  exact insertionSort_insertionSort_append r l2 l3
| a :: l1, l2, l3 => by
  simp only [List.cons_append, List.insertionSort_cons]
  rw [insertionSort_append_insertionSort_append r l1 l2 l3]

```

lemma orderedInsert_length[\[source\]](#)

```

@[simp]
lemma orderedInsert_length {α : Type} (r : α → α → Prop) [DecidableRel r] (a : α) (l : List α)
  :
  (List.orderedInsert r a l).length = (a :: l).length

```

lemma takeWhile_orderedInsert[\[source\]](#)

```

lemma takeWhile_orderedInsert {α : Type} (r : α → α → Prop) [DecidableRel r]
  [Std.Total r] [IsTrans α r]
  (a b : α) (hr : ¬ r a b) : (l : List α) →
  (List.takeWhile (fun c => !decide (r a c)) (List.orderedInsert r b l)).length =
  (List.takeWhile (fun c => !decide (r a c)) l).length + 1
| [] => by
  simp [hr]
| c :: l => by
  simp only [List.orderedInsert]
  by_cases h : r b c
  · simp only [h, ↓reduceIte]
    rw [List.takeWhile_cons_of_pos]
    simp only [List.length_cons]
    simp [hr]
  · simp only [h, ↓reduceIte]
    have hrba : r b a

```

lemma takeWhile_orderedInsert'[\[source\]](#)

```

lemma takeWhile_orderedInsert' {α : Type} (r : α → α → Prop) [DecidableRel r]

```

```

[Std.Total r] [IsTrans α r]
(a b : α) (hr : ¬ r a b) : (l : List α) →
(List.takeWhile (fun c => !decide (r b c)) (List.orderedInsert r a l)).length =
(List.takeWhile (fun c => !decide (r b c)) l).length
| [] => by
simp only [List.orderedInsert, List.takeWhile_nil, List.length_nil, List.length_eq_zero_iff
,
List.takeWhile_eq_nil_iff, List.length_singleton, zero_lt_one, Fin.zero_eta, Fin.isValue,
List.get_eq_getElem, Fin.val_eq_zero, List.getElem_cons_zero, Bool.not_eq_eq_eq_not,
Bool.not_true, decide_eq_false_iff_not, Decidable.not_not, forall_const]
have ht

```

lemma insertionSortEquiv_commute[\[source\]](#)

```

lemma insertionSortEquiv_commute {α : Type} (r : α → α → Prop) [DecidableRel r]
[Std.Total r] [IsTrans α r] (a b : α) (hr : ¬ r a b) (n : ℕ) : (l : List α) →
(hn : n + 2 < (a :: b :: l).length) →
insertionSortEquiv r (a :: b :: l) (n + 2, hn) = (finCongr (by simp))
(insertionSortEquiv r (b :: a :: l) (n + 2, hn))

```

lemma insertionSortEquiv_orderedInsert_append[\[source\]](#)

```

lemma insertionSortEquiv_orderedInsert_append {α : Type} (r : α → α → Prop) [DecidableRel r]
[Std.Total r] [IsTrans α r] (a a2 : α) : (l1 l2 : List α) →
(insertionSortEquiv r (List.orderedInsert r a l1 ++ a2 :: l2) (l1.length + 1, by
simp))
= (finCongr (by
simp only [List.cons_append, List.insertionSort_cons, orderedInsert_length, List.
length_cons,
List.length_insertionSort, List.length_append]
omega))
((insertionSortEquiv r (a :: l1 ++ a2 :: l2)) (l1.length + 1, by simp))
| [], l2 => by
simp only [List.orderedInsert.eq_1, List.cons_append, List.nil_append, List.insertionSort,
List.length_cons, List.length_nil, zero_add, Fin.mk_one, finCongr_refl, Equiv.refl_apply]
| b :: l1, l2 => by
by_cases h : r a b
· have h1 : (List.orderedInsert r a (b :: l1) ++ a2 :: l2) = (a :: b :: l1 ++ a2 :: l2)

```

lemma insertionSortEquiv_insertionSort_append[\[source\]](#)

```

lemma insertionSortEquiv_insertionSort_append {α : Type} (r : α → α → Prop) [DecidableRel r]
[Std.Total r] [IsTrans α r] (a : α) : (l1 l2 : List α) →
(insertionSortEquiv r (List.insertionSort r l1 ++ a :: l2) (l1.length, by simp))
= finCongr (by simp) (insertionSortEquiv r (l1 ++ a :: l2) (l1.length, by simp))
| [], l2 => by
apply insertionSortEquiv_congr_apply
simp
| b :: l1, l2 => by
simp only [List.insertionSort, List.length_cons, List.cons_append, finCongr_apply]
have h1

```

lemma orderedInsert_filter_of_pos[\[source\]](#)

```

lemma orderedInsert_filter_of_pos {α : Type} (r : α → α → Prop) [DecidableRel r]

```

```

[IsTrans  $\alpha$  r] (a :  $\alpha$ ) (p :  $\alpha \rightarrow \text{Prop}$ ) [DecidablePred p] (h : p a) : (l : List  $\alpha$ )  $\rightarrow$ 
(hl : l.Pairwise r)  $\rightarrow$ 
List.filter p (List.orderedInsert r a l) = List.orderedInsert r a (List.filter p l)
| [], hl => by
  simp only [List.orderedInsert, List.filter_nil, List.orderedInsert_nil, List.filter_eq_self]
  '
  List.mem_singleton, decide_eq_true_eq, forall_eq]
exact h
| b :: l, hl => by
  simp only [List.orderedInsert]
  by_cases hpb : p b <=> by_cases hab : r a b
  · simp only [hab,  $\downarrow$ reduceIte, hpb, decide_true, List.filter_cons_of_pos,
    List.orderedInsert.eq_2]
    rw [List.filter_cons_of_pos (by simp [h])]
    rw [List.filter_cons_of_pos (by simp [hpb])]
  · simp only [hab,  $\downarrow$ reduceIte]
    rw [List.filter_cons_of_pos (by simp [hpb])]
    rw [List.filter_cons_of_pos (by simp [hpb])]
    simp only [List.orderedInsert, hab,  $\downarrow$ reduceIte, List.cons.injEq, true_and]
    simp only [List.pairwise_cons] at hl
    exact orderedInsert_filter_of_pos r a p h l hl.2
  · simp only [hab,  $\downarrow$ reduceIte]
    rw [List.filter_cons_of_pos (by simp [h]),
      List.filter_cons_of_neg (by simp [hpb])]
    rw [List.orderedInsert_eq_take_drop]
    have hl : List.takeWhile (fun b => decide  $\neg$  r a b)
      (List.filter (fun b => decide (p b)) l) = []

```

lemma orderedInsert_filter_of_neg[\[source\]](#)

```

lemma orderedInsert_filter_of_neg { $\alpha$  : Type} (r :  $\alpha \rightarrow \alpha \rightarrow \text{Prop}$ ) [DecidableRel r]
(a :  $\alpha$ ) (p :  $\alpha \rightarrow \text{Prop}$ ) [DecidablePred p] (h :  $\neg$  p a) (l : List  $\alpha$ ) :
List.filter p (List.orderedInsert r a l) = (List.filter p l)

```

lemma insertionSort_filter[\[source\]](#)

```

lemma insertionSort_filter { $\alpha$  : Type} (r :  $\alpha \rightarrow \alpha \rightarrow \text{Prop}$ ) [DecidableRel r] [Std.Total r]
[IsTrans  $\alpha$  r] (p :  $\alpha \rightarrow \text{Prop}$ ) [DecidablePred p] : (l : List  $\alpha$ )  $\rightarrow$ 
List.insertionSort r (List.filter p l) =
List.filter p (List.insertionSort r l)
| [] => by simp
| a :: l => by
  simp only [List.insertionSort_cons]
  by_cases h : p a
  · rw [orderedInsert_filter_of_pos]
    rw [List.filter_cons_of_pos]
    simp only [List.insertionSort_cons]
    rw [insertionSort_filter]
    simp_all only [decide_true]
    simp_all only
    exact List.pairwise_insertionSort r l
  · rw [orderedInsert_filter_of_neg]
    rw [List.filter_cons_of_neg]
    rw [insertionSort_filter]
    simp_all only [decide_false, Bool.false_eq_true, not_false_eq_true]
    exact h

```

lemma takeWhile_sorted_eq_filter[\[source\]](#)

```

lemma takeWhile_sorted_eq_filter {α : Type} (r : α → α → Prop) [DecidableRel r]
  [IsTrans α r] (a : α) : (l : List α) → (hl : l.Pairwise r) →
  List.takeWhile (fun c => ¬ r a c) l = List.filter (fun c => ¬ r a c) l
| [], _ => by simp
| b :: l, hl => by
  simp only [List.pairwise_cons] at hl
  by_cases hb : ¬ r a b
  · simp only [decide_not, hb, decide_false, Bool.not_false, List.takeWhile_cons_of_pos,
    List.filter_cons_of_pos, List.cons.injEq, true_and]
    simp using takeWhile_sorted_eq_filter r a l hl.2
  · simp_all only [Decidable.not_not, decide_not, decide_true, Bool.not_true, Bool.
    false_eq_true,
    not_false_eq_true, List.takeWhile_cons_of_neg, List.filter_cons_of_neg, List.nil_eq,
    List.filter_eq_nil_iff, Bool.not_eq_eq_eq_not, decide_eq_false_iff_not]
    intro c hc
    apply IsTrans.trans a b c hb
    exact hl.1 c hc

```

lemma dropWhile_sorted_eq_filter[\[source\]](#)

```

lemma dropWhile_sorted_eq_filter {α : Type} (r : α → α → Prop) [DecidableRel r]
  [IsTrans α r] (a : α) : (l : List α) → (hl : l.Pairwise r) →
  List.dropWhile (fun c => ¬ r a c) l = List.filter (fun c => r a c) l
| [], _ => by simp
| b :: l, hl => by
  simp only [List.pairwise_cons] at hl
  by_cases hb : ¬ r a b
  · simp only [decide_not, hb, decide_false, Bool.not_false, List.dropWhile_cons_of_pos,
    Bool.false_eq_true, not_false_eq_true, List.filter_cons_of_neg]
    simp using dropWhile_sorted_eq_filter r a l hl.2
  · simp_all only [Decidable.not_not, decide_not, decide_true, Bool.not_true, Bool.
    false_eq_true,
    not_false_eq_true, List.dropWhile_cons_of_neg, List.filter_cons_of_pos, List.cons.injEq,
    true_and]
    symm
    rw [List.filter_eq_self]
    intro c hc
    simp only [decide_eq_true_eq]
    apply IsTrans.trans a b c hb
    exact hl.1 c hc

```

lemma dropWhile_sorted_eq_filter_filter[\[source\]](#)

```

lemma dropWhile_sorted_eq_filter_filter {α : Type} (r : α → α → Prop) [DecidableRel r]
  [IsTrans α r] (a : α) : (l : List α) → (hl : l.Pairwise r) →
  List.filter (fun c => r a c) l =
  List.filter (fun c => r a c ∧ r c a) l ++ List.filter (fun c => r a c ∧ ¬ r c a) l
| [], _ => by
  simp
| b :: l, hl => by
  simp only [List.pairwise_cons] at hl
  by_cases hb : ¬ r a b
  · simp only [hb, decide_false, Bool.false_eq_true, not_false_eq_true, List.
    filter_cons_of_neg,
    Bool.decide_and, Bool.false_and, decide_not]

```

```

    simp using dropWhile_sorted_eq_filter_filter r a l hl.2
  · simp_all only [Decidable.not_not, decide_true, List.filter_cons_of_pos, Bool.decide_and,
    decide_not]
    by_cases hba : r b a
  · simp only [hba, decide_true, Bool.not_true, Bool.and_false, Bool.false_eq_true,
    not_false_eq_true, List.filter_cons_of_neg]
    rw [List.filter_cons_of_pos]
    rw [dropWhile_sorted_eq_filter_filter]
    simp only [Bool.decide_and, decide_not, List.cons_append]
    exact hl.2
    simp_all
  · simp only [hba, decide_false, Bool.and_false, Bool.false_eq_true, not_false_eq_true,
    List.filter_cons_of_neg]
    have hl : List.filter (fun c => decide (r a c) && decide (r c a)) l = []

```

lemma filter_rel_eq_insertionSort[\[source\]](#)

```

lemma filter_rel_eq_insertionSort {α : Type} (r : α → α → Prop) [DecidableRel r]
  [Std.Total r] [IsTrans α r] (a : α) : (l : List α) →
  List.filter (fun c => r a c ∧ r c a) (l.insertionSort r) =
  List.filter (fun c => r a c ∧ r c a) l
| [] => by simp
| b :: l => by
  simp only [List.insertionSort]
  by_cases h : r a b ∧ r b a
  · have hl

```

lemma insertionSort_of_eq_list[\[source\]](#)

```

lemma insertionSort_of_eq_list {α : Type} (r : α → α → Prop) [DecidableRel r]
  [Std.Total r] [IsTrans α r] (a : α) (l1 l2 : List α)
  (h : ∀ b ∈ l, r a b ∧ r b a) :
  List.insertionSort r (l1 ++ l ++ l2) =
  (List.takeWhile (fun c => ¬ r a c) ((l1 ++ l2).insertionSort r))
  ++ (List.filter (fun c => r a c ∧ r c a) l1)
  ++ l
  ++ (List.filter (fun c => r a c ∧ r c a) l2)
  ++ (List.filter (fun c => r a c ∧ ¬ r c a) ((l1 ++ l2).insertionSort r))

```

lemma insertionSort_of_takeWhile_filter[\[source\]](#)

```

lemma insertionSort_of_takeWhile_filter {α : Type} (r : α → α → Prop) [DecidableRel r]
  [Std.Total r] [IsTrans α r] (a : α) (l1 l2 : List α) :
  List.insertionSort r (l1 ++ l2) =
  (List.takeWhile (fun c => ¬ r a c) ((l1 ++ l2).insertionSort r))
  ++ (List.filter (fun c => r a c ∧ r c a) l1)
  ++ (List.filter (fun c => r a c ∧ r c a) l2)
  ++ (List.filter (fun c => r a c ∧ ¬ r c a) ((l1 ++ l2).insertionSort r))

```

2.18 Physlib.Mathematics.PiTensorProduct

[Physlib/Mathematics/PiTensorProduct.lean](#) on GitHub.

lemma induction_tm1[\[source\]](#)

```

lemma induction_tmul {f g : (( $\bigotimes_{[R]}$  i :  $\mathfrak{v}1$ , s1 i)  $\otimes_{[R]}$  ( $\bigotimes_{[R]}$  i :  $\mathfrak{v}2$ , s2 i))  $\rightarrow_l$ [R] M}
  (h :  $\forall$  p q, f (PiTensorProduct.tprod R p  $\otimes_t$ [R] PiTensorProduct.tprod R q)
    = g (PiTensorProduct.tprod R p  $\otimes_t$ [R] PiTensorProduct.tprod R q)) : f = g

```

lemma induction_assoc

[\[source\]](#)

```

lemma induction_assoc
  {f g : (( $\bigotimes_{[R]}$  i :  $\mathfrak{v}1$ , s1 i)  $\otimes_{[R]}$  ( $\bigotimes_{[R]}$  i :  $\mathfrak{v}2$ , s2 i)  $\otimes_{[R]}$  ( $\bigotimes_{[R]}$  i :  $\mathfrak{v}3$ , s3 i))  $\rightarrow_l$ [R] M}
  (h :  $\forall$  p q m, f (PiTensorProduct.tprod R p  $\otimes_t$ [R]
    PiTensorProduct.tprod R q  $\otimes_t$ [R] PiTensorProduct.tprod R m)
    = g (PiTensorProduct.tprod R p  $\otimes_t$ [R] PiTensorProduct.tprod R q
       $\otimes_t$ [R] PiTensorProduct.tprod R m)) : f = g

```

lemma induction_assoc'

[\[source\]](#)

```

lemma induction_assoc'
  {f g : ((( $\bigotimes_{[R]}$  i :  $\mathfrak{v}1$ , s1 i)  $\otimes_{[R]}$  ( $\bigotimes_{[R]}$  i :  $\mathfrak{v}2$ , s2 i))  $\otimes_{[R]}$  ( $\bigotimes_{[R]}$  i :  $\mathfrak{v}3$ , s3 i))  $\rightarrow_l$ [R] M}
  (h :  $\forall$  p q m, f ((PiTensorProduct.tprod R p  $\otimes_t$ [R] PiTensorProduct.tprod R q)  $\otimes_t$ [R]
    PiTensorProduct.tprod R m) = g ((PiTensorProduct.tprod R p  $\otimes_t$ [R] PiTensorProduct.tprod R q)
       $\otimes_t$ [R] PiTensorProduct.tprod R m)) : f = g

```

lemma induction_tmul_mod

[\[source\]](#)

```

lemma induction_tmul_mod
  {f g : (( $\bigotimes_{[R]}$  i :  $\mathfrak{v}1$ , s1 i)  $\otimes_{[R]}$  N)  $\rightarrow_l$ [R] M}
  (h :  $\forall$  p m, f (PiTensorProduct.tprod R p  $\otimes_t$ [R] m) = g (PiTensorProduct.tprod R p  $\otimes_t$ [R] m))
  :
  f = g

```

lemma induction_mod_tmul

[\[source\]](#)

```

lemma induction_mod_tmul
  {f g : (N  $\otimes_{[R]}$  ( $\bigotimes_{[R]}$  i :  $\mathfrak{v}1$ , s1 i))  $\rightarrow_l$ [R] M}
  (h :  $\forall$  m p, f (m  $\otimes_t$ [R] PiTensorProduct.tprod R p) = g (m  $\otimes_t$ [R] PiTensorProduct.tprod R p))
  :
  f = g

```

instance (i : $\mathfrak{v}1 \otimes \mathfrak{v}2$) $\rightarrow$ AddCommMonoid ((fun i => Sum.elim s1 s2 i) i)

[\[source\]](#)

Given two maps s1 and s2 whose targets carry an instance of an additive commutative monoid, the target of the sum of these two maps also carry an instance thereof.

```

instance : (i :  $\mathfrak{v}1 \otimes \mathfrak{v}2$ )  $\rightarrow$  AddCommMonoid ((fun i => Sum.elim s1 s2 i) i)

```

instance (i : $\mathfrak{v}1 \otimes \mathfrak{v}2$) $\rightarrow$ Module R ((fun i => Sum.elim s1 s2 i) i)

[\[source\]](#)

Given two maps s1 and s2 whose targets carry an instance of a module over R, the target of the sum of these two maps also carry an instance thereof.

```

instance : (i :  $\mathfrak{v}1 \otimes \mathfrak{v}2$ )  $\rightarrow$  Module R ((fun i => Sum.elim s1 s2 i) i)

```

def pureInl[\[source\]](#)

Takes a map $(i : \iota_1 \oplus \iota_2) \rightarrow \text{Sum.elim } s_1 \ s_2 \ i$ to the underlying map $(i : \iota_1) \rightarrow s_1 \ i$.

```
def pureInl (f : (i :  $\iota_1 \oplus \iota_2$ )  $\rightarrow$  Sum.elim s1 s2 i) : (i :  $\iota_1$ )  $\rightarrow$  s1 i
```

def pureInr[\[source\]](#)

Takes a map $(i : \iota_1 \oplus \iota_2) \rightarrow \text{Sum.elim } s_1 \ s_2 \ i$ to the underlying map $(i : \iota_2) \rightarrow s_2 \ i$.

```
def pureInr (f : (i :  $\iota_1 \oplus \iota_2$ )  $\rightarrow$  Sum.elim s1 s2 i) : (i :  $\iota_2$ )  $\rightarrow$  s2 i
```

lemma pureInl_update_left[\[source\]](#)

```
lemma pureInl_update_left [DecidableEq  $\iota_1$ ] (f : (i :  $\iota_1 \oplus \iota_2$ )  $\rightarrow$  Sum.elim s1 s2 i) (x :  $\iota_1$ )
  (v1 : s1 x) : pureInl (Function.update f (Sum.inl x) v1) =
  Function.update (pureInl f) x v1
```

lemma pureInr_update_left[\[source\]](#)

```
lemma pureInr_update_left (f : (i :  $\iota_1 \oplus \iota_2$ )  $\rightarrow$  Sum.elim s1 s2 i) (x :  $\iota_1$ )
  (v2 : s1 x) :
  pureInr (Function.update f (Sum.inl x) v2) = (pureInr f)
```

lemma pureInr_update_right[\[source\]](#)

```
lemma pureInr_update_right [DecidableEq  $\iota_2$ ] (f : (i :  $\iota_1 \oplus \iota_2$ )  $\rightarrow$  Sum.elim s1 s2 i) (x :  $\iota_2$ )
  (v2 : s2 x) : pureInr (Function.update f (Sum.inr x) v2) =
  Function.update (pureInr f) x v2
```

lemma pureInl_update_right[\[source\]](#)

```
lemma pureInl_update_right (f : (i :  $\iota_1 \oplus \iota_2$ )  $\rightarrow$  Sum.elim s1 s2 i) (x :  $\iota_2$ )
  (v1 : s2 x) :
  pureInl (Function.update f (Sum.inr x) v1) = (pureInl f)
```

def domCoproduct[\[source\]](#)

The multilinear map from $(\text{Sum.elim } s_1 \ s_2)$ to $((\bigotimes_{[R]} i : \iota_1, s_1 \ i) \otimes_{[R]} \bigotimes_{[R]} i : \iota_2, s_2 \ i)$ defined by splitting elements of $(\text{Sum.elim } s_1 \ s_2)$ into two parts.

```
def domCoproduct :
  MultilinearMap R (Sum.elim s1 s2) (( $\bigotimes_{[R]} i : \iota_1, s_1 \ i$ )  $\otimes_{[R]}$  ( $\bigotimes_{[R]} i : \iota_2, s_2 \ i$ )) where
```

def tmulSymm[\[source\]](#)

Expand PiTensorProduct on sums into a TensorProduct of two factors.

```
def tmulSymm : ( $\bigotimes_{[R]} i : \iota_1 \oplus \iota_2, (\text{Sum.elim } s_1 \ s_2) \ i$ )  $\rightarrow_{\iota} [R]$ 
  (( $\bigotimes_{[R]} i : \iota_1, s_1 \ i$ )  $\otimes_{[R]}$  ( $\bigotimes_{[R]} i : \iota_2, s_2 \ i$ ))
```

def elimPureTensor[\[source\]](#)

Produces a map $(i : \iota_1 \oplus \iota_2) \rightarrow \text{Sum.elim } s_1 \ s_2 \ i$ from a map $(i : \iota_1) \rightarrow s_1 \ i$ and a map $q : (i : \iota_2) \rightarrow s_2 \ i$.

```
def elimPureTensor (p : (i :  $\iota_1$ )  $\rightarrow$  s1 i) (q : (i :  $\iota_2$ )  $\rightarrow$  s2 i) : (i :  $\iota_1 \oplus \iota_2$ )  $\rightarrow$  Sum.elim s1 s2 i
```

lemma elimPureTensor_update_right[\[source\]](#)

```
lemma elimPureTensor_update_right (p : (i :  $\iota_1$ )  $\rightarrow$  s1 i) (q : (i :  $\iota_2$ )  $\rightarrow$  s2 i)
  (y :  $\iota_2$ ) (r : s2 y) : elimPureTensor p (Function.update q y r) =
  Function.update (elimPureTensor p q) (Sum.inr y) r
```

lemma elimPureTensor_update_left[\[source\]](#)

```
@[simp]
lemma elimPureTensor_update_left (p : (i :  $\iota_1$ )  $\rightarrow$  s1 i) (q : (i :  $\iota_2$ )  $\rightarrow$  s2 i)
  (x :  $\iota_1$ ) (r : s1 x) : elimPureTensor (Function.update p x r) q =
  Function.update (elimPureTensor p q) (Sum.inl x) r
```

def elimPureTensorMullin[\[source\]](#)

The multilinear map valued in multilinear maps defined by combining $(i : \iota_1) \rightarrow s_1 \ i$ and $q : (i : \iota_2) \rightarrow s_2 \ i$ into a PiTensorProduct .

```
def elimPureTensorMullin : MultilinearMap R s1
  (MultilinearMap R s2 ( $\bigotimes$ [R] i :  $\iota_1 \oplus \iota_2$ , (Sum.elim s1 s2) i)) where
```

def tmul[\[source\]](#)

Collapse a TensorProduct of PiTensorProduct into a PiTensorProduct .

```
def tmul : (( $\bigotimes$ [R] i :  $\iota_1$ , s1 i)  $\otimes$ [R] ( $\bigotimes$ [R] i :  $\iota_2$ , s2 i))  $\rightarrow_l$  [R]
  ( $\bigotimes$ [R] i :  $\iota_1 \oplus \iota_2$ , (Sum.elim s1 s2) i)
```

def tmulEquiv[\[source\]](#)

The equivalence formed by combining a TensorProduct into a PiTensorProduct .

```
def tmulEquiv : (( $\bigotimes$ [R] i :  $\iota_1$ , s1 i)  $\otimes$ [R] ( $\bigotimes$ [R] i :  $\iota_2$ , s2 i))  $\approx_l$  [R]
  ( $\bigotimes$ [R] i :  $\iota_1 \oplus \iota_2$ , (Sum.elim s1 s2) i)
```

lemma tmulEquiv_tmul_tprod[\[source\]](#)

```
@[simp]
lemma tmulEquiv_tmul_tprod (p : (i :  $\iota_1$ )  $\rightarrow$  s1 i) (q : (i :  $\iota_2$ )  $\rightarrow$  s2 i) :
  tmulEquiv ((PiTensorProduct.tprod R) p  $\otimes$ [R] (PiTensorProduct.tprod R) q) =
  (PiTensorProduct.tprod R) (elimPureTensor p q)
```

2.19 Physlib.Mathematics.RatComplexNum

[Physlib/Mathematics/RatComplexNum.lean](#) on GitHub.

structure RatComplexNum[\[source\]](#)

Rational complex numbers. This type is mainly used when decidability is needed.

```
structure RatComplexNum where
  fst : ℚ
  snd : ℚ
```

lemma ext[\[source\]](#)

```
@[ext]
lemma ext {x y : RatComplexNum} (h1 : x.1 = y.1) (h2 : x.2 = y.2) : x = y
```

def equivToProd[\[source\]](#)

The equivalence as a type of RatComplexNum with $\mathbb{Q} \times \mathbb{Q}$.

```
def equivToProd : RatComplexNum ≈ ℚ × ℚ where
```

instance DecidableEq RatComplexNum[\[source\]](#)

```
instance : DecidableEq RatComplexNum
```

instance Add RatComplexNum[\[source\]](#)

```
instance : Add RatComplexNum where
```

lemma add_fst[\[source\]](#)

```
@[simp]
lemma add_fst (x y : RatComplexNum) : (x + y).fst = x.fst + y.fst
```

lemma add_snd[\[source\]](#)

```
@[simp]
lemma add_snd (x y : RatComplexNum) : (x + y).snd = x.snd + y.snd
```

instance AddCommGroup RatComplexNum[\[source\]](#)

```
instance : AddCommGroup RatComplexNum where
```

instance Mul RatComplexNum[\[source\]](#)

```
instance : Mul RatComplexNum where
```

lemma mul_fst[\[source\]](#)

```
@[simp]
lemma mul_fst (x y : RatComplexNum) : (x * y).fst = x.fst * y.fst - x.snd * y.snd
```

lemma mul_snd[\[source\]](#)`@[simp]``lemma mul_snd (x y : RatComplexNum) : (x * y).snd = x.fst * y.snd + x.snd * y.fst`**instance Ring RatComplexNum**[\[source\]](#)`instance : Ring RatComplexNum where`**lemma one\_fst**[\[source\]](#)`@[simp]``lemma one_fst : (1 : RatComplexNum).fst = 1`**lemma one\_snd**[\[source\]](#)`@[simp]``lemma one_snd : (1 : RatComplexNum).snd = 0`**lemma zero\_fst**[\[source\]](#)`@[simp]``lemma zero_fst : (0 : RatComplexNum).fst = 0`**lemma zero\_snd**[\[source\]](#)`@[simp]``lemma zero_snd : (0 : RatComplexNum).snd = 0`**def toComplexNum**[\[source\]](#)*The inclusion of RatComplexNum into the complex numbers.*`noncomputable def toComplexNum : RatComplexNum →+* ℂ where`**lemma I\_mul\_toComplexNum**[\[source\]](#)`@[simp]``lemma I_mul_toComplexNum (a : RatComplexNum) : I * toComplexNum a = toComplexNum ((0, 1) * a)`**lemma ofNat\_mul\_toComplexNum**[\[source\]](#)`lemma ofNat_mul_toComplexNum (n : ℕ) (a : RatComplexNum) :
 n * toComplexNum a = toComplexNum (n * a)`**lemma toComplexNum\_injective**[\[source\]](#)`lemma toComplexNum_injective : Function.Injective toComplexNum`

2.20 Physlib.Mathematics.SpecialFunctions.PhysHermite

[Physlib/Mathematics/SpecialFunctions/PhysHermite.lean](#) on GitHub.

def physHermite

[\[source\]](#)

The Physicists Hermite polynomial are defined as polynomials over $\mathbb{Z}$ in X recursively with $\text{physHermite } 0 = 1$ and

$\text{physHermite } (n + 1) = 2 \cdot X \cdot \text{physHermite } n - \text{derivative } (\text{physHermite } n).$

This polynomial will often be cast as a function $\mathbb{R} \rightarrow \mathbb{R}$ by evaluating the polynomial at X .

```
noncomputable def physHermite :  $\mathbb{N} \rightarrow \text{Polynomial } \mathbb{Z}$ 
| 0 => 1
| n + 1 => 2 * X * physHermite n - derivative (physHermite n)
```

lemma physHermite_succ

[\[source\]](#)

```
lemma physHermite_succ (n :  $\mathbb{N}$ ) :
  physHermite (n + 1) = 2 * X * physHermite n - derivative (physHermite n)
```

lemma physHermite_eq_iterate

[\[source\]](#)

```
lemma physHermite_eq_iterate (n :  $\mathbb{N}$ ) :
  physHermite n = (fun p => 2 * X * p - derivative p)^[n] 1
```

lemma physHermite_zero

[\[source\]](#)

```
@[simp]
lemma physHermite_zero : physHermite 0 = C 1
```

lemma physHermite_one

[\[source\]](#)

```
lemma physHermite_one : physHermite 1 = 2 * X
```

lemma derivative_physHermite_succ

[\[source\]](#)

```
lemma derivative_physHermite_succ : (n :  $\mathbb{N}$ ) →
  derivative (physHermite (n + 1)) = 2 * (n + 1) • physHermite n
| 0 => by
  simp [physHermite_one]
| n + 1 => by
  rw [physHermite_succ]
  simp only [nsmul_eq_mul, Nat.cast_ofNat, derivative_sub, derivative_mul, derivative_ofNat,
    zero_mul, derivative_X, mul_one, zero_add, Nat.cast_add, Nat.cast_one]
  rw [derivative_physHermite_succ]
  simp only [physHermite_succ, nsmul_eq_mul, Nat.cast_ofNat, Nat.cast_add, Nat.cast_one,
    derivative_mul, derivative_ofNat, zero_mul, derivative_add, derivative_natCast,
    derivative_one, add_zero, zero_add]
  ring
```

lemma derivative_physHermite[\[source\]](#)

```

lemma derivative_physHermite : (n : ℕ) →
  derivative (physHermite n) = 2 * n • physHermite (n - 1)
| 0 => by simp
| n + 1 => by simp [derivative_physHermite_succ]

```

lemma physHermite_succ'[\[source\]](#)

```

lemma physHermite_succ' (n : ℕ) :
  physHermite (n + 1) = 2 • X * physHermite n - 2 * n • physHermite (n - 1)

```

lemma coeff_physHermite_succ_zero[\[source\]](#)

```

lemma coeff_physHermite_succ_zero (n : ℕ) :
  coeff (physHermite (n + 1)) 0 = - coeff (physHermite n) 1

```

lemma coeff_physHermite_succ_succ[\[source\]](#)

```

lemma coeff_physHermite_succ_succ (n k : ℕ) : coeff (physHermite (n + 1)) (k + 1) =
  2 * coeff (physHermite n) k - (k + 2) * coeff (physHermite n) (k + 2)

```

lemma coeff_physHermite_of_lt[\[source\]](#)

```

lemma coeff_physHermite_of_lt {n k : ℕ} (h : n < k) : coeff (physHermite n) k = 0

```

lemma coeff_physHermite_self_succ[\[source\]](#)

```

@[simp]
lemma coeff_physHermite_self_succ (n : ℕ) : coeff (physHermite n) n = 2 ^ n

```

lemma degree_physHermite[\[source\]](#)

```

@[simp]
lemma degree_physHermite (n : ℕ) : degree (physHermite n) = n

```

lemma natDegree_physHermite[\[source\]](#)

```

@[simp]
lemma natDegree_physHermite {n : ℕ} : (physHermite n).natDegree = n

```

lemma iterate_derivative_physHermite_of_gt[\[source\]](#)

```

lemma iterate_derivative_physHermite_of_gt {n m : ℕ} (h : n < m) :
  derivative^[m] (physHermite n) = 0

```

lemma iterate_derivative_physHermite_self[\[source\]](#)

```

@[simp]
lemma iterate_derivative_physHermite_self {n : ℕ} :
  derivative^[n] (physHermite n) = C ((n ! : ℤ) * 2 ^ n)

```

lemma physHermite_leadingCoeff[\[source\]](#)

```
@[simp]
lemma physHermite_leadingCoeff {n : ℕ} : (physHermite n).leadingCoeff = 2 ^ n
```

lemma physHermite_ne_zero[\[source\]](#)

```
@[simp]
lemma physHermite_ne_zero {n : ℕ} : physHermite n ≠ 0
```

instance CoeFun (Polynomial ℤ) (fun _ ↦ ℝ → ℝ)[\[source\]](#)

```
noncomputable instance : CoeFun (Polynomial ℤ) (fun _ ↦ ℝ → ℝ) where
```

lemma physHermite_eq_aeval[\[source\]](#)

```
lemma physHermite_eq_aeval (n : ℕ) (x : ℝ) :
  physHermite n x = (physHermite n).aeval x
```

lemma physHermite_zero_apply[\[source\]](#)

```
lemma physHermite_zero_apply (x : ℝ) : physHermite 0 x = 1
```

lemma physHermite_pow[\[source\]](#)

```
lemma physHermite_pow (n m : ℕ) (x : ℝ) : physHermite n x ^ m = aeval x (physHermite n ^ m)
```

lemma physHermite_succ_fun[\[source\]](#)

```
lemma physHermite_succ_fun (n : ℕ) :
  (physHermite (n + 1) : ℝ → ℝ) = 2 • (fun x => x) *
  (physHermite n : ℝ → ℝ) - (2 * n : ℝ) • (physHermite (n - 1) : ℝ → ℝ)
```

lemma physHermite_succ_fun'[\[source\]](#)

```
lemma physHermite_succ_fun' (n : ℕ) :
  (physHermite (n + 1) : ℝ → ℝ) = fun x => 2 • x *
  physHermite n x -
  (2 * n : ℝ) • physHermite (n - 1) x
```

lemma iterated_deriv_physHermite_eq_aeval[\[source\]](#)

```
lemma iterated_deriv_physHermite_eq_aeval (n : ℕ) : (m : ℕ) →
  deriv^[m] (physHermite n) = fun x => (derivative^[m] (physHermite n)).aeval x
| 0 => by simp
| m + 1 => by
  simp only [Function.iterate_succ_apply']
  rw [iterated_deriv_physHermite_eq_aeval n m]
  funext x
  rw [Polynomial.deriv_aeval]
```

lemma physHermite_differentiableAt[\[source\]](#)

```
@[fun_prop]
lemma physHermite_differentiableAt (n : ℕ) (x : ℝ) :
  DifferentiableAt ℝ (physHermite n) x
```

lemma deriv_physHermite_differentiableAt[\[source\]](#)

```
@[fun_prop]
lemma deriv_physHermite_differentiableAt (n m : ℕ) (x : ℝ) :
  DifferentiableAt ℝ (deriv^[m] (physHermite n)) x
```

lemma deriv_physHermite[\[source\]](#)

```
lemma deriv_physHermite (n : ℕ) :
  deriv (physHermite n) = 2 * n * (physHermite (n - 1))
```

lemma fderiv_physHermite[\[source\]](#)

```
lemma fderiv_physHermite
  {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E] (x : E)
  (f : E → ℝ) (hf : DifferentiableAt ℝ f x) (n : ℕ) :
  fderiv ℝ (fun x => physHermite n (f x)) x
  = (2 * n * physHermite (n - 1) (f x)) • fderiv ℝ f x
```

lemma deriv_physHermite'[\[source\]](#)

```
@[simp]
lemma deriv_physHermite' (x : ℝ)
  (f : ℝ → ℝ) (hf : DifferentiableAt ℝ f x) (n : ℕ) :
  deriv (fun x => physHermite n (f x)) x
  = (2 * n * physHermite (n - 1) (f x)) * deriv f x
```

lemma physHermite_parity[\[source\]](#)

```
lemma physHermite_parity: (n : ℕ) → (x : ℝ) →
  physHermite n (-x) = (-1)^n * physHermite n x
| 0, x => by simp
| 1, x => by
  simp [physHermite_one, aeval, aevalEquiv]
| n + 2, x => by
  rw [physHermite_succ_fun']
  have h1
```

lemma deriv_gaussian_eq_physHermite_mul_gaussian[\[source\]](#)

```
lemma deriv_gaussian_eq_physHermite_mul_gaussian (n : ℕ) (x : ℝ) :
  deriv^[n] (fun y => Real.exp (- y ^ 2)) x =
  (-1 : ℝ) ^ n * physHermite n x * Real.exp (- x ^ 2)
```

lemma physHermite_eq_deriv_gaussian[\[source\]](#)

```

lemma physHermite_eq_deriv_gaussian (n : ℕ) (x : ℝ) :
  physHermite n x = (-1 : ℝ) ^ n * deriv^[n]
    (fun y => Real.exp (- y ^ 2)) x / Real.exp (- x ^ 2)

```

lemma physHermite_eq_deriv_gaussian'

[\[source\]](#)

```

lemma physHermite_eq_deriv_gaussian' (n : ℕ) (x : ℝ) :
  physHermite n x = (-1 : ℝ) ^ n * deriv^[n] (fun y => Real.exp (- y ^ 2)) x *
    Real.exp (x ^ 2)

```

lemma gaussian_integrable_polynomial

[\[source\]](#)

```

@[fun_prop]
lemma gaussian_integrable_polynomial {b : ℝ} (hb : 0 < b) (P : Polynomial ℤ) :
  MeasureTheory.Integrable fun x : ℝ => (P.aeval x) * Real.exp (-b * x ^ 2)

```

lemma gaussian_integrable_polynomial_cons

[\[source\]](#)

```

@[fun_prop]
lemma gaussian_integrable_polynomial_cons {b c : ℝ} (hb : 0 < b) (P : Polynomial ℤ) :
  MeasureTheory.Integrable fun x : ℝ => (P.aeval (c * x)) * Real.exp (-b * x ^ 2)

```

lemma physHermite_gaussian_integrable

[\[source\]](#)

```

@[fun_prop]
lemma physHermite_gaussian_integrable (n p m : ℕ) :
  MeasureTheory.Integrable (deriv^[m] (physHermite p) * deriv^[n] fun x => Real.exp (-x ^ 2))
  MeasureTheory.volume

```

lemma integral_physHermite_mul_physHermite_eq_integral_deriv_exp

[\[source\]](#)

```

lemma integral_physHermite_mul_physHermite_eq_integral_deriv_exp (n m : ℕ) :
  ∫ x : ℝ, (physHermite n x * physHermite m x) * Real.exp (-x ^ 2) =
    (-1 : ℝ) ^ m * ∫ x : ℝ, (physHermite n x * (deriv^[m] fun x => Real.exp (-x ^ 2)) x)

```

lemma integral_physHermite_mul_physHermite_eq_integral_deriv_inductive

[\[source\]](#)

```

lemma integral_physHermite_mul_physHermite_eq_integral_deriv_inductive (n m : ℕ) :
  (p : ℕ) → (hpm : p ≤ m) →
  ∫ x : ℝ, (physHermite n x * physHermite m x) * Real.exp (- x ^ 2) =
  (-1 : ℝ) ^ (m - p) * ∫ x : ℝ, (deriv^[p] (physHermite n) x *
    (deriv^[m - p] fun x => Real.exp (-x ^ 2)) x)
| 0, h => by
  exact integral_physHermite_mul_physHermite_eq_integral_deriv_exp n m
| p + 1, h => by
  rw [integral_physHermite_mul_physHermite_eq_integral_deriv_inductive n m p (by omega)]
  have h1 : m - p = m - (p + 1) + 1

```

lemma integral_physHermite_mul_physHermite_eq_integral_deriv

[\[source\]](#)

```

lemma integral_physHermite_mul_physHermite_eq_integral_deriv (n m : ℕ) :
  ∫ x : ℝ, (physHermite n x * physHermite m x) * Real.exp (- x ^ 2) =

```

```
∫ x : ℝ, (deriv^[m] (physHermite n) x * (Real.exp (-x ^ 2)))
```

lemma physHermite_orthogonal_lt[\[source\]](#)

```
lemma physHermite_orthogonal_lt {n m : ℕ} (hnm : n < m) :
  ∫ x : ℝ, (physHermite n x * physHermite m x) * Real.exp (- x ^ 2) = 0
```

theorem physHermite_orthogonal[\[source\]](#)

```
theorem physHermite_orthogonal {n m : ℕ} (hnm : n ≠ m) :
  ∫ x : ℝ, (physHermite n x * physHermite m x) * Real.exp (- x ^ 2) = 0
```

lemma physHermite_orthogonal_cons[\[source\]](#)

```
lemma physHermite_orthogonal_cons {n m : ℕ} (hnm : n ≠ m) (c : ℝ) :
  ∫ x : ℝ, (physHermite n (c * x) * physHermite m (c * x)) *
    Real.exp (- c ^ 2 * x ^ 2) = 0
```

theorem physHermite_norm[\[source\]](#)

```
theorem physHermite_norm (n : ℕ) :
  ∫ x : ℝ, (physHermite n x * physHermite n x) * Real.exp (- x ^ 2) =
    ↑n ! * 2 ^ n * √Real.pi
```

lemma physHermite_norm_cons[\[source\]](#)

```
lemma physHermite_norm_cons (n : ℕ) (c : ℝ) :
  ∫ x : ℝ, (physHermite n (c * x) * physHermite n (c * x)) * Real.exp (- c ^ 2 * x ^ 2) =
    |c-1| * (↑n ! * 2 ^ n * √Real.pi)
```

lemma polynomial_mem_physHermite_span_induction[\[source\]](#)

```
lemma polynomial_mem_physHermite_span_induction (P : Polynomial ℤ) : (n : ℕ) →
  (hn : P.natDegree = n) →
  (P : ℝ → ℝ) ∈ Submodule.span ℝ (Set.range (fun n => (physHermite n : ℝ → ℝ)))
| 0, h => by
  rw [natDegree_eq_zero] at h
  obtain ⟨x, rfl⟩
```

lemma polynomial_mem_physHermite_span[\[source\]](#)

```
lemma polynomial_mem_physHermite_span (P : Polynomial ℤ) :
  (P : ℝ → ℝ) ∈ Submodule.span ℝ (Set.range (fun n => (physHermite n : ℝ → ℝ)))
```

lemma cos_mem_physHermite_span_topologicalClosure[\[source\]](#)

```
lemma cos_mem_physHermite_span_topologicalClosure (c : ℝ) :
  (fun (x : ℝ) => Real.cos (c * x)) ∈
  (Submodule.span ℝ (Set.range (fun n => (physHermite n : ℝ → ℝ)))).topologicalClosure
```


2.21 Physlib.Meta.AllFilePaths

[Physlib/Meta/AllFilePaths.lean](#) on GitHub.

def allPhyslibModules [\[source\]](#)

Gets an array of all module names in Physlib.

```
def allPhyslibModules : IO (Array Lean.Name)
```

2.22 Physlib.Meta.Basic

[Physlib/Meta/Basic.lean](#) on GitHub.

def allImports [\[source\]](#)

Gets all imports within Physlib.

```
def Physlib.allImports : IO (Array Import)
```

def noImports [\[source\]](#)

Number of files within Physlib.

```
def Physlib.noImports : IO Nat
```

def getConsts [\[source\]](#)

Gets all constants from an import.

```
def Physlib.Imports.getConsts (imp : Import) : IO (Array ConstantInfo)
```

def getUserConsts [\[source\]](#)

Gets all user defined constants from an import.

```
def Physlib.Imports.getUserConsts (imp : Import) : CoreM (Array ConstantInfo)
```

def getLines [\[source\]](#)

Lines from import.

```
def Physlib.Imports.getLines (imp : Import) : IO (Array String)
```

def noDefs [\[source\]](#)

Number of definitions.

```
def noDefs : CoreM Nat
```

def noLemmas [\[source\]](#)

Number of definitions.

```
def noLemmas : CoreM Nat
```

def allDocStrings [\[source\]](#)

All docstrings present in Physlib.

```
def allDocStrings : CoreM (Array String)
```

def noDefsNoDocString [\[source\]](#)

Number of definitions without a doc-string.

```
def noDefsNoDocString : CoreM Nat
```

def noLemmasNoDocString [\[source\]](#)

Number of definitions without a doc-string.

```
def noLemmasNoDocString : CoreM Nat
```

def noLines [\[source\]](#)

The number of lines in Physlib.

```
def noLines : IO Nat
```

def noTODOs [\[source\]](#)

The number of TODO items.

```
def noTODOs : IO Nat
```

def noFilesWithTODOs [\[source\]](#)

The number of files with a TODO item.

```
def noFilesWithTODOs : IO Nat
```

def allUserConsts [\[source\]](#)

All user defined declarations.

```
def allUserConsts : CoreM (Array ConstantInfo)
```

2.23 Physlib.Meta.Informal.Post

[Physlib/Meta/Informal/Post.lean](#) on GitHub.

def noInformalLemmas [\[source\]](#)

The number of informal lemmas in Physlib.

```
def noInformalLemmas : CoreM Nat
```

def noInformalDefs[\[source\]](#)*The number of informal definitions in Physlib.*

```
def noInformalDefs : CoreM Nat
```

def AllInformal[\[source\]](#)*All informal results.*

```
def AllInformal : CoreM (Array ConstantInfo)
```

2.24 Physlib.Meta.Notes.Basic[Physlib/Meta/Notes/Basic.lean](#) on GitHub.**structure NoteInfo**[\[source\]](#)*The information from a note ... command. To be used in a note file*

```
structure NoteInfo where
  content : String
  fileName : Name
  line : Nat
```

2.25 Physlib.Meta.Notes.HTMLNote[Physlib/Meta/Notes/HTMLNote.lean](#) on GitHub.**structure HTMLNote**[\[source\]](#)*A HTMLNote is a structure containing the html information from individual contributions (commands, informal commands, note ..) etc. to a note file.*

```
structure HTMLNote where
  fileName : Name
  content : String
  line : Nat
```

def ofNodeInfo[\[source\]](#)*Converts a note info into a HTMLNote.*

```
def HTMLNote.ofNodeInfo (ni : NoteInfo) : HTMLNote
```

def ofFormal[\[source\]](#)*An formal definition or lemma to html for a note.*

```
def HTMLNote.ofFormal (name : Name) : CoreM HTMLNote
```

def ofInformal[\[source\]](#)

An informal definition or lemma to html for a note.

```
unsafe def HTMLNote.ofInformal (name : Name) : CoreM HTMLNote
```

2.26 Physlib.Meta.Notes.NoteFile

[Physlib/Meta/Notes/NoteFile.lean](#) on GitHub.

structure NoteFile[\[source\]](#)

A note consists of a title and a list of Lean files which make up the note.

```
structure NoteFile where
  title : String
  abstract : String
  authors : List String
  files : List Name
```

def imports[\[source\]](#)

All imports associated to a note.

```
def imports : Array Import
```

2.27 Physlib.Meta.Remark.Basic

[Physlib/Meta/Remark/Basic.lean](#) on GitHub.

structure RemarkInfo[\[source\]](#)

The information from a remark ... command. To be used in a note file

```
structure RemarkInfo where
  content : String
  fileName : Name
  line : Nat
  name : Name
  nameSpace : Name
```

2.28 Physlib.Meta.Sorry

[Physlib/Meta/Sorry.lean](#) on GitHub.

structure State[\[source\]](#)

A structure used for collecting names of results dependent on the sorryAx axiom and the Lean.ofReduceBool axiom.

```
structure State where
  visited : NameSet := {}
  containsSorry : NameSet := {}
  containsOfReduceBool : NameSet := {}
```

abbrev M[\[source\]](#)

A monad used for collecting names of results dependent on the `sorryAx` axiom and the `Lean.ofReduceBool` axiom.

```
abbrev M
```

def collect[\[source\]](#)

Given a `c : Name` updating the monad `M` based on which results which `c` uses depend on the `sorryAx` axiom and the `Lean.ofReduceBool` axiom.

```
partial def collect (c : Name) (parents : NameSet) : M Unit
```

def allSorryPseudo[\[source\]](#)

Given a `c : Array Name` updating the monad `M` based on which results depend on the `sorryAx` axiom and the `Lean.ofReduceBool` axiom.

```
partial def allSorryPseudo (c : Array Name) : M Unit
```

def collectSorryPseudo[\[source\]](#)

Given a `c : Array Name` the names of all results used to defined the results in `c` with the `sorryAx` axiom and the `Lean.ofReduceBool` axiom.

```
def collectSorryPseudo (c : Array Name) : CoreM (Array Name × Array Name)
```

def allWithSorryPseudo[\[source\]](#)

The axioms of a constant.

```
def allWithSorryPseudo : CoreM (Array Name × Array Name)
```

def allSorryfulAttributed[\[source\]](#)

All names which are attributed `sorryful`.

```
unsafe def allSorryfulAttributed : CoreM (Array Name)
```

def allPseudoAttributed[\[source\]](#)

All names which are attributed `pseudo`.

```
unsafe def allPseudoAttributed : CoreM (Array Name)
```

def sorryfulPseudoTest[\[source\]](#)

Checks whether all results attributed `sorryful` depend on the ```sorryAx` axiom and vice versa.

```
unsafe def sorryfulPseudoTest : MetaM Unit
```

2.29 Physlib.Meta.TODO.Basic

[Physlib/Meta/TODO/Basic.lean](#) on GitHub.

structure todoInfo

[\[source\]](#)

The information from a TODO ... command.

```
structure todoInfo where
  content : String
  fileName : Name
  line : Nat
  tag : String
```

2.30 Physlib.Particles.SuperSymmetry.SU5.ChargeSpectrum.Basic

[Physlib/Particles/SuperSymmetry/SU5/ChargeSpectrum/Basic.lean](#) on GitHub.

lemma empty_eq

[\[source\]](#)

```
lemma empty_eq : (∅ : ChargeSpectrum  $\mathcal{Z}$ ) = {none, none, {}, {}}
```

2.31 Physlib.QFT.PerturbationTheory.WickAlgebra.Basic

[Physlib/QFT/PerturbationTheory/WickAlgebra/Basic.lean](#) on GitHub.

instance Semiring ($\mathcal{F}$.WickAlgebra)

[\[source\]](#)

```
instance : Semiring ( $\mathcal{F}$ .WickAlgebra)
```

instance Algebra $\mathbb{C}$ ($\mathcal{F}$.WickAlgebra)

[\[source\]](#)

```
instance : Algebra  $\mathbb{C}$  ( $\mathcal{F}$ .WickAlgebra)
```

instance Ring ($\mathcal{F}$.WickAlgebra)

[\[source\]](#)

```
instance: Ring ( $\mathcal{F}$ .WickAlgebra)
```

instance Coe ($\mathcal{F}$.FieldOpFreeAlgebra) ($\mathcal{F}$.WickAlgebra)

[\[source\]](#)

```
instance : Coe ( $\mathcal{F}$ .FieldOpFreeAlgebra) ( $\mathcal{F}$ .WickAlgebra)
```

lemma ofCrAnList_nil

[\[source\]](#)

```
lemma ofCrAnList_nil : ofCrAnList ( $\mathcal{F}$  :=  $\mathcal{F}$ ) [] = 1
```

2.32 Physlib.QFT.QED.AnomalyCancellation.Odd.BasisLinear

[Physlib/QFT/QED/AnomalyCancellation/Odd/BasisLinear.lean](#) on GitHub.

lemma oddShiftZero_eq_oddFst[\[source\]](#)

```
lemma oddShiftZero_eq_oddFst : oddShiftZero = oddFst (0 : Fin n.succ)
```

lemma oddShiftFst_castSucc_eq_oddFst_succ[\[source\]](#)

```
lemma oddShiftFst_castSucc_eq_oddFst_succ (j : Fin n) :
  oddShiftFst j.castSucc = oddFst j.succ
```

lemma oddShiftFst_last_eq_oddMid[\[source\]](#)

```
lemma oddShiftFst_last_eq_oddMid : oddShiftFst (Fin.last n) = oddMid
```

lemma oddShiftSnd_eq_oddSnd[\[source\]](#)

```
lemma oddShiftSnd_eq_oddSnd (j : Fin n) : oddShiftSnd j = oddSnd j
```

2.33 Physlib.QuantumMechanics.DDimensions.Hydrogen.LaplaceRungeLenzVector

[Physlib/QuantumMechanics/DDimensions/Hydrogen/LaplaceRungeLenzVector.lean](#) on GitHub.

lemma positionCompMomentumSqr_comm_momentum_add[\[source\]](#)

```
private lemma positionCompMomentumSqr_comm_momentum_add {d : ℕ} (i j : Fin d) :
  {x i ⋅L (p ⋅v p), p j} + {p i, x j ⋅L (p ⋅v p)} = 0
```

lemma positionDotMomentumCompMomentum_comm_momentum_add[\[source\]](#)

```
private lemma positionDotMomentumCompMomentum_comm_momentum_add {d : ℕ} (i j : Fin d) :
  {(x ⋅v p) ⋅L p i, p j} + {p i, (x ⋅v p) ⋅L p j} = 0
```

lemma positionCompMomentumSqr_comm_radiusRegInvCompPosition_add[\[source\]](#)

```
private lemma positionCompMomentumSqr_comm_radiusRegInvCompPosition_add
  {d : ℕ} (ε : ℝ×) (i j : Fin d) : {x i ⋅L (p ⋅v p), r₀ ε (-1) ⋅L x j}
  + {r₀ ε (-1) ⋅L x i, x j ⋅L (p ⋅v p)} = (-2 * I * ħ) • r₀ ε (-1) ⋅L L i j
```

lemma momentum_comm_radiusRegPow_position_symm[\[source\]](#)

```
private lemma momentum_comm_radiusRegPow_position_symm {d : ℕ} (ε : ℝ×) (s : ℝ) (i j : Fin d) :
  {p i, r₀ ε s ⋅L x j} = {p j, r₀ ε s ⋅L x i}
```

lemma positionDotMomentumCompMomentum_comm_radiusRegInvCompPosition_add[\[source\]](#)

```
private lemma positionDotMomentumCompMomentum_comm_radiusRegInvCompPosition_add
  {d : ℕ} (ε : ℝ×) (i j : Fin d) : {(x ⋅v p) ⋅L p i, r₀ ε (-1) ⋅L x j} +
  {r₀ ε (-1) ⋅L x i, (x ⋅v p) ⋅L p j} = (I * ħ * ε.1 ^ 2) • r₀ ε (-3) ⋅L L i j
```

lemma momentum_comm_radiusRegInvCompPosition_add[\[source\]](#)

```
private lemma momentum_comm_radiusRegInvCompPosition_add {d : ℕ} (ε : ℝ×) (i j : Fin d) :
  {p i, r0 ε (-1) ◦L x j} + {r0 ε (-1) ◦L x i, p j} = 0
```

lemma radiusRegInvCompPosition_comm[\[source\]](#)

```
private lemma radiusRegInvCompPosition_comm {d : ℕ} (ε : ℝ×) (i j : Fin d) :
  {r0 ε (-1) ◦L x i, r0 ε (-1) ◦L x j} = 0
```

lemma r_comm_rx[\[source\]](#)

```
private lemma r_comm_rx {d : ℕ} (ε : ℝ×) (i : Fin d) : {r0[d] ε (-1), r0 ε (-1) ◦L x i} = 0
```

lemma r_comm_pL_Lp[\[source\]](#)

```
private lemma r_comm_pL_Lp {d : ℕ} (ε : ℝ×) (i : Fin d) :
  {r0[d] ε (-1), p ·v L i + L i ·v p} =
  -((I * ħ) • r0 ε (-3) ◦L (x ·v L i + L i ·v x))
```

lemma sum_Lpp[\[source\]](#)

```
private lemma sum_Lpp (d : ℕ) : ∑ i : Fin d, ∑ j, L i j ◦L p j ◦L p i = 0
```

lemma sum_ppL[\[source\]](#)

```
private lemma sum_ppL (d : ℕ) : ∑ i : Fin d, ∑ j, p i ◦L p j ◦L L i j = 0
```

2.34 Physlib.QuantumMechanics.DDimensions.Operators.Commutation

[Physlib/QuantumMechanics/DDimensions/Operators/Commutation.lean](#) on GitHub.

lemma angularMomentum_comp_radiusRegPow_commute[\[source\]](#)

```
lemma angularMomentum_comp_radiusRegPow_commute : L i j ◦L r0 ε s = r0 ε s ◦L L i j
```

lemma angularMomentumSqr_comp_radiusRegPow_commute[\[source\]](#)

```
lemma angularMomentumSqr_comp_radiusRegPow_commute : L2 ◦L r0[d] ε s = r0 ε s ◦L L2
```

2.35 Physlib.QuantumMechanics.DDimensions.Operators.Position

[Physlib/QuantumMechanics/DDimensions/Operators/Position.lean](#) on GitHub.

def radiusPowOperator[\[source\]](#)

The radius operator to power s is the linear map from $\mathcal{S}(\text{Space } d, \mathbb{C})$ to $\text{Space } d \rightarrow \mathbb{C}$ that maps ψ to $x \mapsto \|x\|^s \psi(x)$ (which is 'nearly' Schwartz for general s).

```
def radiusPowOperator {d : ℕ} (s : ℝ) :  $\mathcal{S}(\text{Space } d, \mathbb{C}) \rightarrow_l [\mathbb{C}] \text{Space } d \rightarrow \mathbb{C}$  where
```


lemma radiusPowOperator_apply_fun[\[source\]](#)

```
lemma radiusPowOperator_apply_fun {d : ℕ} (s : ℝ) (ψ : S(Space d, ℂ)) :
  r s ψ = fun x ↦ ||x|| ^ s • ψ x
```

lemma radiusPowOperator_apply[\[source\]](#)

```
@[simp]
lemma radiusPowOperator_apply {d : ℕ} (s : ℝ) (ψ : S(Space d, ℂ)) (x : Space d) :
  r s ψ x = ||x|| ^ s • ψ x
```

lemma radiusPowOperator_apply_contDiffAt[\[source\]](#)

$x \mapsto ||x||^s \psi(x)$ is smooth away from $x = 0$.

```
@[fun_prop]
lemma radiusPowOperator_apply_contDiffAt {d : ℕ} (s : ℝ) (n : ℕ) (ψ : S(Space d, ℂ)) {x :
  Space d}
  (hx : x ≠ 0) : ContDiffAt ℝ n (r s ψ) x
```

lemma radiusPowOperator_apply_stronglyMeasurable[\[source\]](#)

$x \mapsto ||x||^s \psi(x)$ is strongly measurable.

```
@[fun_prop]
lemma radiusPowOperator_apply_stronglyMeasurable {d : ℕ} (s : ℝ) (ψ : S(Space d, ℂ)) :
  StronglyMeasurable (r s ψ)
```

lemma radiusPowOperator_apply_memHS[\[source\]](#)

$x \mapsto ||x||^s \psi(x)$ is square-integrable provided s is not too negative.

```
lemma radiusPowOperator_apply_memHS {d : ℕ} (s : ℝ) (h : 0 < d + 2 * s) (ψ : S(Space d, ℂ)) :
  MemHS (r s ψ)
```

abbrev nhdsZeroUnits[\[source\]](#)

Neighborhoods of "0" in the non-zero reals, i.e. those sets containing $(-\varepsilon, 0) \cup (0, \varepsilon) \subseteq \mathbb{R}^\times$ for some $\varepsilon > 0$.

```
abbrev nhdsZeroUnits : Filter ℝ×
```

instance NeBot nhdsZeroUnits[\[source\]](#)

```
instance : NeBot nhdsZeroUnits
```

lemma radiusRegPow_tendsto_radiusPow[\[source\]](#)

$r[\varepsilon, s] \psi$ converges pointwise to $r[s] \psi$ as $\varepsilon \rightarrow 0$ except perhaps at $x = 0$.

```
lemma radiusRegPow_tendsto_radiusPow {d : ℕ} (s : ℝ) (ψ : S(Space d, ℂ)) {x : Space d}
  (hx : x ≠ 0) : Tendsto (fun ε ↦ r ε s ψ x) nhdsZeroUnits (nhds (r s ψ x))
```

lemma radiusRegPow_tendsto_radiusPow'[\[source\]](#)

$\mathbf{r}[\varepsilon, s] \psi$ converges pointwise to $\mathbf{r}[s] \psi$ as $\varepsilon \rightarrow 0$ provided $\mathbf{r}[\varepsilon, s] \psi$ is bounded.

```
lemma radiusRegPow_tendsto_radiusPow' {d : ℕ} (s : ℝ) (ψ : S(Space d, ℂ)) (h : 0 ≤ s ∧ ψ 0 = 0) :
  Tendsto (fun ε ↦ ↑(r0 ε s ψ)) nhdsZeroUnits (nhds (r s ψ))
```

lemma radiusRegPow_ae_tendsto_radiusPow[\[source\]](#)

a.e. version of radiusRegPow_tendsto_radiusPow

```
lemma radiusRegPow_ae_tendsto_radiusPow {d : ℕ} (hd : 0 < d) (s : ℝ) (ψ : S(Space d, ℂ)) :
  ∀m x, Tendsto (fun ε ↦ r0 ε s ψ x) nhdsZeroUnits (nhds (r s ψ x))
```

lemma radiusRegPow_ae_tendsto_iff[\[source\]](#)

```
lemma radiusRegPow_ae_tendsto_iff {d : ℕ} (hd : 0 < d) {s : ℝ} {ψ : S(Space d, ℂ)}
  {φ : Space d → ℂ} : (∀m x, Tendsto (fun ε ↦ r0 ε s ψ x) nhdsZeroUnits (nhds (φ x)))
  ↔ φ =m[volume] r s ψ
```

def positionOperatorSchwartz[\[source\]](#)

The position operators defined on the Schwartz submodule.

```
def positionOperatorSchwartz : schwartzSubmodule d →l[ℂ] schwartzSubmodule d
```

lemma positionOperatorSchwartz_isSymmetric[\[source\]](#)

```
lemma positionOperatorSchwartz_isSymmetric : (positionOperatorSchwartz i).IsSymmetric
```

def positionUnboundedOperator[\[source\]](#)

The symmetric position unbounded operators with domain the Schwartz submodule of the Hilbert space.

```
def positionUnboundedOperator : UnboundedOperator (SpaceDHilbertSpace d) (SpaceDHilbertSpace d)
```

def radiusRegPowOperatorSchwartz[\[source\]](#)

The (regularized) radius operators defined on the Schwartz submodule.

```
def radiusRegPowOperatorSchwartz {d : ℕ} (ε : ℝ*) (s : ℝ) :
  schwartzSubmodule d →l[ℂ] schwartzSubmodule d
```

lemma radiusRegPowOperatorSchwartz_isSymmetric[\[source\]](#)

```
lemma radiusRegPowOperatorSchwartz_isSymmetric {d : ℕ} (ε : ℝ*) (s : ℝ) :
  (radiusRegPowOperatorSchwartz (d := d) ε s).IsSymmetric
```

def radiusRegPowUnboundedOperator[\[source\]](#)

The symmetric (regularized) radius unbounded operators with domain the Schwartz submodule of the Hilbert space.

```
def radiusRegPowUnboundedOperator {d : ℕ} (ε : ℝ+) (s : ℝ) :
  UnboundedOperator (SpaceDHilbertSpace d) (SpaceDHilbertSpace d)
```

2.36 Physlib.QuantumMechanics.DDimensions.SpaceDHilbertSpace.PolyBddSchwartzSubmodule

[Physlib/QuantumMechanics/DDimensions/SpaceDHilbertSpace/PolyBddSchwartzSubmodule.lean](#) on GitHub.

def polyBddSchwartzMap[\[source\]](#)

A function is a bounded Schwartz map if it is both Schwartz and bounded by powers of $\|x\|$.

```
def polyBddSchwartzMap (d : ℕ) (a : ℕ∞) : Submodule ℂ S(Space d, ℂ) where
```

def polyBddSchwartzIncl[\[source\]](#)

The continuous linear map `schwartzIncl` with domain restricted to `polyBddSchwartzMap d a`.

```
def polyBddSchwartzIncl {d : ℕ} {a : ℕ∞} : polyBddSchwartzMap d a → L[ℂ] SpaceDHilbertSpace d
```

abbrev polyBddSchwartzSubmodule[\[source\]](#)

The submodule of `SpaceDHilbertSpace d` corresponding to bounded Schwartz maps.

```
abbrev polyBddSchwartzSubmodule (d : ℕ) (a : ℕ∞) : Submodule ℂ (SpaceDHilbertSpace d)
```

lemma polyBddSchwartzIncl_injective[\[source\]](#)

```
lemma polyBddSchwartzIncl_injective (d : ℕ) (a : ℕ∞) :
  Function.Injective (polyBddSchwartzIncl (d := d) (a := a))
```

def polyBddSchwartzEquiv[\[source\]](#)

The linear equivalence between polynomially-bounded Schwartz maps and the corresponding submodule of the Hilbert space.

```
def polyBddSchwartzEquiv {d : ℕ} {a : ℕ∞} :
  polyBddSchwartzMap d a ≈l[ℂ] polyBddSchwartzSubmodule d a
```

lemma polyBddSchwartzMap_zero_eq_top[\[source\]](#)

```
lemma polyBddSchwartzMap_zero_eq_top (d : ℕ) : polyBddSchwartzMap d 0 = ⊤
```

lemma polyBddSchwartzMap_le_of_ge[\[source\]](#)

```
lemma polyBddSchwartzMap_le_of_ge (d : ℕ) {a b : ℕ∞} (h : a ≤ b) :
  polyBddSchwartzMap d b ≤ polyBddSchwartzMap d a
```

lemma polyBddSchwartzSubmodule_zero_eq[\[source\]](#)

```
lemma polyBddSchwartzSubmodule_zero_eq (d : ℕ) :
  polyBddSchwartzSubmodule d 0 = schwartzSubmodule d
```

lemma polyBddSchwartzSubmodule_le[\[source\]](#)

```
lemma polyBddSchwartzSubmodule_le (d : ℕ) (a : ℕ∞) :
  polyBddSchwartzSubmodule d a ≤ schwartzSubmodule d
```

lemma polyBddSchwartzSubmodule_le_of_ge[\[source\]](#)

```
lemma polyBddSchwartzSubmodule_le_of_ge (d : ℕ) {a b : ℕ∞} (h : a ≤ b) :
  polyBddSchwartzSubmodule d b ≤ polyBddSchwartzSubmodule d a
```

lemma enorm_bump_mul_le_enorm[\[source\]](#)

```
private lemma enorm_bump_mul_le_enorm {ℝ E : Type*} [RCLike ℝ] [NormedAddCommGroup E]
  [NormedSpace ℝ E] [HasContDiffBump E] {c : E} (f : ContDiffBump c) (g : E → ℝ) (x : E) :
  ||f x * g x||E ≤ ||g x||E
```

lemma tendsto_zero_iff_tendsto_zero_lintegral_enorm_sq[\[source\]](#)

```
private lemma tendsto_zero_iff_tendsto_zero_lintegral_enorm_sq
  {d : ℕ} {α : Type*} {l : Filter α} {ψ : α → SpaceDHilbertSpace d} :
  Tendsto ψ l (nhds 0) ↔ Tendsto (fun a ↦ ∫- x : Space d, ||ψ a x||E ^ 2) l (nhds 0)
```

lemma polyBddSchwartzSubmodule_zero_top_dense[\[source\]](#)

```
private lemma polyBddSchwartzSubmodule_zero_top_dense :
  Dense (polyBddSchwartzSubmodule 0 τ : Set (SpaceDHilbertSpace 0))
```

lemma polyBddSchwartzSubmodule_top_dense[\[source\]](#)

```
lemma polyBddSchwartzSubmodule_top_dense (d : ℕ) :
  Dense (polyBddSchwartzSubmodule d τ : Set (SpaceDHilbertSpace d))
```

lemma polyBddSchwartzSubmodule_dense[\[source\]](#)

```
lemma polyBddSchwartzSubmodule_dense (d : ℕ) (a : ℕ∞) :
  Dense (polyBddSchwartzSubmodule d a : Set (SpaceDHilbertSpace d))
```

2.37 Physlib.QuantumMechanics.DDimensions.SpaceDHilbertSpace.SchwartzSubmodule

[Physlib/QuantumMechanics/DDimensions/SpaceDHilbertSpace/SchwartzSubmodule.lean](#) on GitHub.

lemma schwartzEquiv_symm_coe_ae[\[source\]](#)

```
lemma schwartzEquiv_symm_coe_ae : schwartzEquiv.symm ψ = "[volume] ψ
```

lemma schwartzEquiv_apply_coe[\[source\]](#)

```
lemma schwartzEquiv_apply_coe : ↑(schwartzEquiv f) = schwartzIncl f
```

lemma schwartzIncl_ker[\[source\]](#)

```
lemma schwartzIncl_ker : schwartzIncl.ker = (⊥ : Submodule ℂ S(Space d, ℂ))
```

2.38 Physlib.Relativity.SL2C.Basic

[Physlib/Relativity/SL2C/Basic.lean](#) on GitHub.

lemma toSelfAdjointMap_apply[\[source\]](#)

```
lemma toSelfAdjointMap_apply (A : selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ)) :
  toSelfAdjointMap M A = ⟨M.1 * A.1 * Matrix.conjTranspose M, by
    noncomm_ring [selfAdjoint.mem_iff, star_eq_conjTranspose,
      conjTranspose_mul, conjTranspose_conjTranspose,
      (star_eq_conjTranspose A.1).symm.trans $ selfAdjoint.mem_iff.mp A.2]⟩
```

2.39 Physlib.Relativity.Tensors.Basic

[Physlib/Relativity/Tensors/Basic.lean](#) on GitHub.

lemma basis_congr[\[source\]](#)

```
lemma basis_congr {c1 c2 : C} (h : c1 = c2) (x : basisIdx c1)
  (y : basisIdx c2) (hxy : y = basisIdxCongr (by simp [h]) x) :
  S.FD.map (eqToHom (by simp [h])) ((S.basis c1) x) =
  (S.basis c2) y
```

instance FiniteDimensional k (S.Tensor c)[\[source\]](#)

```
instance {k : Type} [Field k] {C G : Type} [Group G]
  {basisIdx : C → Type} [∀ c, Fintype (basisIdx c)] [∀ c, DecidableEq (basisIdx c)]
  (S : TensorSpecies k C G basisIdx)
  {c : Fin n → C} : FiniteDimensional k (S.Tensor c)
```

instance TopologicalSpace (S.Tensor c)[\[source\]](#)

```
noncomputable instance {k : Type} [RCLike k] {C G : Type} [Group G]
  {basisIdx : C → Type} [∀ c, Fintype (basisIdx c)] [∀ c, DecidableEq (basisIdx c)]
  (S : TensorSpecies k C G basisIdx)
  {c : Fin n → C} : TopologicalSpace (S.Tensor c)
```

instance IsTopologicalAddGroup (S.Tensor c)[\[source\]](#)

```
instance {k : Type} [RCLike k] {C G : Type} [Group G]
  {basisIdx : C → Type} [∀ c, Fintype (basisIdx c)] [∀ c, DecidableEq (basisIdx c)]
  (S : TensorSpecies k C G basisIdx)
  {c : Fin n → C} : IsTopologicalAddGroup (S.Tensor c)
```

2.40 Physlib.Relativity.Tensors.ComplexTensor.Basic

[Physlib/Relativity/Tensors/ComplexTensor/Basic.lean](#) on GitHub.

abbrev repDim

[\[source\]](#)

The dimensions of each of the different types of complex Lorentz vector space.

```
abbrev repDim (c : Color) : ℕ
```

lemma basisIdxCongr_eq_cast

[\[source\]](#)

```
lemma basisIdxCongr_eq_cast {c1 c2 : complexLorentzTensor.Color}
  (h : c1 = c2) (i : Fin (repDim c1)) :
  TensorSpecies.basisIdxCongr (basisIdx := fun c => Fin (repDim c)) h i =
  Fin.cast (by simp [h]) i
```

lemma repDim_tau

[\[source\]](#)

```
lemma repDim_tau {c : complexLorentzTensor.Color} :
  repDim (complexLorentzTensor.τ c) = repDim c
```

lemma basis_eq_complexContrBasisFin4

[\[source\]](#)

```
@[simp]
lemma basis_eq_complexContrBasisFin4 :
  complexLorentzTensor.basis Color.up = Lorentz.complexContrBasisFin4
```

lemma basis_eq_complexCoBasisFin4

[\[source\]](#)

```
@[simp]
lemma basis_eq_complexCoBasisFin4 :
  complexLorentzTensor.basis Color.down = Lorentz.complexCoBasisFin4
```

lemma FD_obj_up

[\[source\]](#)

```
@[simp]
lemma FD_obj_up (Λ : SL(2, ℂ)) :
  (complexLorentzTensor.FD.obj { as := Color.up }).p Λ = Lorentz.complexContr.p Λ
```

lemma FD_obj_down

[\[source\]](#)

```
@[simp]
lemma FD_obj_down (Λ : SL(2, ℂ)) :
  (complexLorentzTensor.FD.obj { as := Color.down }).p Λ = Lorentz.complexCo.p Λ
```

lemma repr_p_basis_vector_up

[\[source\]](#)

Component formula for the standard contravariant vector slot Color.up.

For $b, i : \text{Fin } 4$, the i -component of $p \Lambda$ on basis Color.up b equals the corresponding entry of $\text{LorentzGroup.toComplex } (\text{SL2C.toLorentzGroup } \Lambda)$ in $\text{Fin } 1 \oplus \text{Fin } 3$ coordinates.

```

lemma repr_p_basis_vector_up (Λ : SL(2, ℂ)) (b i : Fin 4) :
  ((complexLorentzTensor.basis Color.up).repr
    (((complexLorentzTensor.FD.obj { as := Color.up }).p Λ)
      (complexLorentzTensor.basis Color.up b))) i =
  (LorentzGroup.toComplex (Lorentz.SL2C.toLorentzGroup Λ))
    (finSumFinEquiv.symm i) (finSumFinEquiv.symm b)

```

lemma repr_p_basis_vector_up_of_eq

[\[source\]](#)

Transport version of repr_p_basis_vector_up for a color c_0 that is propositionally Color.up.

```

lemma repr_p_basis_vector_up_of_eq (c₀ : Color) (h : c₀ = Color.up) (Λ : SL(2, ℂ))
  (b i : Fin (repDim c₀)) :
  ((complexLorentzTensor.basis c₀).repr
    (((complexLorentzTensor.FD.obj { as := c₀ }).p Λ)
      (complexLorentzTensor.basis c₀ b))) i =
  (LorentzGroup.toComplex (Lorentz.SL2C.toLorentzGroup Λ))
    (finSumFinEquiv.symm (Fin.cast (by rw [h];) i))
    (finSumFinEquiv.symm (Fin.cast (by rw [h];) b))

```

lemma repr_p_basis_vector_down

[\[source\]](#)

Component formula for the standard covariant vector slot Color.down.

For $b, i : \text{Fin } 4$, the i -component of $p \Lambda$ on basis Color.down b matches the inverse complex Lorentz matrix as in Lorentz.complexCoBasis_p_apply (with transpose indexing on Fin 1 $\otimes$ Fin 3).

```

lemma repr_p_basis_vector_down (Λ : SL(2, ℂ)) (b i : Fin 4) :
  ((complexLorentzTensor.basis Color.down).repr
    (((complexLorentzTensor.FD.obj { as := Color.down }).p Λ)
      (complexLorentzTensor.basis Color.down b))) i =
  (LorentzGroup.toComplex (Lorentz.SL2C.toLorentzGroup Λ))⁻¹
    (finSumFinEquiv.symm b) (finSumFinEquiv.symm i)

```

lemma repr_p_basis_vector_down_of_eq

[\[source\]](#)

Transport version of repr_p_basis_vector_down for a color c_0 that is propositionally Color.down.

```

lemma repr_p_basis_vector_down_of_eq (c₀ : Color) (h : c₀ = Color.down) (Λ : SL(2, ℂ))
  (b i : Fin (repDim c₀)) :
  ((complexLorentzTensor.basis c₀).repr
    (((complexLorentzTensor.FD.obj { as := c₀ }).p Λ)
      (complexLorentzTensor.basis c₀ b))) i =
  (LorentzGroup.toComplex (Lorentz.SL2C.toLorentzGroup Λ))⁻¹
    (finSumFinEquiv.symm (Fin.cast (by rw [h];) b))
    (finSumFinEquiv.symm (Fin.cast (by rw [h];) i))

```

2.41 Physlib.Relativity.Tensors.ComplexTensor.Units.Pre

[Physlib/Relativity/Tensors/ComplexTensor/Units/Pre.lean](#) on GitHub.

lemma contrCoUnitVal_eq_sum_tmul[\[source\]](#)

```
lemma contrCoUnitVal_eq_sum_tmul : contrCoUnitVal =
  ∑ i, complexContrBasis i ⊗ₜ[ℂ] complexCoBasis i
```

lemma coContrUnitVal_eq_sum_tmul[\[source\]](#)

```
lemma coContrUnitVal_eq_sum_tmul : coContrUnitVal =
  ∑ i, complexCoBasis i ⊗ₜ[ℂ] complexContrBasis i
```

2.42 Physlib.Relativity.Tensors.ComplexTensor.Vector.Pre.Basic

[Physlib/Relativity/Tensors/ComplexTensor/Vector/Pre/Basic.lean](#) on GitHub.

lemma complexContrBasisFin4_eq_reindex[\[source\]](#)

```
lemma complexContrBasisFin4_eq_reindex :
  complexContrBasisFin4 = complexContrBasis.reindex finSumFinEquiv
```

lemma complexContrBasis_reindex_apply_eq_fin4[\[source\]](#)

```
lemma complexContrBasis_reindex_apply_eq_fin4 (j : Fin 4) :
  (complexContrBasis.reindex finSumFinEquiv) j = complexContrBasisFin4 j
```

lemma complexCoBasisFin4_eq_reindex[\[source\]](#)

```
lemma complexCoBasisFin4_eq_reindex :
  complexCoBasisFin4 = complexCoBasis.reindex finSumFinEquiv
```

lemma complexCoBasis_reindex_apply_eq_fin4[\[source\]](#)

```
lemma complexCoBasis_reindex_apply_eq_fin4 (j : Fin 4) :
  (complexCoBasis.reindex finSumFinEquiv) j = complexCoBasisFin4 j
```

2.43 Physlib.Relativity.Tensors.Constructors

[Physlib/Relativity/Tensors/Constructors.lean](#) on GitHub.

lemma fromConst_eq[\[source\]](#)

```
lemma fromConst_eq {n : ℕ} {c : Fin n → C}
  (T : ⋈_ (Rep k G) → S.F.obj (OverColor.mk c)) :
  fromConst T = T.hom (1 : k)
```

2.44 Physlib.Relativity.Tensors.RealTensor.Basic

[Physlib/Relativity/Tensors/RealTensor/Basic.lean](#) on GitHub.

lemma basisIdxCongr_apply[\[source\]](#)

```
@[simp]
lemma basisIdxCongr_apply {d : ℕ} {c1 c2 : realLorentzTensor.Color} (h : c1 = c2)
  (i : Fin 1 ⊗ Fin d) :
```



```
TensorSpecies.basisIdxCongr (basisIdx := fun _ => Fin 1 ⊗ Fin d) h i = i
```

lemma basis_eq_contrBasis

[\[source\]](#)

```
lemma basis_eq_contrBasis {d : ℕ} :
  (realLorentzTensor d).basis Color.up = Lorentz.contrBasis (d := d)
```

lemma basis_eq_coBasis

[\[source\]](#)

```
lemma basis_eq_coBasis {d : ℕ} :
  (realLorentzTensor d).basis Color.down = Lorentz.coBasis (d := d)
```

lemma FD_obj_up

[\[source\]](#)

```
lemma FD_obj_up {d : ℕ} :
  (realLorentzTensor d).FD.obj { as := Color.up } = Lorentz.Contr d
```

lemma FD_obj_down

[\[source\]](#)

```
lemma FD_obj_down {d : ℕ} :
  (realLorentzTensor d).FD.obj { as := Color.down } = Lorentz.Co d
```

2.45 Physlib.Relativity.Tensors.RealTensor.ToComplex

[Physlib/Relativity/Tensors/RealTensor/ToComplex.lean](#) on GitHub.

lemma repDim_colorToComplex

[\[source\]](#)

```
lemma repDim_colorToComplex {c : realLorentzTensor.Color} :
  complexLorentzTensor.repDim (colorToComplex c) = 4
```

lemma colorToComplex_match_up

[\[source\]](#)

simp helper: reduce match c j after a case split on c j (avoids dependent rw / Pi.smul_apply).

```
lemma colorToComplex_match_up {n} {c : Fin n → realLorentzTensor.Color} {j}
  (hc : c j = realLorentzTensor.Color.up) :
  (match c j with
   | .up => complexLorentzTensor.Color.up
   | .down => complexLorentzTensor.Color.down)
  = complexLorentzTensor.Color.up
```

lemma colorToComplex_match_down

[\[source\]](#)

```
lemma colorToComplex_match_down {n} {c : Fin n → realLorentzTensor.Color} {j}
  (hc : c j = realLorentzTensor.Color.down) :
  (match c j with
   | .up => complexLorentzTensor.Color.up
   | .down => complexLorentzTensor.Color.down)
  = complexLorentzTensor.Color.down
```

lemma colorToComplex_comp_eq_match[\[source\]](#)

```

lemma colorToComplex_comp_eq_match {n} (c : Fin n → realLorentzTensor.Color) (j : Fin n) :
  (colorToComplex ∘ c) j =
    (match c j with
    | .up => complexLorentzTensor.Color.up
    | .down => complexLorentzTensor.Color.down)

```

lemma toComplex_repr[\[source\]](#)

The representation of toComplex v in the complexified basis equals the real representation coerced to complex.

```

lemma toComplex_repr {n} {c : Fin n → realLorentzTensor.Color}
  (v : ℝT(3, c)) (i : ComponentIdx (S := realLorentzTensor) c) :
  (Tensor.basis (S := complexLorentzTensor) (colorToComplex ∘ c)).repr
    (toComplex v) i.complexify =
  ↑((Tensor.basis (S := realLorentzTensor) c).repr v i)

```

lemma toComplex_equivariant_slot_repr_up[\[source\]](#)

Local lemma for toComplex_equivariant: isolates the heavy matrix step for a contravariant slot.

```

lemma toComplex_equivariant_slot_repr_up {n} {c : Fin n → realLorentzTensor.Color}
  (k : Fin n) (h : c k = realLorentzTensor.Color.up) (Λ : SL(2, ℂ))
  (b : ComponentIdx (S := realLorentzTensor) c)
  (i : ComponentIdx (S := complexLorentzTensor) (colorToComplex ∘ c)) :
  (↑(((realLorentzTensor.basis (c k)).repr
    (((realLorentzTensor.FD.obj { as := c k}).p (Lorentz.SL2C.toLorentzGroup Λ))
    ((realLorentzTensor.basis (c k)) (b k))))
    (ComponentIdx.complexify.symm i k))) =
  (LorentzGroup.toComplex (Lorentz.SL2C.toLorentzGroup Λ)
    (finSumFinEquiv.symm (Fin.cast repDim_colorToComplex (i k)) (b k))

```

lemma toComplex_equivariant_slot_repr_down[\[source\]](#)

Local lemma for toComplex_equivariant: isolates the heavy matrix step for a covariant slot.

```

lemma toComplex_equivariant_slot_repr_down {n} {c : Fin n → realLorentzTensor.Color}
  (k : Fin n) (h : c k = realLorentzTensor.Color.down) (Λ : SL(2, ℂ))
  (b : ComponentIdx (S := realLorentzTensor) c)
  (i : ComponentIdx (S := complexLorentzTensor) (colorToComplex ∘ c)) :
  (↑(((realLorentzTensor.basis (c k)).repr
    (((realLorentzTensor.FD.obj { as := c k}).p (Lorentz.SL2C.toLorentzGroup Λ))
    ((realLorentzTensor.basis (c k)) (b k))))
    (ComponentIdx.complexify.symm i k))) =
  (LorentzGroup.toComplex (Lorentz.SL2C.toLorentzGroup Λ))-1
    (b k)
    (finSumFinEquiv.symm (Fin.cast repDim_colorToComplex (i k))

```

2.46 Physlib.Relativity.Tensors.RealTensor.Vector.Pre.Basic

[Physlib/Relativity/Tensors/RealTensor/Vector/Pre/Basic.lean](#) on GitHub.

lemma contrBasis_repr_apply[\[source\]](#)

```
lemma contrBasis_repr_apply {d : ℕ} (p : Contr d) (i : Fin 1 ⊗ Fin d) :
  (contrBasis d).repr p i = p.val i
```

lemma coBasis_repr_apply[\[source\]](#)

```
lemma coBasis_repr_apply {d : ℕ} (p : Co d) (i : Fin 1 ⊗ Fin d) :
  (coBasis d).repr p i = p.val i
```

2.47 Physlib.Relativity.Tensors.RealTensor.Velocity.Basic

[Physlib/Relativity/Tensors/RealTensor/Velocity/Basic.lean](#) on GitHub.

instance TopologicalSpace (Velocity d)[\[source\]](#)

The instance of a topological space on Velocity d defined as the subspace topology.

```
noncomputable instance : TopologicalSpace (Velocity d)
```

2.48 Physlib.Relativity.Tensors.TensorSpecies.Basic

[Physlib/Relativity/Tensors/TensorSpecies/Basic.lean](#) on GitHub.

def basisIdxCongr[\[source\]](#)

The casting between basisIdx c and basisIdx c1.

```
def basisIdxCongr {c c1 : C} (h : c = c1) :
  basisIdx c ≈ basisIdx c1
```

lemma basisIdxCongr_refl[\[source\]](#)

```
@[simp]
lemma basisIdxCongr_refl (c : C) (i : basisIdx c) :
  basisIdxCongr (Eq.refl c) i = i
```

lemma basisIdxCongr_apply_apply[\[source\]](#)

```
@[simp]
lemma basisIdxCongr_apply_apply {c c1 c2 : C} (h1 : c = c1) (h2 : c1 = c2) (i : basisIdx c) :
  basisIdxCongr h2 (basisIdxCongr h1 i) = basisIdxCongr (by simp [h1, h2]) i
```

lemma map_basis_eq[\[source\]](#)

```
lemma map_basis_eq {c c1 : C} (h : c = c1) (i : basisIdx c) :
  (S.FD.map (Discrete.eqToHom h)).hom (S.basis c i) = S.basis c1 (basisIdxCongr h i)
```

lemma repr_p_basis_FDTransport[\[source\]](#)

Categorical bookkeeping: relate basis.repr of p g on a basis vector before and after identifying slot colors c and c1 via Discrete.eqToHom (basis_congr_repr, Rep.hom_comm_apply, FD_map_basis chained). Not a physical statement; use for component calculations.

```

lemma repr_p_basis_FDTransport {c c₁ : C} (h : c = c₁) (g : G) (i : basisIdx c)
  (b : basisIdx c) :
  (S.basis c).repr (((S.FD.obj { as := c }).p g) (S.basis c b)) i =
  (S.basis c₁).repr
    (((S.FD.obj { as := c₁ }).p g) (S.basis c₁ (basisIdxCongr (by simp [h]) b)))
    (basisIdxCongr (by simp [h]) i))

```

2.49 Physlib.SpaceAndTime.Space.Derivatives.Curl

[Physlib/SpaceAndTime/Space/Derivatives/Curl.lean](#) on GitHub.

def homotopyOperatorIntegrand

[\[source\]](#)

The homotopy operator is defined as $\int t \text{ in } 0..1, (t \cdot x) \times f(t \cdot x)$. This is the integrand of that function.

```

private noncomputable def homotopyOperatorIntegrand (f : Space → EuclideanSpace ℝ (Fin 3)) :
  Space → ℝ → EuclideanSpace ℝ (Fin 3)

```

lemma homotopyOperatorIntegrand_eq

[\[source\]](#)

```

private lemma homotopyOperatorIntegrand_eq (f : Space → EuclideanSpace ℝ (Fin 3)) :
  homotopyOperatorIntegrand f = fun x t => (t • basis.repr x) ×e3 f (t • x)

```

lemma differentiable_homotopyOperatorIntegrand_space

[\[source\]](#)

```

@[fun_prop]
private lemma differentiable_homotopyOperatorIntegrand_space {f : Space → EuclideanSpace ℝ (Fin 3)}
  (hf : Differentiable ℝ f) (t : ℝ) :
  Differentiable ℝ (homotopyOperatorIntegrand f • t)

```

lemma homotopyOperatorIntegrand_continuous_param

[\[source\]](#)

```

@[fun_prop]
private lemma homotopyOperatorIntegrand_continuous_param {f : Space → EuclideanSpace ℝ (Fin 3)}
  (hf : Differentiable ℝ f) (x : Space) : Continuous (homotopyOperatorIntegrand f x •)

```

lemma intervalIntegrable_homotopyOperatorIntegrand

[\[source\]](#)

```

private lemma intervalIntegrable_homotopyOperatorIntegrand {f : Space → EuclideanSpace ℝ (Fin 3)}
  (hf : Differentiable ℝ f) (x : Space) :
  IntervalIntegrable (homotopyOperatorIntegrand f x •) volume (0 : ℝ) 1

```

lemma fderiv_homotopyOperatorIntegrand_eq_fderiv_crossProduct

[\[source\]](#)

```

private lemma fderiv_homotopyOperatorIntegrand_eq_fderiv_crossProduct
  {f : Space → EuclideanSpace ℝ (Fin 3)}
  (hf : Differentiable ℝ f) (x : Space) (t : ℝ) (y : Space) (i : Fin 3) :
  fderiv ℝ (homotopyOperatorIntegrand f • t) x y i =
  t • (fderiv ℝ (fun x => (WithLp.toLp 2 ((crossProduct (basis.repr x).ofLp)
    (f (t • x)).ofLp)).ofLp i) x) y

```

lemma fderiv_homotopyOperatorIntegrand_apply_eq[\[source\]](#)

```
private lemma fderiv_homotopyOperatorIntegrand_apply_eq {f : Space → EuclideanSpace ℝ (Fin 3)}
  (hf : Differentiable ℝ f) (x : Space) (t : ℝ) (y : Space) (i : Fin 3) :
  fderiv ℝ (homotopyOperatorIntegrand f · t) x y i =
    t * (x (i+1) * t * fderiv ℝ f (t • x) y (i-1) + f (t • x) (i-1) * y (i+1) -
      (x (i-1) * t * fderiv ℝ f (t • x) y (i+1) + f (t • x) (i+1) * y (i-1)))
```

lemma continuous_uncurry_fderiv_homotopyOperatorIntegrand[\[source\]](#)

```
private lemma continuous_uncurry_fderiv_homotopyOperatorIntegrand
  {f : Space → EuclideanSpace ℝ (Fin 3)} (hf : ContDiff ℝ 1 f) :
  Continuous (Function.uncurry (fun x t => fderiv ℝ (homotopyOperatorIntegrand f · t) x))
```

lemma hasFDerivAt_intervalIntegral_homotopyOperatorIntegrand[\[source\]](#)

```
private lemma hasFDerivAt_intervalIntegral_homotopyOperatorIntegrand
  {f : Space → EuclideanSpace ℝ (Fin 3)} (hf : ContDiff ℝ 1 f) (x₀ : Space) :
  HasFDerivAt (fun (x : Space) => ∫ (t : ℝ) in 0..1, homotopyOperatorIntegrand f x t ∂(volume))
    (∫ (t : ℝ) in 0..1, fderiv ℝ (homotopyOperatorIntegrand f · t) x₀ ∂(volume)) x₀
```

lemma deriv_intervalIntegral_homotopyOperatorIntegrand_sub[\[source\]](#)

```
private lemma deriv_intervalIntegral_homotopyOperatorIntegrand_sub
  (f : Space → EuclideanSpace ℝ (Fin 3)) (hf : ContDiff ℝ 1 f) (i j k m : Fin 3) (x₀ : Space) :
  ∂[i] (fun x => (∫ (t : ℝ) in 0..1, homotopyOperatorIntegrand f x t ∂(volume)) k) x₀ -
  ∂[j] (fun x => (∫ (t : ℝ) in 0..1, homotopyOperatorIntegrand f x t ∂(volume)) m) x₀ =
  ∫ (t : ℝ) in 0..1, (fderiv ℝ (homotopyOperatorIntegrand f · t) x₀ (basis i) k -
    fderiv ℝ (homotopyOperatorIntegrand f · t) x₀ (basis j) m) ∂(volume)
```

lemma exists_curl_of_div_zero[\[source\]](#)

```
lemma exists_curl_of_div_zero (f : Space → EuclideanSpace ℝ (Fin 3)) (hf : ContDiff ℝ 1 f)
  (hdiv : ∇ · f = 0) :
  ∃ g : Space → EuclideanSpace ℝ (Fin 3), f = curl g ∧ Differentiable ℝ g
```

lemma deriv_comm_of_curl_zero[\[source\]](#)

```
lemma deriv_comm_of_curl_zero (f : Space → EuclideanSpace ℝ (Fin 3)) (hf : Differentiable ℝ f)
  (hcurl : curl f = 0) (x : Space) (i j : Fin 3) :
  ∂[i] f x j = ∂[j] f x i
```

lemma exists_grad_of_curl_zero[\[source\]](#)

```
lemma exists_grad_of_curl_zero (f : Space → EuclideanSpace ℝ (Fin 3)) (hf : Differentiable ℝ f)
  (hcurl : curl f = 0) : ∃ g : Space → ℝ, f = grad g ∧ Differentiable ℝ g
```

2.50 Physlib.SpaceAndTime.Space.Derivatives.Div

[Physlib/SpaceAndTime/Space/Derivatives/Div.lean](#) on GitHub.

lemma div_eq_sum_fderiv[\[source\]](#)

```
lemma div_eq_sum_fderiv {d} (f : Space d → EuclideanSpace ℝ (Fin d))
  (hf : Differentiable ℝ f) (x : Space d) :
  (∇ · f) x = ∑ i, fderiv ℝ f x (basis i) i
```

2.51 Physlib.SpaceAndTime.Space.Derivatives.Grad

[Physlib/SpaceAndTime/Space/Derivatives/Grad.lean](#) on GitHub.

lemma gradient_apply_eq_grad[\[source\]](#)

```
lemma gradient_apply_eq_grad {d} (f : Space d → ℝ) (x : Space d) :
  gradient f x = basis.repr.symm (∇ f x)
```

lemma hasGradientAt_grad[\[source\]](#)

```
lemma _root_.DifferentiableAt.hasGradientAt_grad {d} {f : Space d → ℝ} (x : Space d)
  (hf : DifferentiableAt ℝ f x) :
  HasGradientAt f (basis.repr.symm (∇ f x)) x
```

2.52 Physlib.SpaceAndTime.Space.Derivatives.Iterated

[Physlib/SpaceAndTime/Space/Derivatives/Iterated.lean](#) on GitHub.

def iteratedDeriv[\[source\]](#)

The iterated coordinate derivative on Space d indexed by a multi-index.

```
noncomputable def iteratedDeriv [AddCommGroup M] [Module ℝ M] [TopologicalSpace M]
  (I : MultiIndex d) (f : Space d → M) : Space d → M
```

lemma iteratedDerivList_contDiff[\[source\]](#)

```
private lemma iteratedDerivList_contDiff (L : List (Fin d)) {f : Space d → ℝ}
  (hf : ContDiff ℝ ∞ f) :
  ContDiff ℝ ∞ (L.foldr (fun i g => deriv i g) f)
```

lemma iteratedDerivList_add[\[source\]](#)

```
private lemma iteratedDerivList_add (L : List (Fin d)) {f g : Space d → ℝ}
  (hf : ContDiff ℝ ∞ f) (hg : ContDiff ℝ ∞ g) :
  L.foldr (fun i h => deriv i h) (f + g) =
  L.foldr (fun i h => deriv i h) f + L.foldr (fun i h => deriv i h) g
```

lemma iteratedDerivList_const_smul[\[source\]](#)

```
private lemma iteratedDerivList_const_smul (L : List (Fin d)) (c : ℝ) {f : Space d → ℝ}
  (hf : ContDiff ℝ ∞ f) :
  L.foldr (fun i h => deriv i h) (c • f) =
  c • L.foldr (fun i h => deriv i h) f
```

lemma iteratedDeriv_zero[\[source\]](#)

```
@[simp]
lemma iteratedDeriv_zero [AddCommGroup M] [Module ℝ M] [TopologicalSpace M]
  (f : Space d → M) : ∂^[0] f = f
```

lemma iteratedDeriv_increment_zero[\[source\]](#)

```
@[simp]
lemma iteratedDeriv_increment_zero [AddCommGroup M] [Module ℝ M] [TopologicalSpace M]
  (I : MultiIndex d.succ) (f : Space d.succ → M) :
  ∂^[MultiIndex.increment I 0] f = ∂[0] (∂^[I] f)
```

lemma iteratedDeriv_single[\[source\]](#)

```
@[simp]
lemma iteratedDeriv_single [AddCommGroup M] [Module ℝ M] [TopologicalSpace M]
  (i : Fin d) (f : Space d → M) :
  ∂^[MultiIndex.increment 0 i] f = ∂[i] f
```

lemma iteratedDeriv_add[\[source\]](#)

```
lemma iteratedDeriv_add (I : MultiIndex d) {f g : Space d → ℝ}
  (hf : ContDiff ℝ ∞ f) (hg : ContDiff ℝ ∞ g) :
  ∂^[I] (f + g) = ∂^[I] f + ∂^[I] g
```

lemma iteratedDeriv_const_smul[\[source\]](#)

```
lemma iteratedDeriv_const_smul (I : MultiIndex d) (c : ℝ) {f : Space d → ℝ}
  (hf : ContDiff ℝ ∞ f) :
  ∂^[I] (c • f) = c • ∂^[I] f
```

lemma iteratedDeriv_contDiff[\[source\]](#)

Iterated spatial derivatives preserve smoothness for scalar-valued functions.

```
lemma iteratedDeriv_contDiff (I : MultiIndex d) {f : Space d → ℝ}
  (hf : ContDiff ℝ ∞ f) :
  ContDiff ℝ ∞ (∂^[I] f)
```

lemma tsupport_deriv_subset[\[source\]](#)

The topological support of a spatial derivative is contained in that of the original function.

```
lemma tsupport_deriv_subset (i : Fin d) {f : Space d → ℝ} :
  tsupport (deriv i f) ⊆ tsupport f
```

lemma iteratedDerivList_commute_deriv[\[source\]](#)

```
private lemma iteratedDerivList_commute_deriv (L : List (Fin d)) (i : Fin d)
  {f : Space d → ℝ} (hf : ContDiff ℝ ∞ f) :
  L.foldr (fun j g => deriv j g) (deriv i f) =
  deriv i (L.foldr (fun j g => deriv j g) f)
```

lemma deriv_iteratedDeriv_commute[\[source\]](#)

An extra spatial derivative commutes with iterated spatial derivatives for smooth scalar-valued functions.

```
lemma deriv_iteratedDeriv_commute (i : Fin d) (I : MultiIndex d) {f : Space d → ℝ}
  (hf : ContDiff ℝ ∞ f) :
  deriv i (∂^[I] f) = ∂^[I] (deriv i f)
```

lemma tsupport_iteratedDerivList_subset[\[source\]](#)

```
private lemma tsupport_iteratedDerivList_subset (L : List (Fin d)) {f : Space d → ℝ} :
  tsupport (L.foldr (fun i g => deriv i g) f) ⊆ tsupport f
```

lemma tsupport_iteratedDeriv_subset[\[source\]](#)

The topological support of an iterated spatial derivative is contained in that of the original function.

```
lemma tsupport_iteratedDeriv_subset (I : MultiIndex d) {f : Space d → ℝ} :
  tsupport (∂^[I] f) ⊆ tsupport f
```

lemma iteratedDeriv_eq_zero_of_notMem_tsupport[\[source\]](#)

An iterated spatial derivative vanishes outside the topological support of the original function.

```
lemma iteratedDeriv_eq_zero_of_notMem_tsupport (I : MultiIndex d) {f : Space d → ℝ} {x : Space
  d}
  (hx : x ∉ tsupport f) :
  ∂^[I] f x = 0
```

2.53 Physlib.SpaceAndTime.Space.Derivatives.MultiIndex

[Physlib/SpaceAndTime/Space/Derivatives/MultiIndex.lean](#) on GitHub.

structure MultiIndex[\[source\]](#)

A multi-index on d source coordinates.

```
structure MultiIndex (d : ℕ) where
  toFun : Fin d → ℕ
```

instance CoeFun (MultiIndex d) (fun _ => Fin d → ℕ)[\[source\]](#)

```
instance : CoeFun (MultiIndex d) (fun _ => Fin d → ℕ)
```

instance Zero (MultiIndex d)[\[source\]](#)

```
instance : Zero (MultiIndex d)
```


instance Add (MultiIndex d)

[\[source\]](#)

```
instance : Add (MultiIndex d)
```

def order

[\[source\]](#)

The order $|I|$ of a multi-index I , defined as the sum of its components.

```
def order (I : MultiIndex d) : Nat
```

def increment

[\[source\]](#)

Increment the i -th coordinate of a multi-index by one.

```
def increment (I : MultiIndex d) (i : Fin d) : MultiIndex d
```

lemma ext

[\[source\]](#)

```
@[ext]
```

```
lemma ext {I J : MultiIndex d} (h : ∀ i, I i = J i) : I = J
```

lemma zero_apply

[\[source\]](#)

```
@[simp]
```

```
lemma zero_apply (i : Fin d) : (0 : MultiIndex d) i = 0
```

lemma add_apply

[\[source\]](#)

```
@[simp]
```

```
lemma add_apply (I J : MultiIndex d) (i : Fin d) : (I + J) i = I i + J i
```

lemma increment_apply_same

[\[source\]](#)

```
@[simp]
```

```
lemma increment_apply_same (I : MultiIndex d) (i : Fin d) :  
  increment I i i = I i + 1
```

lemma increment_apply_ne

[\[source\]](#)

```
@[simp]
```

```
lemma increment_apply_ne (I : MultiIndex d) {i j : Fin d} (h : j ≠ i) :  
  increment I i j = I j
```

lemma order_zero

[\[source\]](#)

```
@[simp]
```

```
lemma order_zero : order (0 : MultiIndex d) = 0
```

lemma order_add

[\[source\]](#)

```
lemma order_add (I J : MultiIndex d) : order (I + J) = order I + order J
```

lemma order_single[\[source\]](#)

```
@[simp]
lemma order_single (i : Fin d) : order ((Pi.single i 1) : MultiIndex d) = 1
```

lemma order_increment[\[source\]](#)

```
@[simp]
lemma order_increment (I : MultiIndex d) (i : Fin d) :
  order (increment I i) = order I + 1
```

def tail[\[source\]](#)

The tail of a multi-index on $d + 1$ coordinates, dropping the θ -th coordinate.

```
def tail (I : MultiIndex d.succ) : MultiIndex d
```

def toList[\[source\]](#)

The canonical ordered list of coordinate directions encoded by a multi-index.

```
def toList : {d : ℕ} → MultiIndex d → List (Fin d)
| 0, _ => []
| _ + 1, I => List.replicate (I 0) 0 ++ (toList (tail I)).map Fin.succ
```

lemma tail_zero[\[source\]](#)

```
@[simp]
lemma tail_zero : tail (0 : MultiIndex d.succ) = 0
```

lemma tail_increment_zero[\[source\]](#)

```
@[simp]
lemma tail_increment_zero (I : MultiIndex d.succ) : tail (increment I 0) = tail I
```

lemma tail_increment_succ[\[source\]](#)

```
@[simp]
lemma tail_increment_succ (I : MultiIndex d.succ) (i : Fin d) :
  tail (increment I i.succ) = increment (tail I) i
```

lemma toList_zero[\[source\]](#)

```
@[simp]
lemma toList_zero : toList (0 : MultiIndex d) = []
```

lemma length_toList[\[source\]](#)

```
lemma length_toList (I : MultiIndex d) : I.toList.length = I.order
```

lemma toList_increment_zero[\[source\]](#)

```
@[simp]
lemma toList_increment_zero (I : MultiIndex d.succ) :
  toList (increment I 0) = 0 :: toList I
```

lemma toList_single[\[source\]](#)

```
@[simp]
lemma toList_single (i : Fin d) : toList (increment 0 i : MultiIndex d) = [i]
```

2.54 Physlib.SpaceAndTime.Space.Module

[Physlib/SpaceAndTime/Space/Module.lean](#) on GitHub.

instance MeasurableSpace (Space d)[\[source\]](#)

```
noncomputable instance {d : ℕ} : MeasurableSpace (Space d)
```

lemma basis_induction_on[\[source\]](#)

```
lemma basis_induction_on {d} {P : Space d → Prop}
  (hb : ∀ i, P (basis i)) (hzero : P 0)
  (hadd : ∀ p1 p2, P p1 → P p2 → P (p1 + p2))
  (hsmul : ∀ (c : ℝ) p, P p → P (c • p)) (p : Space d) : P p
```

lemma fderiv_eval_apply[\[source\]](#)

```
@[simp]
lemma fderiv_eval_apply {d : ℕ} (p y : Space d) (i : Fin d) :
  fderiv ℝ (fun p => p.val i) p y = y i
```

2.55 Physlib.SpaceAndTime.SpaceTime.Derivatives

[Physlib/SpaceAndTime/SpaceTime/Derivatives.lean](#) on GitHub.

lemma tensorDeriv_eq_sum_tensor_basis[\[source\]](#)

The expansion of tensorDeriv in terms of the tensor basis vector.

```
lemma tensorDeriv_eq_sum_tensor_basis
  {f : SpaceTime d → M} (hf : Differentiable ℝ f) (x : SpaceTime d) :
  tensorDeriv f x = ∑ b, ∂_ (CoVector.indexEquiv (ComponentIdx.prod b).1)
    (fun x => (Tensor.basis _).repr (toTensor (f x)) (ComponentIdx.prod b).2) x •
  toTensor.symm (Tensor.basis _ b)
```

2.56 Physlib.SpaceAndTime.Time.Basic

[Physlib/SpaceAndTime/Time/Basic.lean](#) on GitHub.

lemma norm_eq_val[\[source\]](#)

```
lemma norm_eq_val (t : Time) :
  ||t|| = ||t.val||
```

2.57 Physlib.SpaceAndTime.Time.Derivatives

[Physlib/SpaceAndTime/Time/Derivatives.lean](#) on GitHub.

lemma deriv_div

[\[source\]](#)

*Quotient rule for Time.deriv on real-valued functions: if c and g are differentiable at t and $g\ t \neq 0$, then $\partial_t (c / g)\ t = (\partial_t c\ t * g\ t - c\ t * \partial_t g\ t) / (g\ t)^2$.*

```
lemma deriv_div {c g : Time → ℝ}
  (hc : DifferentiableAt ℝ c t) (hg : DifferentiableAt ℝ g t) (hg2 : g t ≠ 0) :
  ∂t (fun s => c s / g s) t =
    (∂t c t * g t - c t * ∂t g t) / (g t) ^ 2
```

2.58 QuantumInfo.Finite.Braket

[QuantumInfo/Finite/Braket.lean](#) on GitHub.

lemma Ket.coe_fun_eq

[\[source\]](#)

```
lemma _root_.Ket.coe_fun_eq (ψ : Ket d) : (ψ : d → ℂ) = ψ.vec
```

lemma Bra.coe_fun_eq

[\[source\]](#)

```
lemma _root_.Bra.coe_fun_eq (ψ : Bra d) : (ψ : d → ℂ) = ψ.vec
```

2.59 QuantumInfo.Finite.CPTPMap.Bundled

[QuantumInfo/Finite/CPTPMap/Bundled.lean](#) on GitHub.

lemma HPMMap.apply_hermitianMat_eq

[\[source\]](#)

```
lemma apply_hermitianMat_eq (Λ : HPMMap dIn dOut ℂ) (p : HermitianMat dIn ℂ) :
  Λ p = (Λ.map p.1, Λ.HP p.2)
```

lemma PMap.apply_hermitianMat_eq

[\[source\]](#)

```
lemma apply_hermitianMat_eq (Λ : PMap dIn dOut ℂ) (p : HermitianMat dIn ℂ) :
  Λ p = (Λ.map p.1, Λ.HP p.2)
```

lemma PTPMap.apply_hermitianMat_eq_toPMap

[\[source\]](#)

```
lemma apply_hermitianMat_eq_toPMap (Λ : PTPMap dIn dOut ℂ) (p : HermitianMat dIn ℂ) :
  Λ p = Λ.toPMap p
```

lemma apply_mstate_eq

[\[source\]](#)

```
lemma apply_mstate_eq [DecidableEq dIn] [DecidableEq dOut] (Λ : PTPMap dIn dOut ℂ) (p : MState
  dIn) :
  Λ p = MState.mk
    (Λ.toHPMap p.M) (HermitianMat.zero_le_iff.mpr (Λ.pos p.psd)) (by
      rw [HermitianMat.trace_eq_one_iff, ← p.tr']
      exact Λ.TP p)
```

lemma apply_mState_eq_toPTPMap[\[source\]](#)

```
lemma apply_mState_eq_toPTPMap [DecidableEq dOut] (Λ : CPTPMap dIn dOut) (p : MState dIn) :
  Λ p = Λ.toPTPMap p
```

lemma PUMap.apply_hermitianMat_eq_toPMap[\[source\]](#)

```
lemma apply_hermitianMat_eq_toPMap (Λ : PUMap dIn dOut ℂ) (p : HermitianMat dIn ℂ) :
  Λ p = Λ.toPMap p
```

lemma CUPMap.apply_hermitianMat_eq_toPMap[\[source\]](#)

```
lemma apply_hermitianMat_eq_toPMap (Λ : CUPMap dIn dOut ℂ) (p : HermitianMat dIn ℂ) :
  Λ p = Λ.toPMap p
```

2.60 QuantumInfo.Finite.MState

[QuantumInfo/Finite/MState.lean](#) on GitHub.

lemma default_eq[\[source\]](#)

```
lemma default_eq [Nonempty d] : (default : MState d) = uniform
```

2.61 scripts.check_file_imports

[scripts/check_file_imports.lean](#) on GitHub.

def expectedPhyslibImports[\[source\]](#)

Compute imports expected by Physlib.lean by looking at file structure.

```
def expectedPhyslibImports : IO (Array Name)
```

2.62 scripts.find_TODOs

[scripts/find_TODOs.lean](#) on GitHub.

def PhyslibTODOItem[\[source\]](#)

Given a list of lines, outputs an error message and a line number.

```
def PhyslibTODOItem : Type
```

def physlibLintFile[\[source\]](#)

```
def physlibLintFile (path : FilePath) : IO String
```

2.63 scripts.mathlib_textLint_on_physlib

[scripts/mathlib_textLint_on_physlib.lean](#) on GitHub.

def physlib_lint_style[\[source\]](#)*Setting up command line options and help text for lake exe lint_style.*

```
def physlib_lint_style : Cmd
```

2.64 scripts.MetaPrograms.regex_lint[scripts/MetaPrograms/regex_lint.lean](#) on GitHub.**def PhyslibTextLinter**[\[source\]](#)*Given a list of lines, outputs an error message and a line number.*

```
def PhyslibTextLinter : Type
```

structure PhyslibErrorContext[\[source\]](#)

```
structure PhyslibErrorContext where
  error : String
  lineNumber : ℕ
  columnNumber : ℕ
  path : FilePath
```

def physlibLintFile[\[source\]](#)

```
def physlibLintFile (path : FilePath) : IO (Array PhyslibErrorContext)
```

2.65 scripts.MetaPrograms.style_lint[scripts/MetaPrograms/style_lint.lean](#) on GitHub.**def PhyslibTextLinter**[\[source\]](#)*Given a list of lines, outputs an error message and a line number.*

```
def PhyslibTextLinter : Type
```

structure PhyslibErrorContext[\[source\]](#)

```
structure PhyslibErrorContext where
  error : String
  lineNumber : ℕ
  columnNumber : ℕ
  path : FilePath
```

def physlibLintFile[\[source\]](#)

```
def physlibLintFile (path : FilePath) : IO (Array PhyslibErrorContext)
```

2.66 scripts.MetaPrograms.TODO_to_yaml[scripts/MetaPrograms/TOD0_to_yaml.lean](#) on GitHub.

inductive PhyslibCategory[\[source\]](#)

```

inductive PhyslibCategory
| ClassicalMechanics
| CondensedMatter
| Cosmology
| Elctromagnetism
| Mathematics
| Meta
| Optics
| Particles
| QFT
| QuantumMechanics
| Relativity
| StringTheory
| StatisticalMechanics
| Thermodynamics
| Other

```

def string[\[source\]](#)

```

def PhyslibCategory.string : PhyslibCategory → String
| ClassicalMechanics => "Classical Mechanics"
| CondensedMatter => "Condensed Matter"
| Cosmology => "Cosmology"
| Elctromagnetism => "Electromagnetism"
| Mathematics => "Mathematics"
| Meta => "Meta"
| Optics => "Optics"
| Particles => "Particles"
| QFT => "QFT"
| QuantumMechanics => "Quantum Mechanics"
| Relativity => "Relativity"
| StringTheory => "String Theory"
| StatisticalMechanics => "Statistical Mechanics"
| Thermodynamics => "Thermodynamics"
| Other => "Other"

```

def emoji[\[source\]](#)

```

def PhyslibCategory.emoji : PhyslibCategory → String
| ClassicalMechanics => "🌀"
| CondensedMatter => ""
| Cosmology => ""
| Elctromagnetism => ">"
| Mathematics => ""
| Meta => ""
| Optics => ""
| Particles => ""
| QFT => ""
| QuantumMechanics => "⚗️"
| Relativity => ""
| StringTheory => ""
| StatisticalMechanics => ""
| Thermodynamics => ""
| Other => ""

```

def List[\[source\]](#)

```
def PhyslibCategory.List : List PhyslibCategory
```

instance ToString PhyslibCategory[\[source\]](#)

```
instance : ToString PhyslibCategory where
```

def ofFileName[\[source\]](#)

```
def PhyslibCategory.ofFileName (n : Name) : PhyslibCategory
```

2.67 scripts.type_former_lint

[scripts/type_former_lint.lean](#) on GitHub.

def IsUpperCamel[\[source\]](#)

A rough definition of Upper Camel, checking that if a string starts with a latin letter then that letter is capital.

```
def IsUpperCamel (s : String) : Bool
```